User Manual

for S32 FLS Driver

Document Number: UM11FLSASR4.4 Rev0000R4.0.2 Rev. 1.0

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.06.2023	NXP RTD Team	S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release

Chapter 2

Introduction

This User Manual describes NXP Semiconductors' AUTOSAR Flash Driver for S32.

AUTOSAR Flash Driver configuration parameters description can be found in the Tresos Configuration Plug-in section. Deviations from the specification are described in the Deviations from Requirements section.

AUTOSAR Flash driver requirements and APIs are described in the Flash Driver Software Specification Document (version 4.4.0) [1] and in the Modules Documentation section.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525

- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525
- s32r45_bga780

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
DET	Default Error Tracer
ECC	Error Correcting Code
VLE	Variable Length Encoding
N/A	Not Available
MCU	Microcontroller Unit
ECU	Electronic Control Unit
EEPROM	Electrically Erasable Programmable Read-Only Memory
FEE	Flash EEPROM Emulation
FLS	Flash
RTD	Real Time Drivers
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of FLS Driver	AUTOSAR Release 4.4.0
2	Reference Manual	S32G2 Reference Manual, Rev. 7, February 2023
		S32G3 Reference Manual, Rev. 3, February 2023
		S32R45 Reference Manual, Rev. 3, 12/2021
3	Errata	S32G2: Mask Set Errata for Mask 0P77B v3.0
		S32G2: Mask Set Errata for Mask 1P77B v1.0
		S32G3: Mask Set Errata for Mask 1P72B v2.1
		S32R45: Mask Set Errata for Mask P57D, Rev. 2.0
4	Data Sheet	S32G2 Data Sheet, Rev 6, December 2022
		S32G3 Data Sheet, Rev 2, RC, February 2023
		VR5510 Data Sheet, Rev 5, April 2022
		S32R45 Data Sheet, Rev. 3, RC — 11/2022

Chapter 3

Driver

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See Table Reference List).

3.2 Driver Design Summary

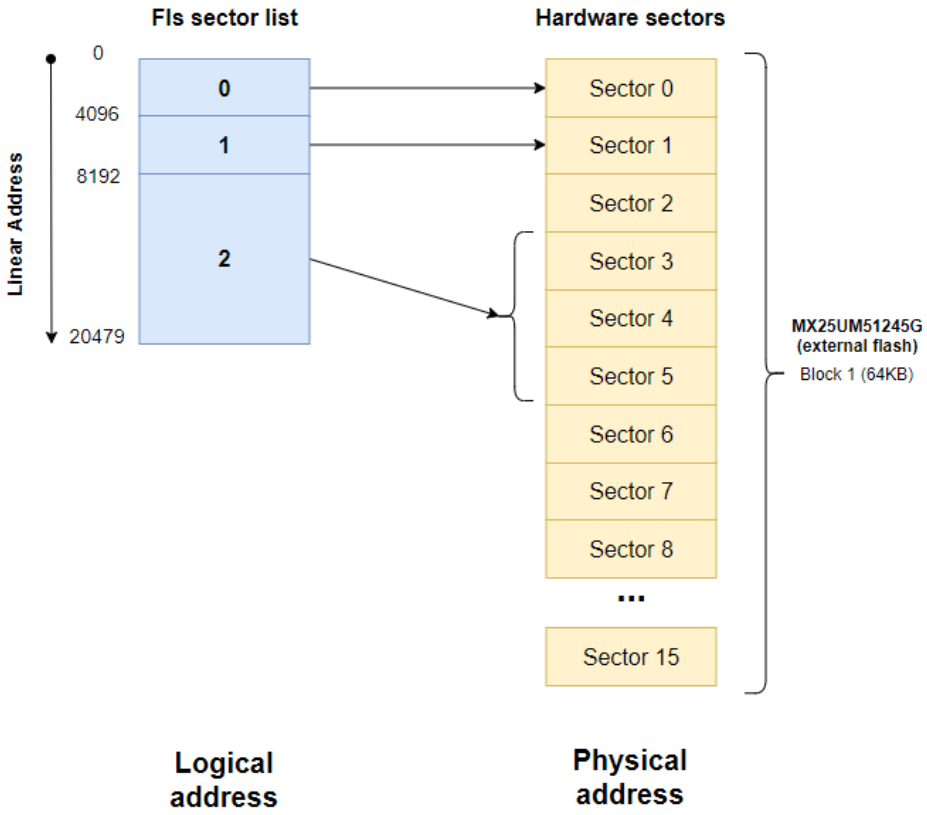
3.2.1 Linear Address

The FLS driver provides services for reading, writing and erasing flash memory and it combines configured flash memory sectors into one linear address space. The FLS module shall combine all available flash memory areas into one linear address space, it will always start at address 0 and continues without any gap.

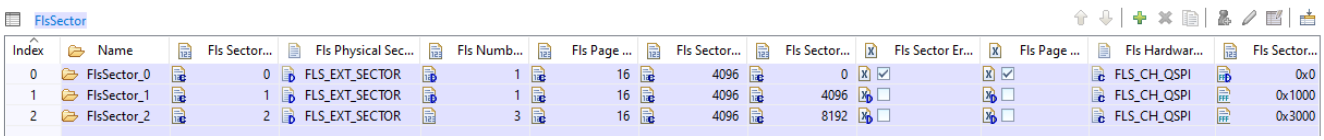
- Sectors details Example: Suppose users want to configure following sectors

FlsSectorList	Fls Sector Size	Fls Number Of Sector	Fls Logical Start Address	Fls Hardware Start Address
FlsSector_0	4096 (0x1000)	1	0 (0x0000)	0 (0x0000)
FlsSector_1	4096 (0x1000)	1	4096 (0x1000)	4096 (0x1000)
FlsSector_2	4096 (0x1000)	3	8192 (0x2000)	12288 (0x3000)

The layout of configured sectors:



The FlsSector List should be configured in the following way:



As you can see:

- Fls Sector Start Address for FlsSector_0 will be 0x0000 and Fls Sector Start Address for FlsSector_1 will be 0x1000 (4096)
- If users want to write FlsSector_1, users need to write to the logical address 0x1000 - 0x1FFF
- If users want to erase it, users need to erase sector from address 0x1000 with size 0x1000

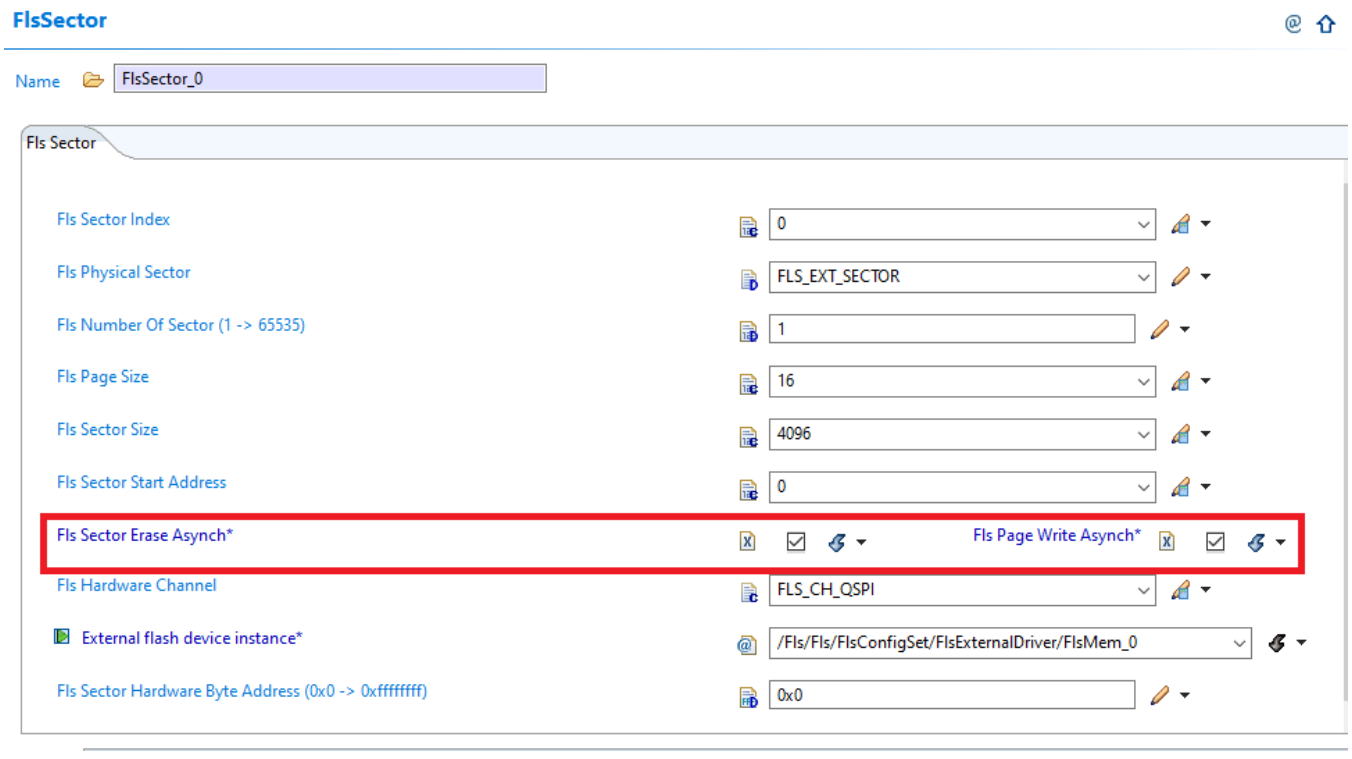
Note

Users do not need to calculate the Fls Sector Start Address, it will be automatically computed

3.2.2 Page Programming Size

- Every write access to the underlying hardware memory will be done by using writes that are as big as allowed on the hardware
  - External flash: maximum of QSPI Tx buffer size (256 bytes)

3.2.3 Flash sectors It's possible to modify the behavior of the sector erase / page write and also of the read operation, using two configuration parameters (FlsSectorEraseAsynch or FlsPageWriteAsynch) in FlsSector TAB:



- Fls QuadSPI driver architecture

3.2.4 Fls QuadSPI driver architecture

This section describes the detail for a high-level overview of QuadSPI components in Fls driver, how they interact and how the driver should be used.

Table of content:

- High-level overview
- Use cases
- Supported memories

Related information:

- Clocking and IOMUX for QuadSPI (chapter "3.3 Hardware Resources" in User Manual)
- QuadSPI in multicore context (if supported by the platform, chapter "5.8 Multicore support" in Integration Manual)
- QuadSPI external memory assumptions (chapter "9 External assumptions for driver" in Integration Manual)

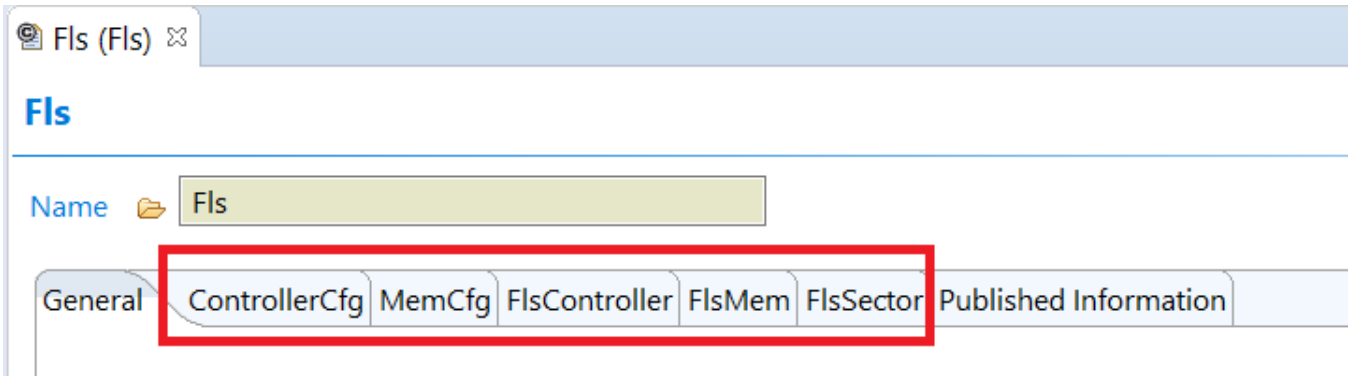
3.2.4.1 High-level overview

This sub-chapter describes the the architecture of the driver:

- The interaction between the HLD, Controller and Memory components
- What each part does and how they interact
- Examples of configuration

3.2.4.1.1 QuadSPI components in Fls driver

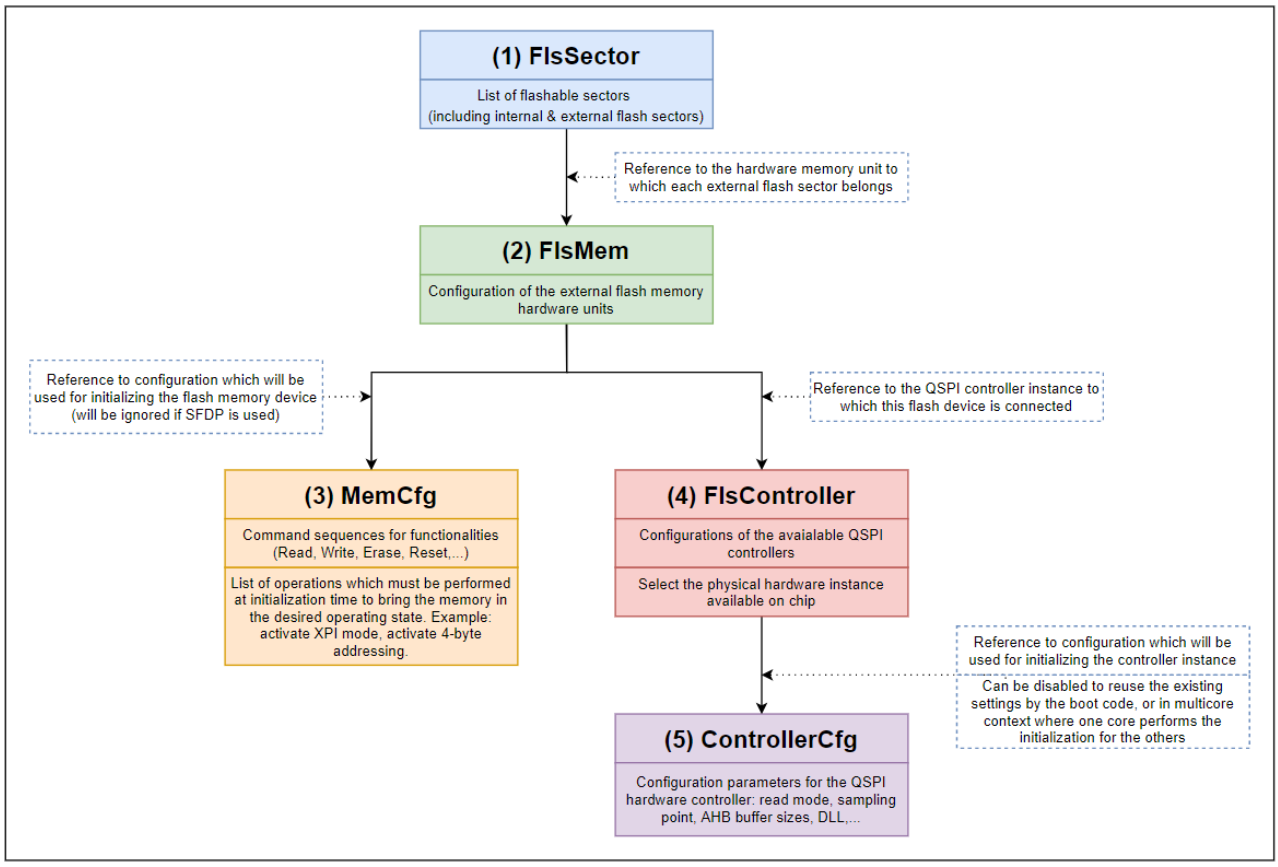
QuadSPI components user interface:



There are five components connect together in the order below:

#	Device name
1	FlsSector
2	FlsMem
3	MemCfg
4	FlsController
5	ControllerCfg

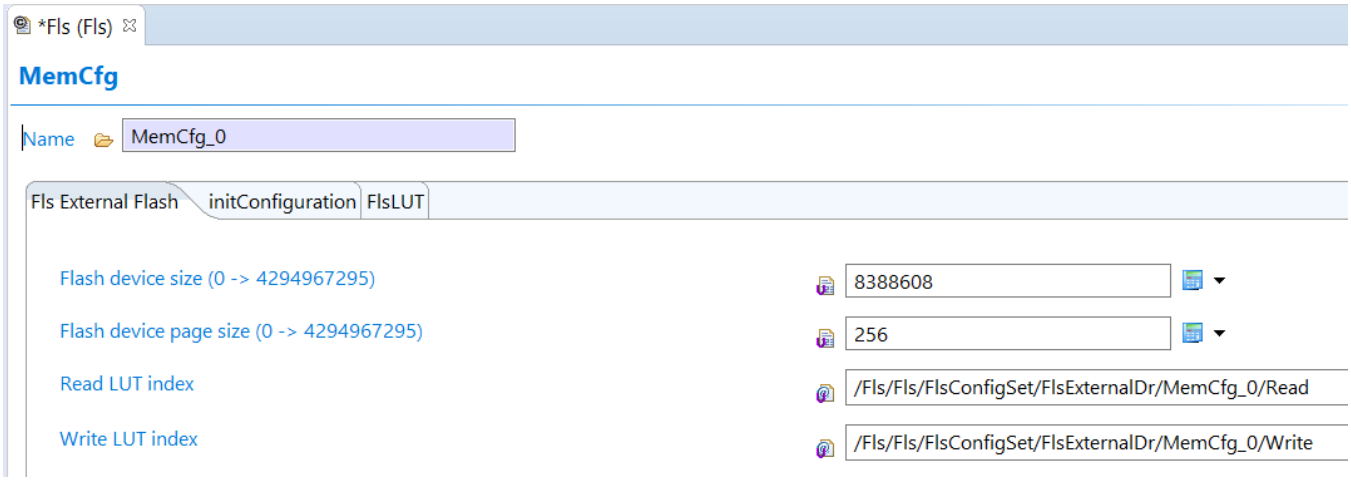
Connections between components:



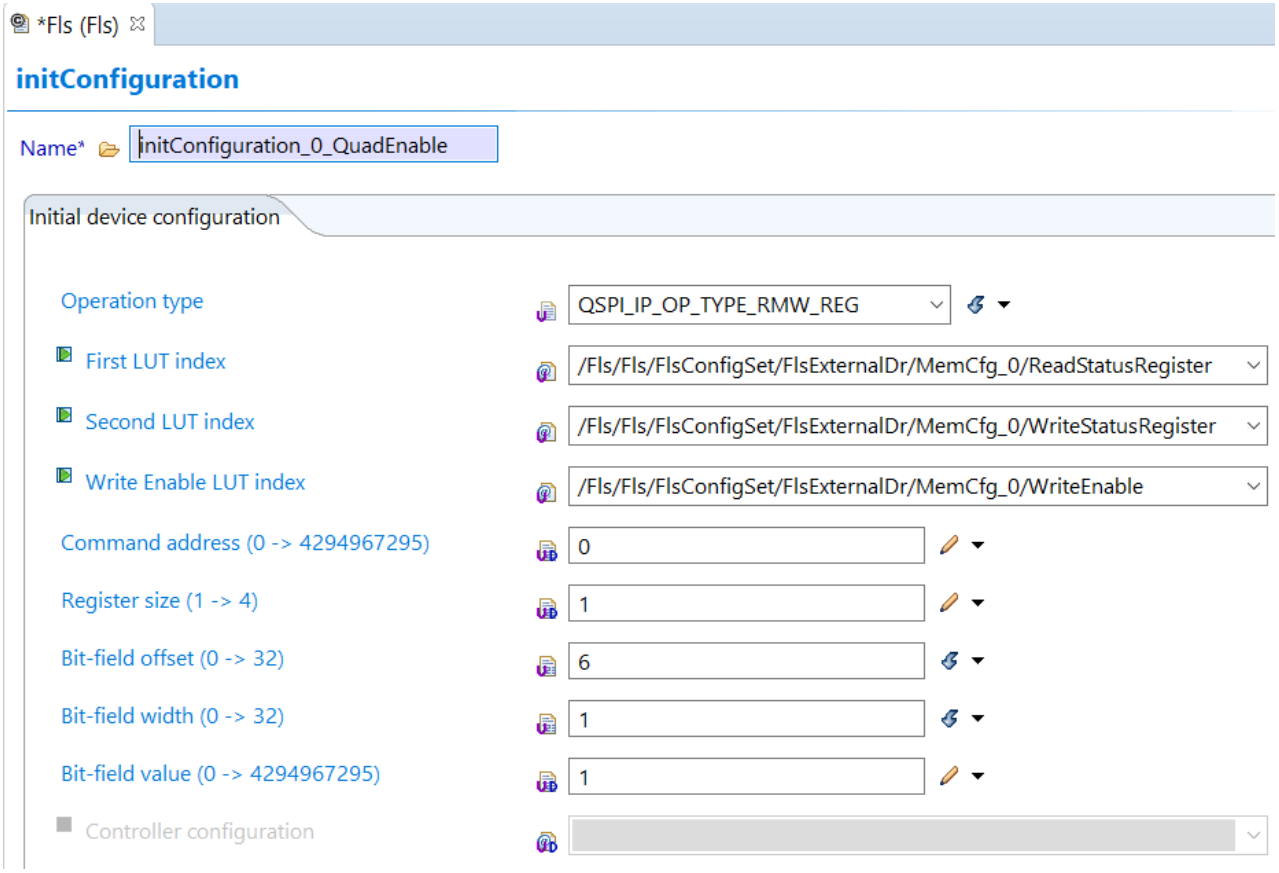
3.2.4.1.2 MemCfg

This container contains all specific settings for the memory device:

- Memory characteristics: device size, page size
- LUT command sequences for basic functionality



It also provides a list of operations (**initConfiguration**) which must be performed at initialization time to bring the memory in the desired operating state (for example: setting registers value). Here is an example of an operation to enable the Quad mode by set bit 6th in the Status register:



In this example, QuadSPI driver will:

- Read 1 byte value of the status register by using the **First LUT index**
- Modify the **6th** bit to the desired value is **1**
- If needed, the **Write Enable LUT index** will be issued before a write command
- Write back that byte value to memory device by using the **Second LUT index**

Note

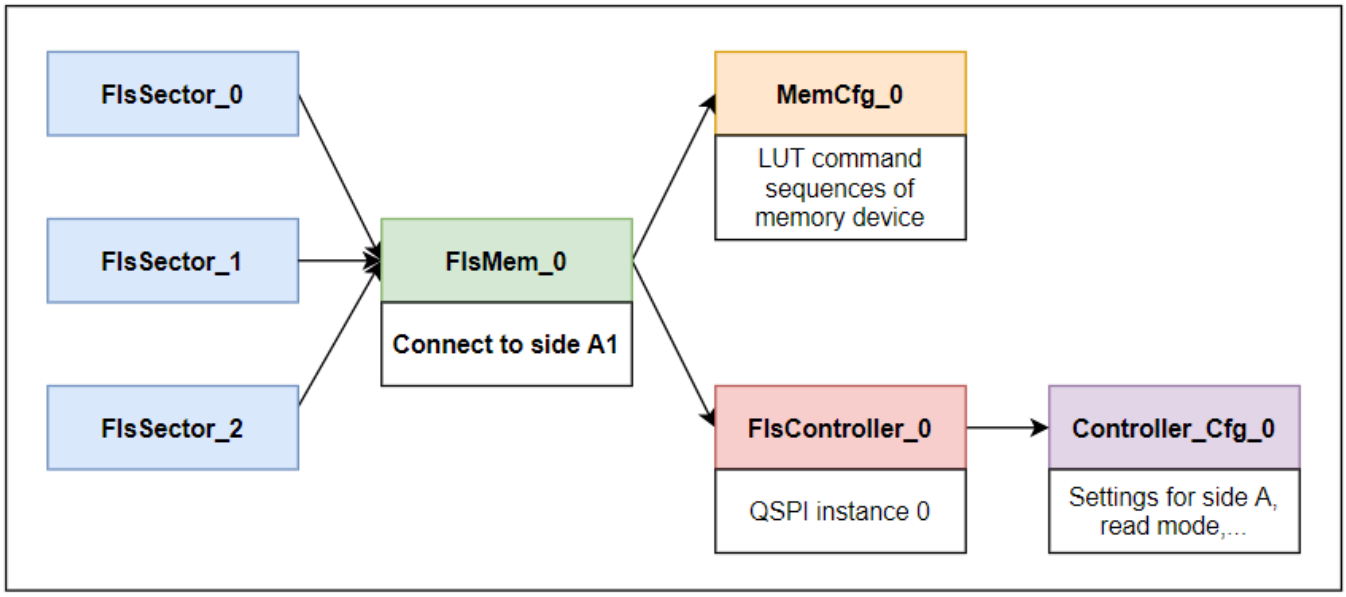
- When changing the value of non-volatile bits, users need to insert one more read operation (QSPI_IP↩ _OP_TYPE_READ_REG) right behind to wait for the write operation to complete

3.2.4.1.3 Examples of configuration

3.2.4.1.3.1 Example 1

Below is the diagram to depict the example from the section "3.2.1 Linear Address". Assume that the memory device connects to the side A1 of QuadSPI controller, we need:

- 01 hardware memory unit (reference to MemCfg_0): contains the LUT command set for initializing the memory device
- 01 controller configuration set (ControllerCfg_0): contains the configuration parameters for QuadSPI hardware instance 0



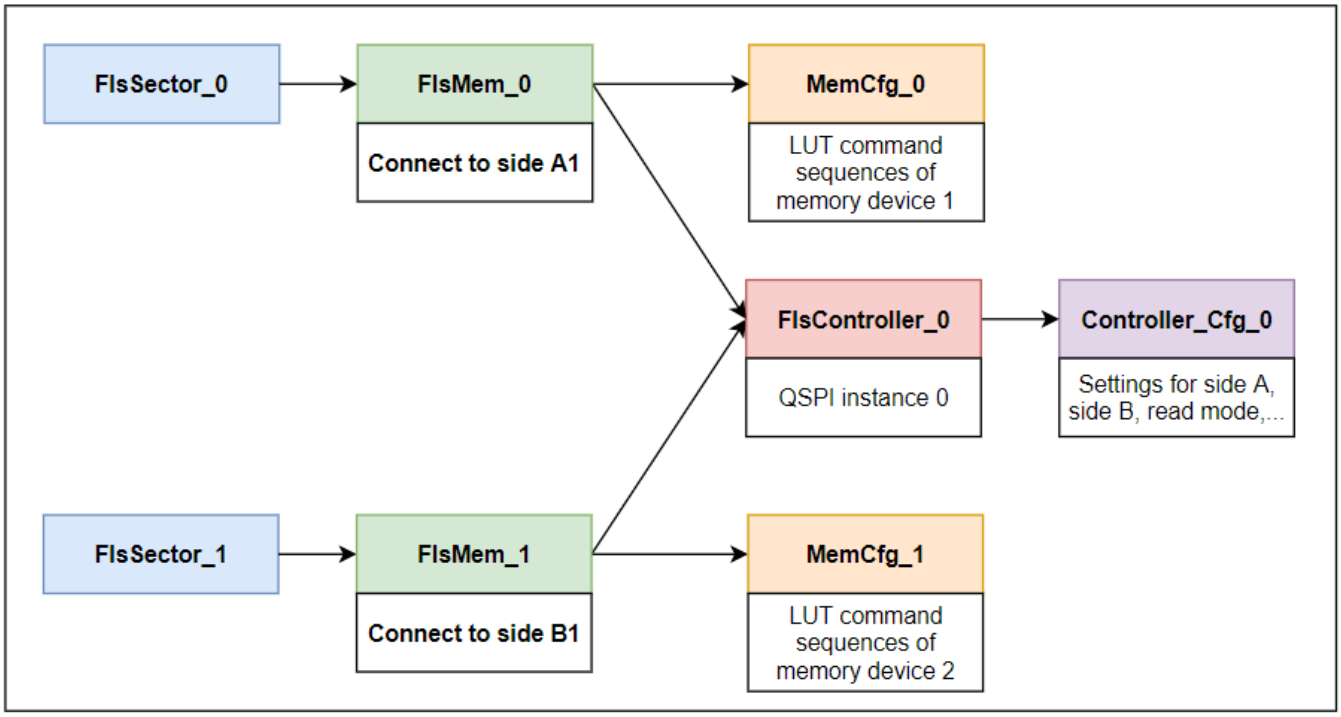
3.2.4.1.3.2 Example 2

Suppose we have 02 different external flash memory devices, one connects to side A and one connects to side B of the QuadSPI controller. And we want to configure 02 external sectors, the first sector in each memory device

FlsSectorList	Fls Sector Size	Fls Number Of Sector	Fls Logical Start Address	Fls Hardware Start Address	Fls Hardware Memory Unit
FlsSector_0	4096 (0x1000)	1	0 (0x0000)	0 (0x0000)	FlsMem_0
FlsSector_1	4096 (0x1000)	1	4096 (0x1000)	0 (0x0000)	FlsMem_1

In this example, we need:

- 02 hardware memory unit (reference to MemCfg_0 and MemCfg_1): contain the LUT command sets for initializing each memory device (if they have different command sets)
- 01 controller configuration set (ControllerCfg_0): contains the configuration parameters for QuadSPI hardware instance 0



As you can see:

- Any operations on the FlsSector_0 will be mapped to the hardware sector 0 (0x0000 - 0x0FFF) of the memory device 1
- Any operations on the FlsSector_1 will be mapped to the hardware sector 0 (0x0000 - 0x0FFF) of the memory device 2
- The QuadSPI controller communicates with each memory device by using the commands set from the corresponding **MemCfg**

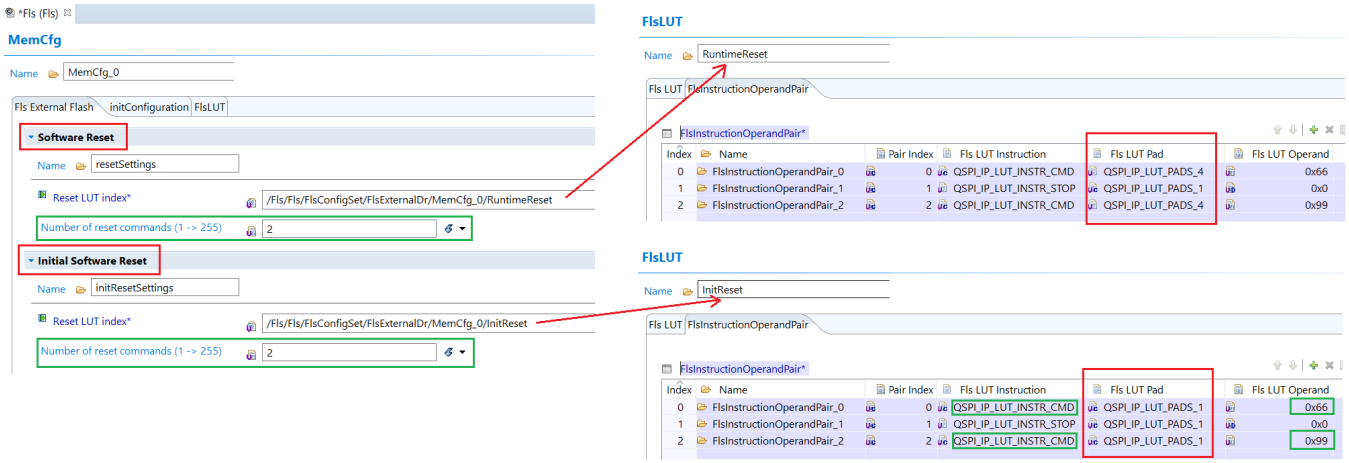
3.2.4.2 Use cases

This sub-chapter provides various useful practical examples.

3.2.4.2.1 Software reset

The driver provides two ways to use the software reset command, for resettings the flash device:

- Software Reset (used by Fls_Cancel() and Qspi_Ip_Reset(), at any time during runtime)
- Initial Software Reset (used by Fls_Init(), only one time)



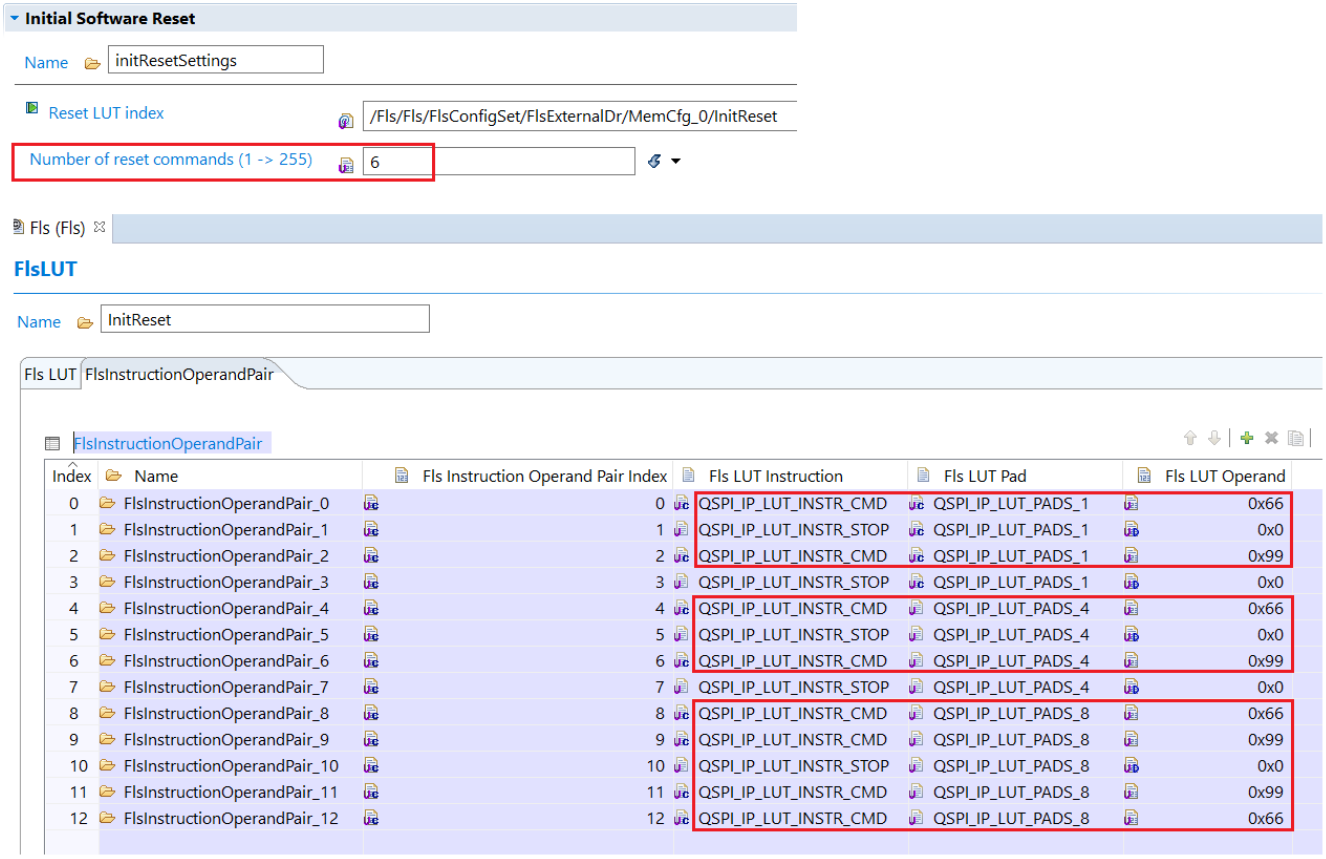
Note

- The number of reset commands is the number of sequences needed by the reset operation, separated by a stop phase
- A stop phase will be inserted automatically at the end of each command sequence

The **Initial Software Reset** procedure applies only at driver intialization. It might be different from the normal reset command, depending on the initial state of the flash. If not, set the same as reset command. In the above example, the memory device works in quad mode (4S-4S-4S), hence we need the reset commands in quad mode.

- The **Initial Software Reset** feature is useful in case we do not know the current state of the device memory (for example when bootrom leaves the memory in a certain state), and we need a reset sequence to bring it to the default state before performing initialization.

Here is an example of a combination of reset command sets (in three modes: SPI, QPI and OPI) to force memory device into its default state:



Note

- Fls_Cancel() will use **Software Reset** to abort the on-going write/erase operation
- This action causes loss of synchronization between QuadSPI controller and memory device (for example when working in DOPI mode with external DQS)
- The next section will describe the solution to deal with this situation

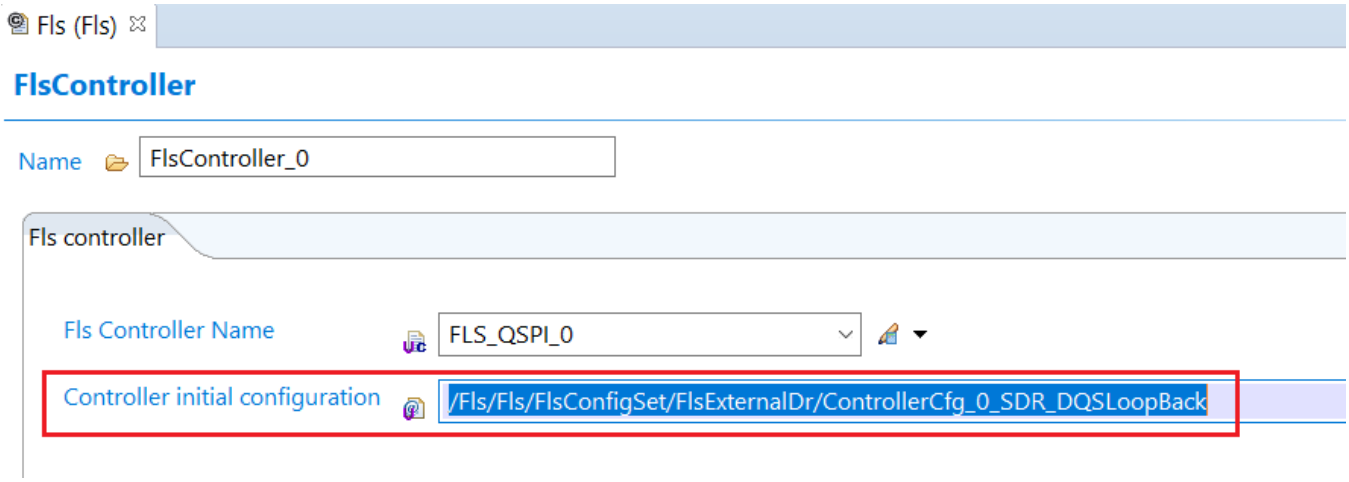
3.2.4.2.2 Controller configuration

There are several controller configuration points in the driver:

#	Controller configuration point	Location	Description
1	Controller initial configuration	FlsController	Configure QuadSPI controller
2	Controller configuration	MemCfg (operations list)	Re-configure QuadSPI controller during memory device initialization
			E.g: switch the controller to External DQS after activating DOPI mode
3	Configure controller on flash Init	MemCfg	Re-configure QuadSPI controller when resetting the memory device
			E.g: switch the controller to initial configuration before re-init the memory device

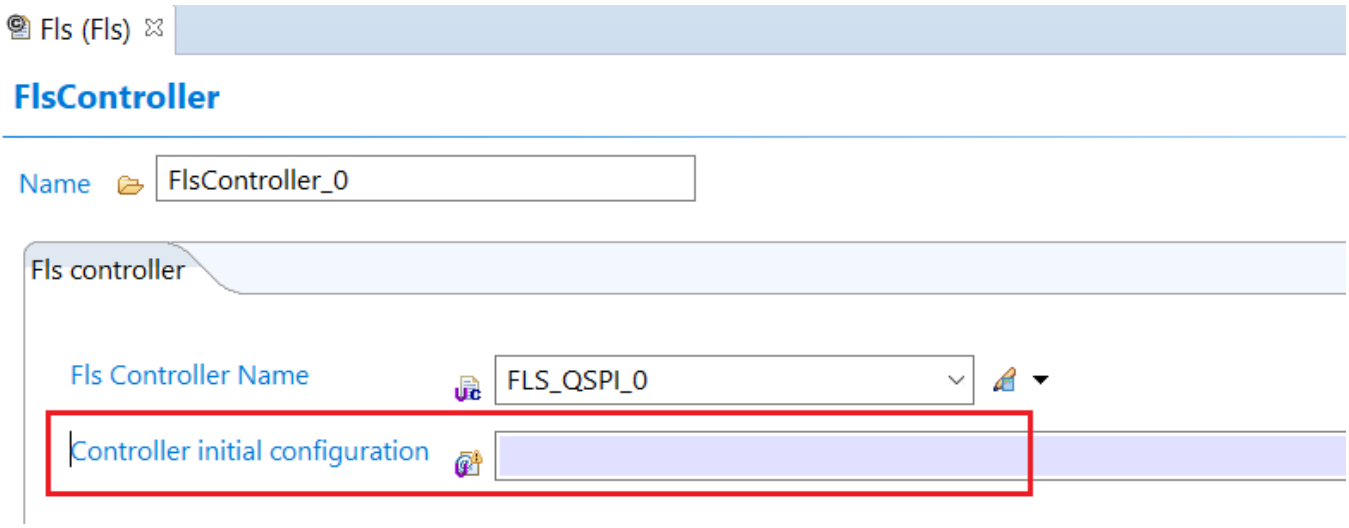
3.2.4.2.2.1 Controller initial configuration

This is the first initialization point which will be done once by Fls_Init():



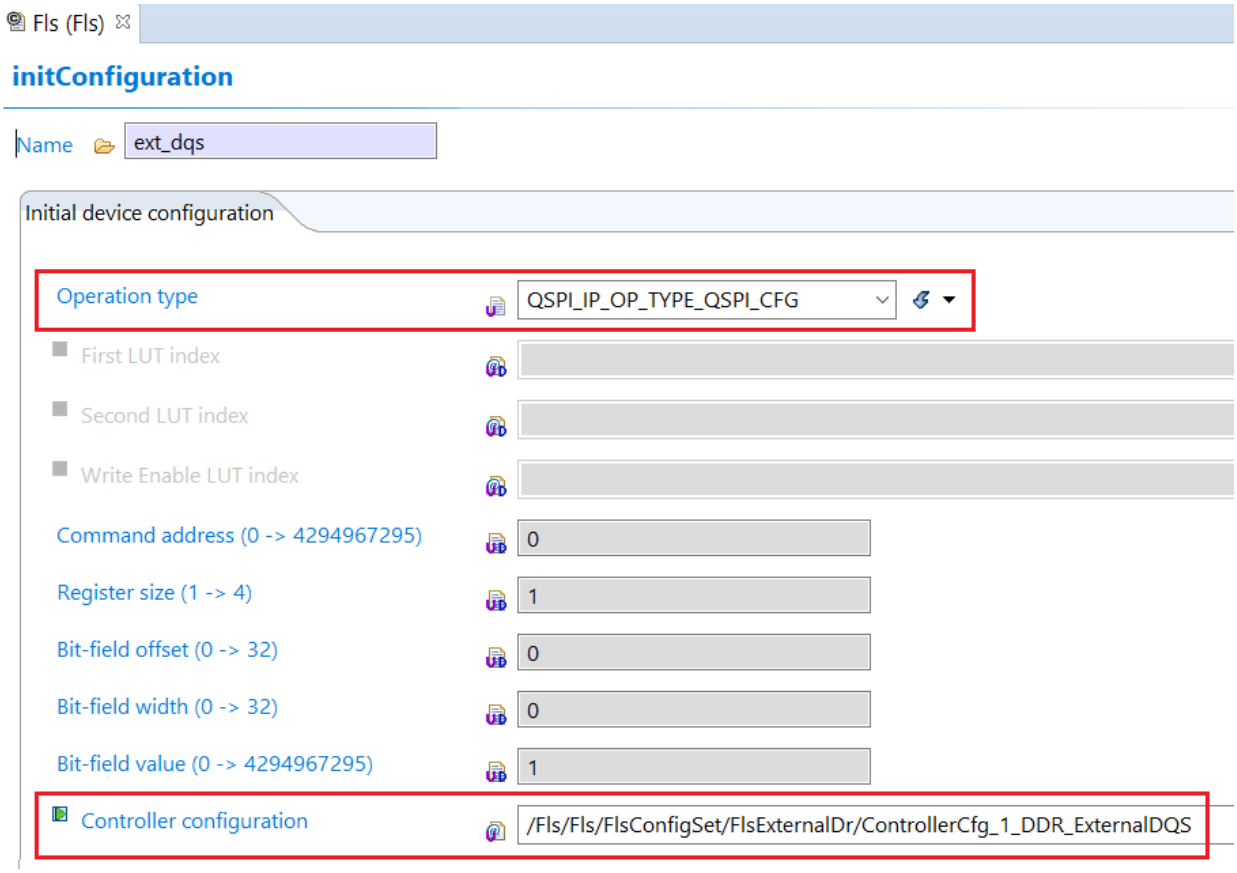
This step can be skipped by leaving the reference node blank, the purpose is:

- Reuse the existing QuadSPI settings by the boot code
- Or in multicore context where one core performs the initialization for the others



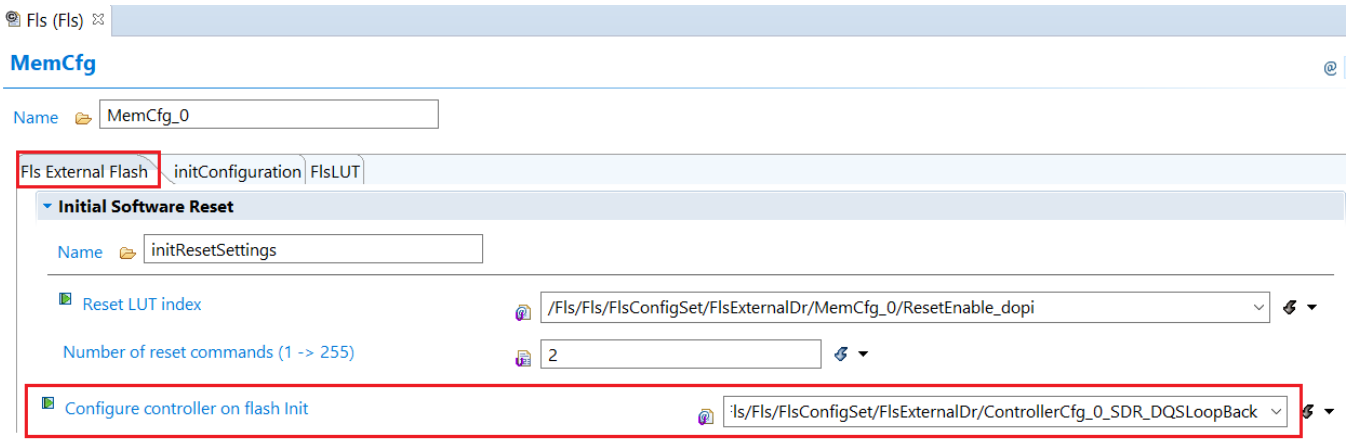
3.2.4.2.2.2 Controller configuration

The second initialization point is in the list of operations of **MemCfg**. This step is needed after we configure the device memory to another mode that is no longer compatible with the controller configuration point #1 (E.g: after activating DOPI mode)

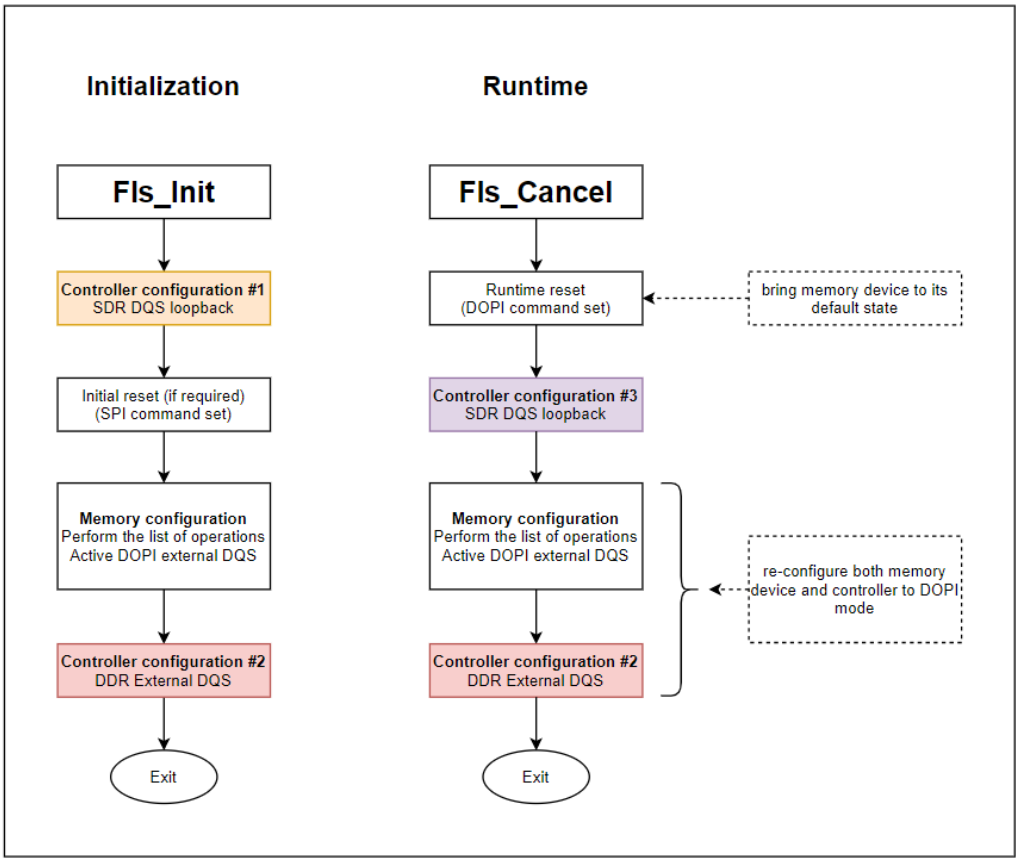


3.2.4.2.2.3 Configure controller on flash Init

The third configuration point is at the end of **MemCfg**. This step is needed when resetting the memory device, then we have to re-configure the controller to a mode that is compatible with the new state of memory device after reset. (E.g: when executing the reset sequence in DOPI mode)

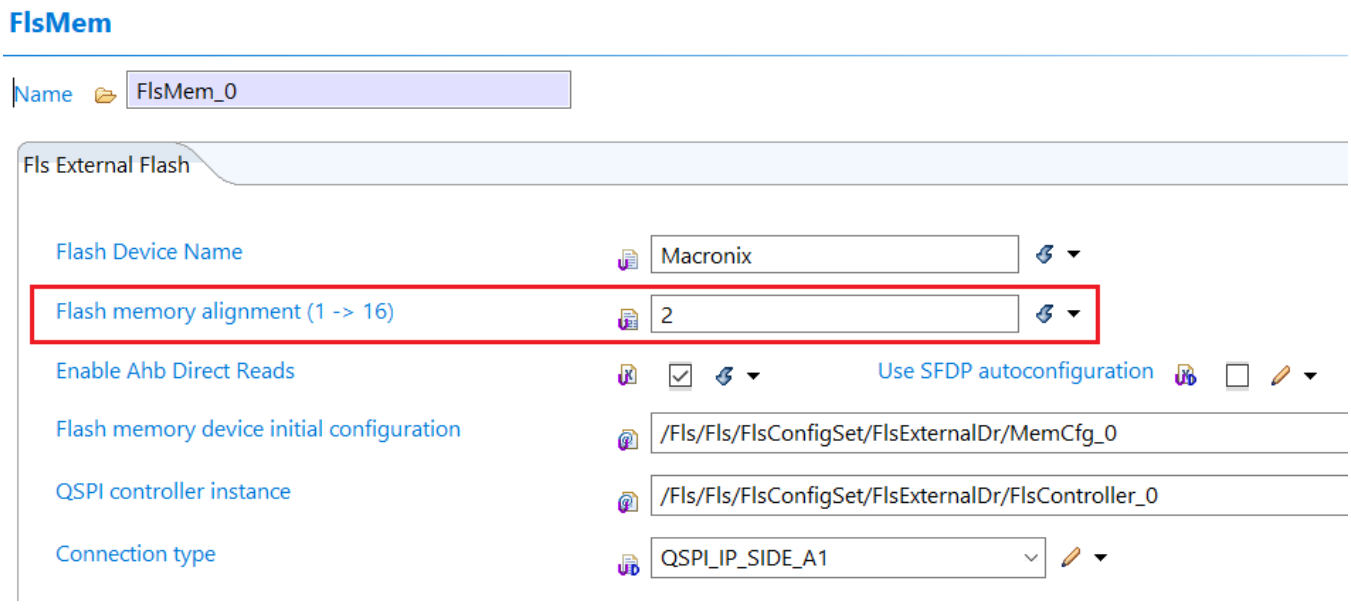


Below is the complete code flow (initialization time and runtime) for both QuadSPI controller and memory device to work in Double data rate - Octal I/O mode (DOPI).



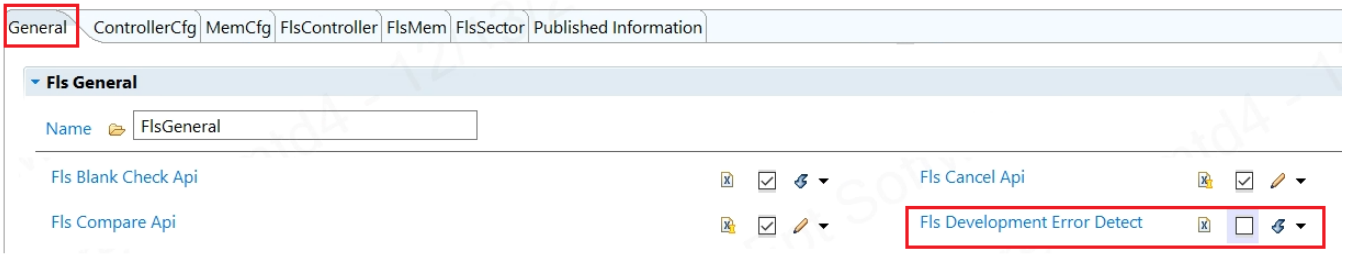
3.2.4.2.3 Read/Write from unaligned addresses

Due to the nature of DDR protocol, both the starting address must be even address and data byte number must be even. Fls driver supports a feature to allow users to read/write with odd addresses and odd data length in DOPI mode, simply by setting the memory alignment value to 2 in the **FlsMem** configuration:

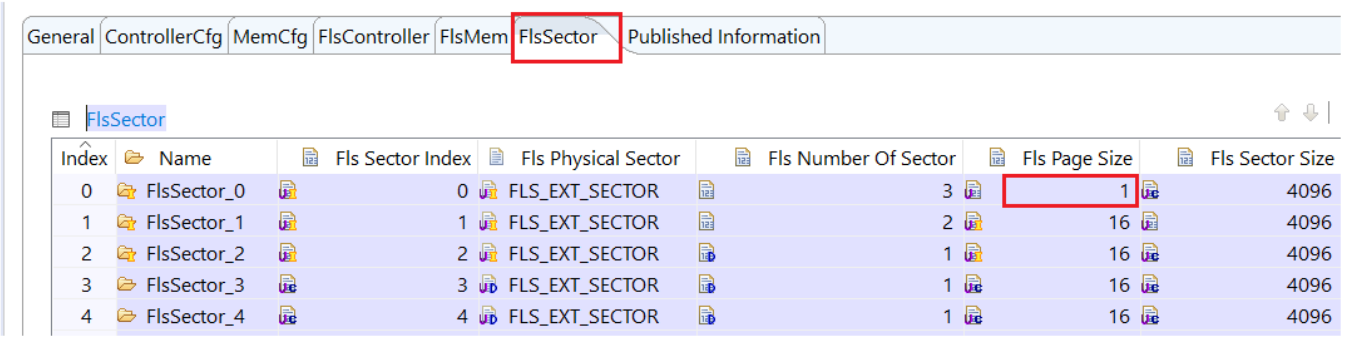


Note

- For write operation, driver will send extra data with FFh to overwrite the overlapping memory area
- Users need to disable the development error detection feature in order to bypass the flash page boundary alignment checks:

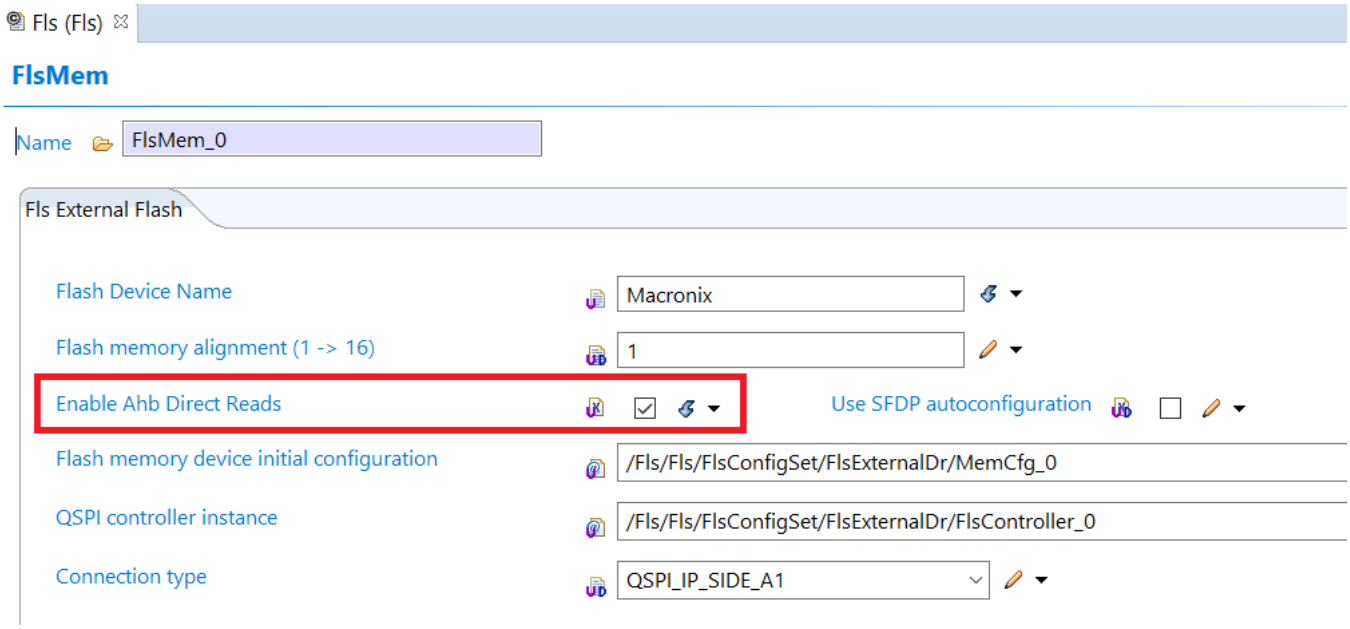


- Or configure the size of flash page boundary to **1** to meet that requirement

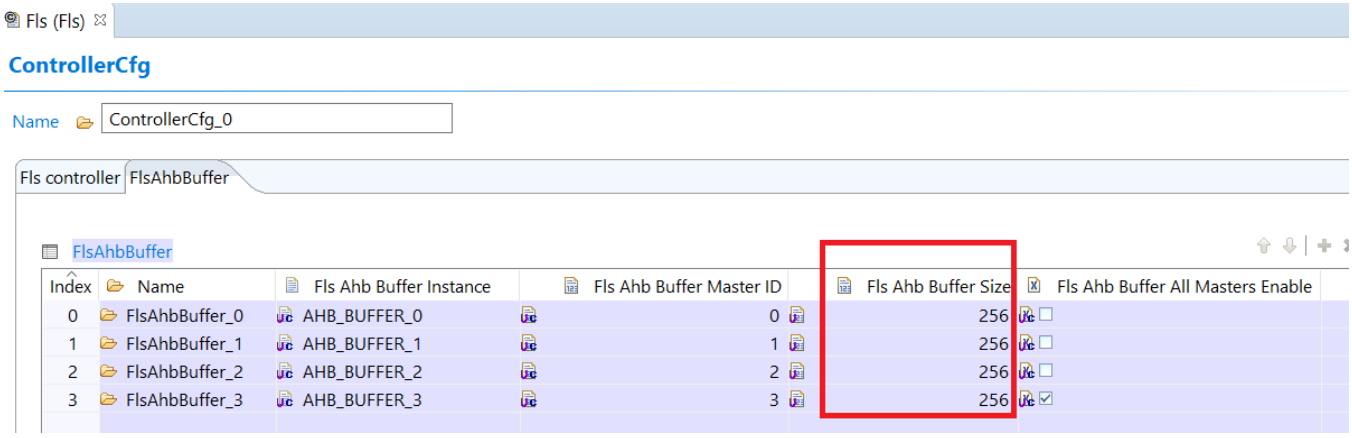


3.2.4.2.4 AHB read

Users can enable the option **AHBReadEnable** in **FlsMem** to use the AHB read feature, this allows application can read directly through Flash memory devices address mapping (QuadSPI's AHB region):



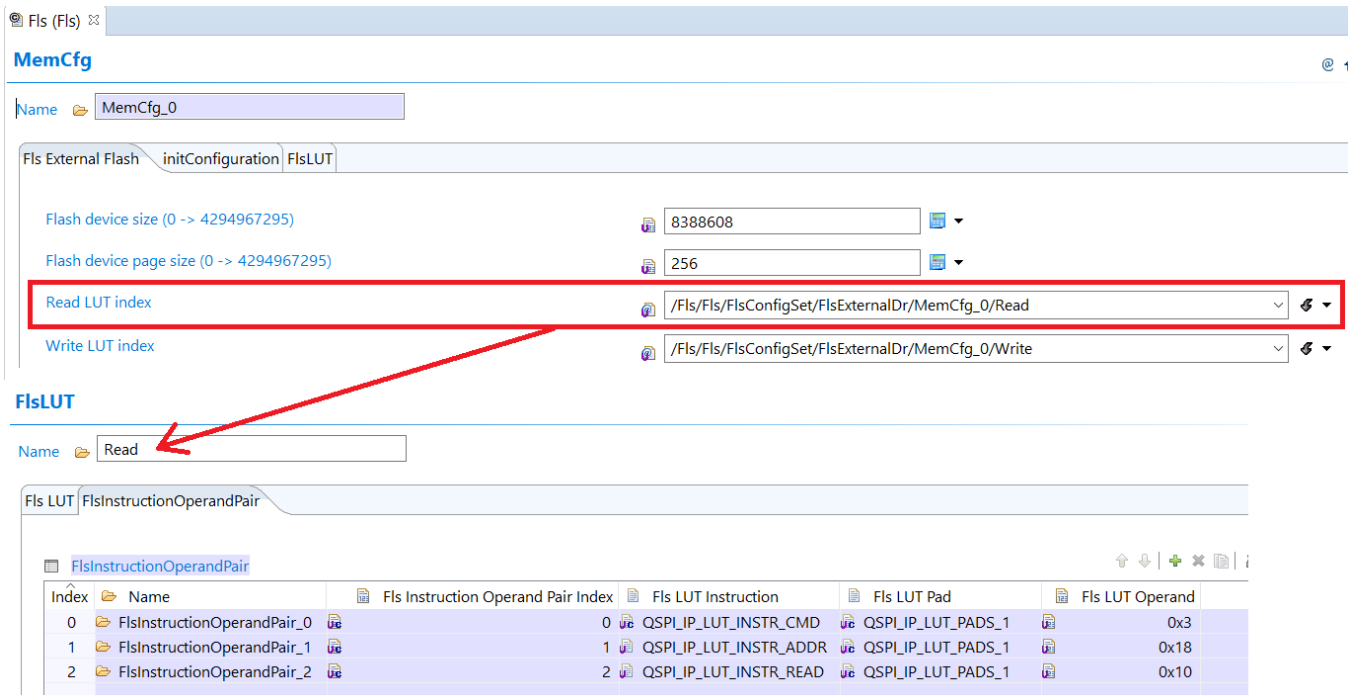
Besides, users need to configure the AHB buffers (master IDs and sizes):



Note

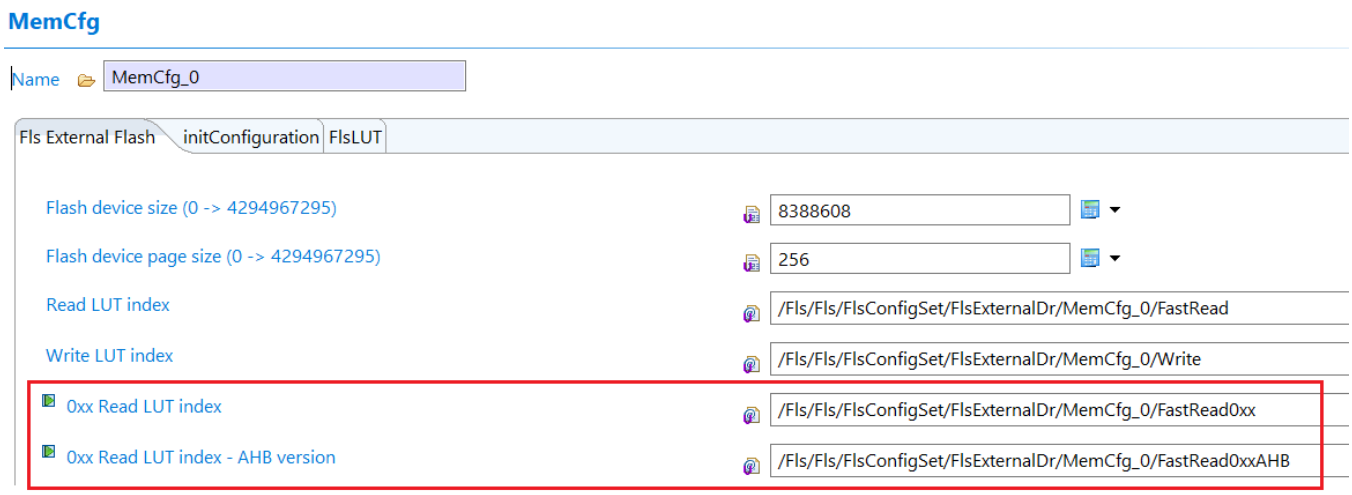
- The driver will configure AHB transfer sizes to match the buffer sizes
- **Qspi_Ip_ControllerGetStatus** can be used to wait for AHB commands complete to avoid conflict with subsequent IP commands

The LUT sequence of IP command read will be used for AHB command read:

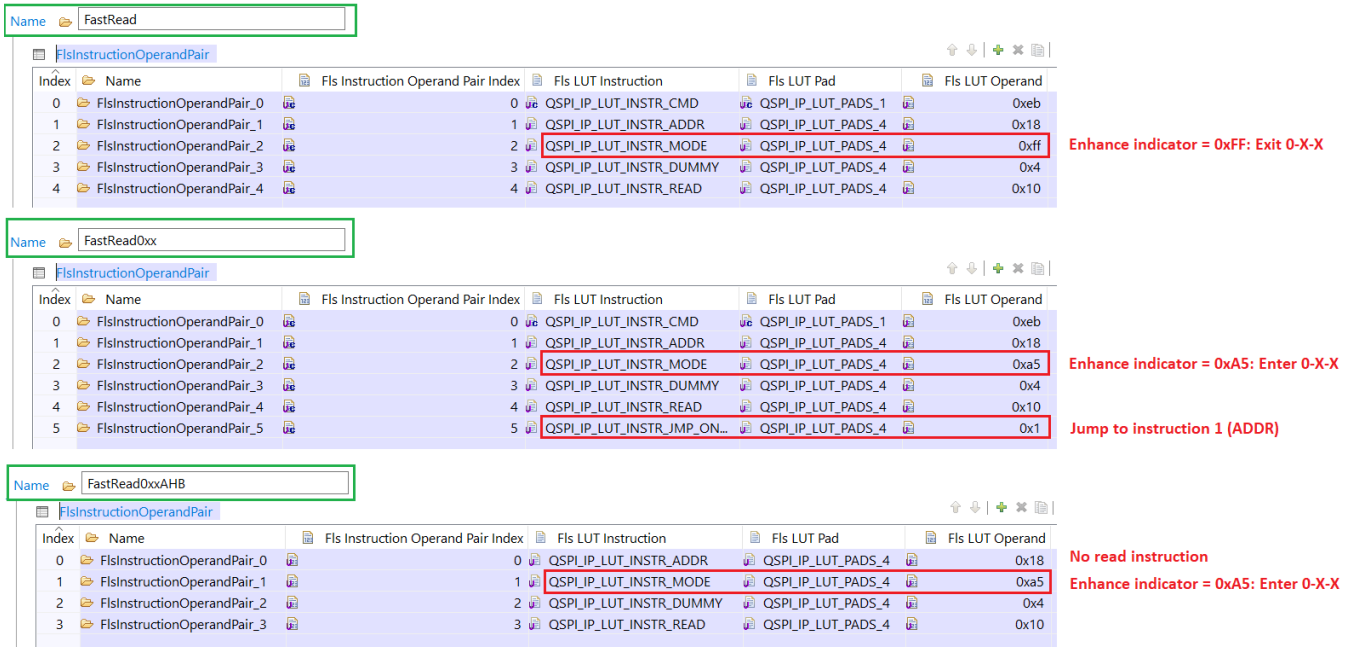


3.2.4.2.5 Performance enhanced mode

The QuadSPI driver supports Continuous Read mode (0-X-X mode - no command for read instructions) which is implemented in some serial Flash memories:



There are two types of command, one for IP and one for AHB operations. Below is the example:



How they work:

1. (Optional) Call the **Qspi_Ip_AhbReadEnable** to enable AHB operation
2. Call the **Qspi_Ip_Enter0XX** to switch to 0-X-X read command sets, driver will perform a dummy read to activate 0-X-X mode

3. Call the **Qspi_Ip_Read** to read data from flash memory without the send of the instruction code
4. (Optional) Access the QuadSPI's AHB region to read data directly, **Qspi_Ip_ControllerGetStatus** can be used to wait for AHB commands complete to avoid conflict with subsequent IP commands
5. Call the **Qspi_Ip_Exit0XX** to disable 0-X-X mode and switch back to normal read command sets

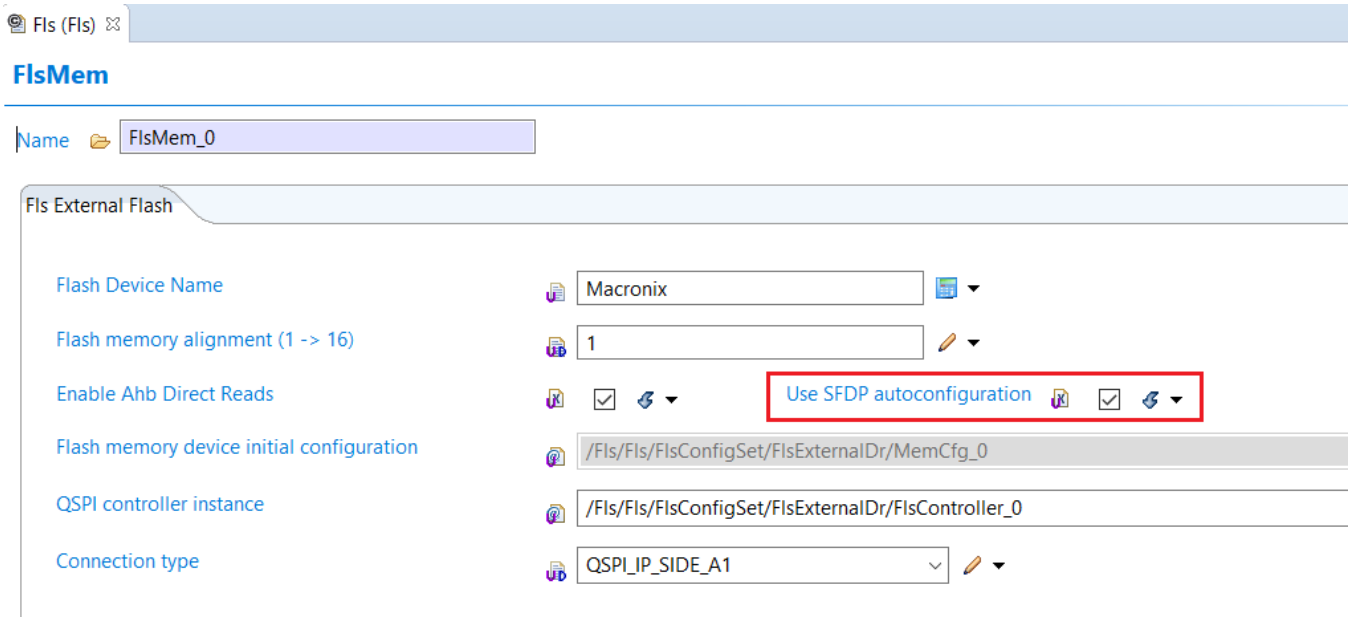
Note

Qspi_Ip_ClearIpSeqPointer() and **Qspi_Ip_ClearAHBSeqPointer()** can be useful for devices which support burst modes for enhancing performance

3.2.4.2.6 SFDP feature

Fls driver contains the SFDP software, user can enable this option to attempt auto-configuration using the information read from the SFDP table

- SFDP (Serial Flash Discoverable Parameters) is a JEDEC standard - JESD216D



Note

- The SFDP software only works for flash devices which support the SFDP feature
- Only standard JEDEC tables are interrogated, all vendor-specific tables will be skipped
- SFDP compliant devices must support 50 MHz operation for the Read SFDP command (instruction 5Ah). Some devices may support a wider frequency range, but user should run at 50 MHz or less and get valid results.
- The SFDP software does not support to read the tables directly in Octal-DDR mode (8D-8D-8D). It tries to issue 8D-8D-8D reset commands to force device to enter SPI mode before reading the SFDP tables. Therefore, user must enable the DDR mode in the QuadSPI controller settings for DDR instructions when working with devices boot up in Octal-DDR mode, or with Hyperflash devices which support the legacy SPI mode. (the Column Address setting is not required)

3.2.4.3 Supported memories

The following external device memories were tested by the Fls driver:

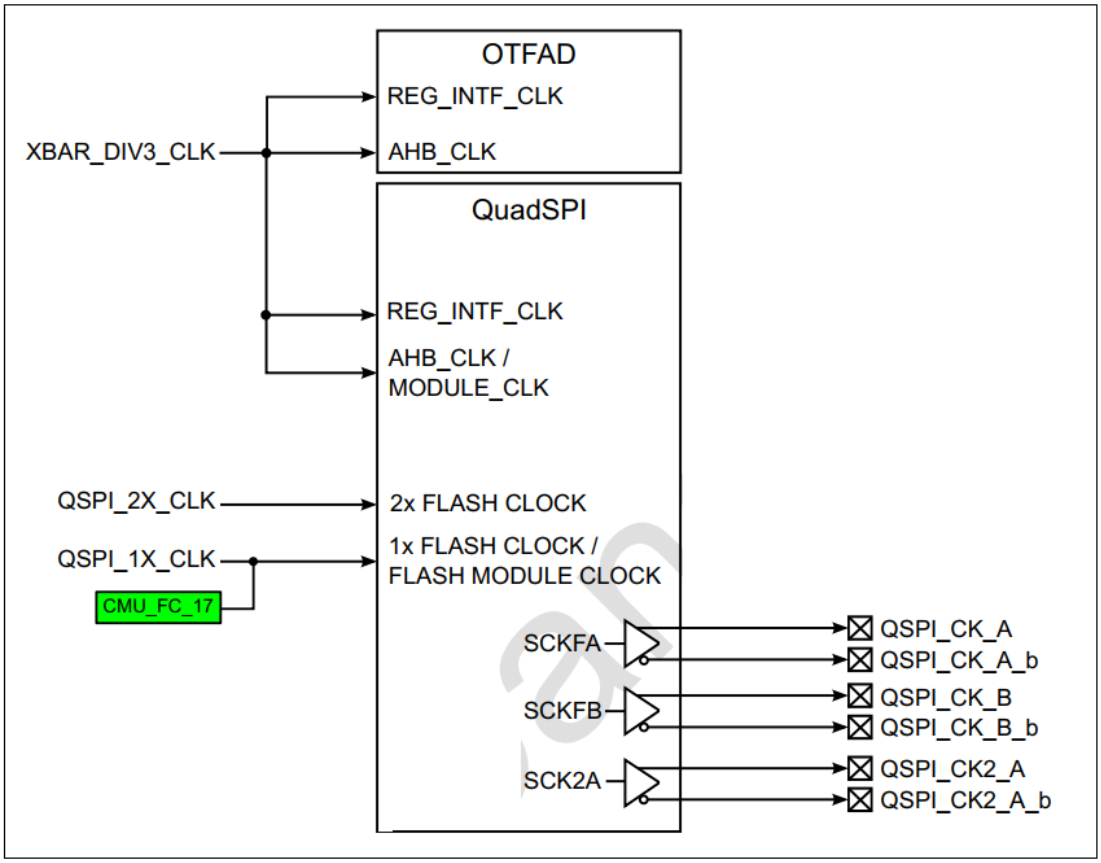
STT	Vendor	Part No	Tested on release	SFDP	Single/ Quad/Octal	Densities (bit)	Voltage	DQS	Frequency Flash Specification		Frequency QSPI supported	
									SDR Freq	DDR Freq	SDR Freq	DDR Freq
1	Macronix	MX25UW51245GXDQ00	S32CC	N	Octal	512 Mb	1.8V	Y	133MHz	200MHz	133MHz	200MHz
2	Macronix	MX25UW51245GXRQ01		Y	Octal	512 Mb	1.8V	Y	133MHz	200MHz	133MHz	200MHz

3.3 Hardware Resources

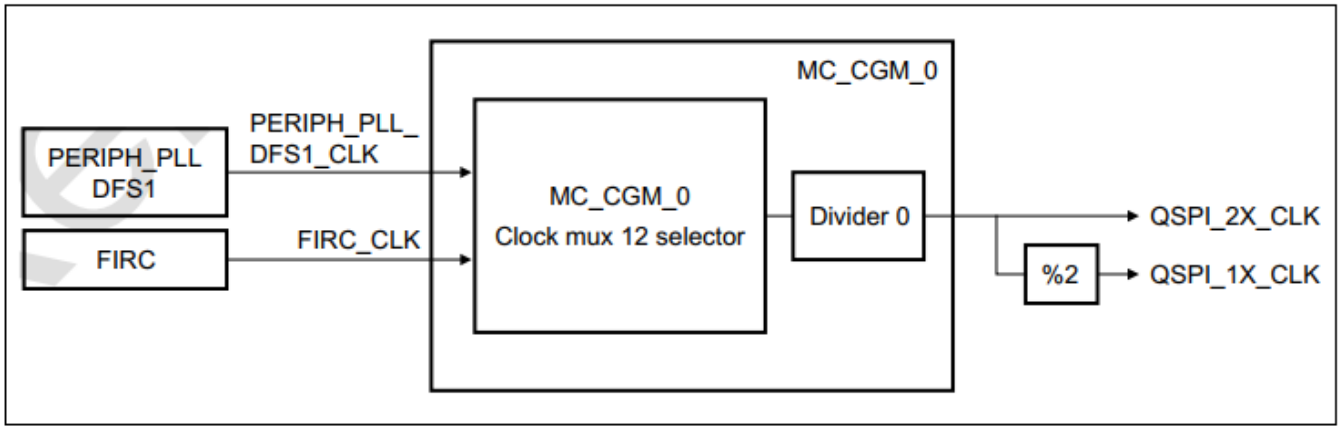
3.3.1 QuadSPI clocking

There are two clock signals used by QuadSPI module:

- XBAR_DIV3_CLK (bus clock) : from CLK_MUX_0 of MC_CGM_0
- QSPI_1X_CLK (flash clock) : from CLK_MUX_12 of MC_CGM_0



CLK_MUX_12 selectors can select QSPI flash clock source from either FIRC or PERIPH_PLL_DFS1, as shown in the following figure



Note

- For the details configurations, please refer to the examples included in the Fls plugin
- For more information about frequency range minimum and maximums, please refer to the chapter "Clock frequency ranges" in the chip data sheet
- The following table gives the frequency range for S32G2

Symbol	Description	Min	Typ	Max	Unit	Condition	Spec Number
fA53_CORE_DIV2_CLK	Cortex-A53 core div2 clock frequency	24	—	500	MHz	A53_CORE_DIV2_CLK	—
fXBAR_DIV3_CLK	XBAR div3 clock frequency	8	—	133	MHz	XBAR_DIV3_CLK	—
fQSPI_2X_CLK	QSPI 2X clock frequency	—	—	400	MHz	QSPI_2X_CLK - DDR 200MHz	—
fQSPI_2X_CLK	QSPI 2X clock frequency	—	—	333	MHz	QSPI_2X_CLK - DDR 166MHz	—
fQSPI_2X_CLK	QSPI 2X clock frequency	—	—	266	MHz	QSPI_2X_CLK - DDR / SDR 133MHz	—
fQSPI_2X_CLK	QSPI 2X clock frequency	—	—	208	MHz	QSPI_2X_CLK - SDR 104MHz	—
fQSPI_2X_CLK	QSPI 2X clock frequency	—	—	200	MHz	QSPI_2X_CLK - SDR 100MHz	—
fQSPI_2X_CLK	QSPI 2X clock frequency	—	—	133	MHz	QSPI_2X_CLK - DDR 66MHz	—
fQSPI_1X_CLK	QSPI 1X clock frequency	—	—	200	MHz	QSPI_1X_CLK - DDR 200MHz	—
fQSPI_1X_CLK	QSPI 1X clock frequency	—	—	166	MHz	QSPI_1X_CLK - DDR 166MHz	—
fQSPI_1X_CLK	QSPI 1X clock frequency	—	—	133	MHz	QSPI_1X_CLK - DDR / SDR 133MHz	—
fQSPI_1X_CLK	QSPI 1X clock frequency	—	—	104	MHz	QSPI_1X_CLK - SDR 104MHz	—
fQSPI_1X_CLK	QSPI 1X clock frequency	—	—	100	MHz	QSPI_1X_CLK - SDR 100MHz	—
fQSPI_1X_CLK	QSPI 1X clock frequency	—	—	66	MHz	QSPI_1X_CLK - DDR 66MHz	—

3.3.2 QuadSPI pin mux

This section provides the external signal information for the QuadSPI module.

Port	CR	CR Instance Name	SSS	Addr	Function[2]	Description	Direction
PC_14			0000_0010		QSPI_DATA_B_O[0]	QuadSPI B Data 0	O
PC_14	552	SIUL_CC	0000_0010	0x4009CAE0	QSPI_DATA_B_I[0]	QuadSPI B Data 0	I
PC_15			0000_0010		QSPI_DATA_B_O[3]	QuadSPI B Data 3	O
PC_15	553	SIUL_CC	0000_0010	0x4009CAE4	QSPI_DATA_B_I[3]	QuadSPI B Data 3	I
PD_00			0000_0010		QSPI_CS_B0	QuadSPI B Chip Select 0	O
PD_01			0000_0010		QSPI_CS_B1	QuadSPI B Chip Select 1	O
PD_01	559	SIUL_CC	0000_0010	0x4009CAFC	QSPI_INTB_b	Quad SPI B Interrupt	I
PD_02			0000_0010		QSPI_DATA_B_O[4]	QuadSPI B Data 4	O
PD_02	557	SIUL_CC	0000_0010	0x4009CAF4	QSPI_DATA_B_I[4]	QuadSPI B Data 4	I
PD_03			0000_0010		QSPI_DATA_B_O[1]	QuadSPI B Data 1	O
PD_03	554	SIUL_CC	0000_0010	0x4009CAE8	QSPI_DATA_B_I[1]	QuadSPI B Data 1	I
PD_04			0000_0010		QSPI_DQS_B_O	Quad SPI B Data Strobe	O
PD_04	558	SIUL_CC	0000_0010	0x4009CAF8	QSPI_DQS_B_I	Quad SPI B Data Strobe	I
PD_05			0000_0010		QSPI_DATA_B_O[7]	QuadSPI B Data 7	O
PD_05	555	SIUL_CC	0000_0010	0x4009CAEC	QSPI_DATA_B_I[7]	QuadSPI B Data 7	I
PD_06			0000_0010		QSPI_CK_B	QuadSPI Serial Clock Flash B +	O
PD_07			0000_0010		QSPI_CK_B_b	QuadSPI Serial Clock Flash B -	O
PD_08			0000_0010		QSPI_DATA_B_O[5]	QuadSPI B Data 5	O
PD_08	550	SIUL_CC	0000_0010	0x4009CAD8	QSPI_DATA_B_I[5]	QuadSPI B Data 5	I
PD_09			0000_0010		QSPI_DATA_B_O[2]	QuadSPI B Data 2	O
PD_09	551	SIUL_CC	0000_0010	0x4009CADC	QSPI_DATA_B_I[2]	QuadSPI B Data 2	I
PD_10			0000_0010		QSPI_DATA_B_O[6]	QuadSPI B Data 6	O
PD_10	556	SIUL_CC	0000_0010	0x4009CAF0	QSPI_DATA_B_I[6]	QuadSPI B Data 6	I
PF_05			0000_0001		QSPI_DATA_A_O[0]	QuadSPI A Data 0	O
PF_05	540	SIUL_CC	0000_0010	0x4009CAB0	QSPI_DATA_A_I[0]	QuadSPI A Data 0	I
PF_06			0000_0001		QSPI_DATA_A_O[1]	QuadSPI A Data 1	O
PF_06	541	SIUL_CC	0000_0010	0x4009CAB4	QSPI_DATA_A_I[1]	QuadSPI A Data 1	I
PF_07			0000_0001		QSPI_DATA_A_O[2]	QuadSPI A Data 2	O
PF_07	542	SIUL_CC	0000_0010	0x4009CAB8	QSPI_DATA_A_I[2]	QuadSPI A Data 2	I
PF_08			0000_0001		QSPI_DATA_A_O[3]	QuadSPI A Data 3	O
PF_08	543	SIUL_CC	0000_0010	0x4009CABC	QSPI_DATA_A_I[3]	QuadSPI A Data 3	I
PF_09			0000_0001		QSPI_DATA_A_O[4]	QuadSPI A Data 4	O
PF_09	544	SIUL_CC	0000_0010	0x4009CAC0	QSPI_DATA_A_I[4]	QuadSPI A Data 4	I
PF_10			0000_0001		QSPI_DATA_A_O[5]	QuadSPI A Data 5	O
PF_10	545	SIUL_CC	0000_0010	0x4009CAC4	QSPI_DATA_A_I[5]	QuadSPI A Data 5	I
PF_11			0000_0001		QSPI_DATA_A_O[6]	QuadSPI A Data 6	O
PF_11	546	SIUL_CC	0000_0010	0x4009CAC8	QSPI_DATA_A_I[6]	QuadSPI A Data 6	I
PF_12			0000_0001		QSPI_DATA_A_O[7]	QuadSPI A Data 7	O
PF_12	547	SIUL_CC	0000_0010	0x4009CACC	QSPI_DATA_A_I[7]	QuadSPI A Data 7	I
PF_13			0000_0001		QSPI_DQS_A_O	Quad SPI A Data Strobe	O
PF_13	548	SIUL_CC	0000_0010	0x4009CAD0	QSPI_DQS_A_I	Quad SPI A Data Strobe	I
PF_14	549	SIUL_CC	0000_0010	0x4009CAD4	QSPI_INTA_b	Quad SPI A Interrupt	I
PG_00			0000_0001		QSPI_CK_A	QuadSPI Serial Clock Flash A +	O
PG_01			0000_0001		QSPI_CK_A_b	QuadSPI Serial Clock Flash A -	O
PG_02			0000_0001		QSPI_CK_2A	QuadSPI Serial Clock 2 Flash A +	O
PG_03			0000_0001		QSPI_CK_2A_b	QuadSPI Serial Clock 2 Flash A -	O
PG_04			0000_0001		QSPI_CS_A0	QuadSPI A Chip Select 0	O
PG_05			0000_0001		QSPI_CS_A1	QuadSPI A Chip Select 1	O

The following table lists the external signals belonging to the module in conjunction with the different modes of operation.

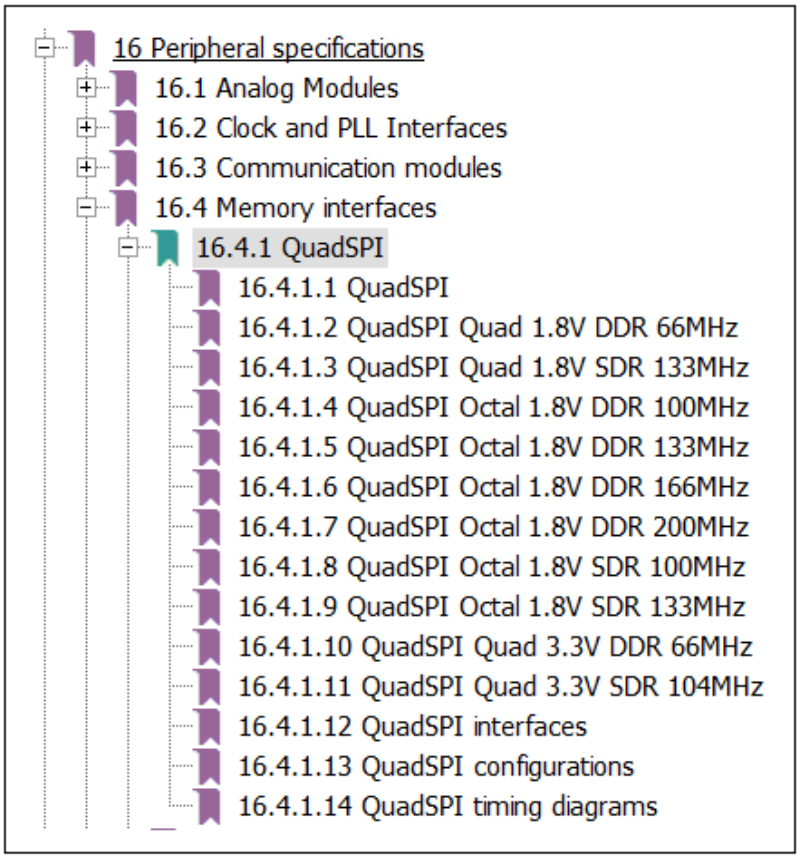
Signal name	Function	Direction	Description
PCSFA1	Peripheral Chip Select Flash Memory A1	O	This signal is the chip select for the serial flash memory device A1 that represents the first device in a dual-die package flash memory A or the first of the two flash memory devices that share IOFA. See Dual-die flash memories for details.
PCSFA2	Peripheral Chip Select Flash Memory A2	O	This signal is the chip select for the serial flash memory device A2 that represents the second device in a dual-die package flash memory A or the second of the two flash memory devices that share IOFA. See Dual-die flash memories for details.
PCSFB1	Peripheral Chip Select Flash memory B1	O	This signal is the chip select for the serial flash memory device B1 that represents the first device in a dual-die package flash memory B or the first of the two flash memory devices that share IOFB. See Dual-die flash memories for details.
PCSFB2	Peripheral Chip Select Flash Memory B2	O	This signal is the chip select for the serial flash memory device B2 that represents the second device in a dual-die package flash memory B or the second of the two flash memory devices that share IOFB. See Dual-die flash memories for details.
SCKFA	Serial Clock Flash Memory A	O	This signal is the serial clock output to the serial flash memory device A.
SCKFB	Serial Clock Flash B	O	This signal is the serial clock output to the serial flash memory device B.
IOFA[7:0]	Serial I/O Flash memory A	I/O	These signals are the data I/O lines to/from the serial flash memory device A. See Driving external signals for details about the signal drive and timing behavior. Note that the signal pins of the serial flash memory device may change their function according to the SFM Command executed, leaving them as control inputs when single and dual instructions are executed. The module supports driving these inputs to dedicated values. In single I/O mode, QuadSPI drives data on IOFA[0] and expects data on IOFA[1].
IOFB[7:0]	Serial I/O Flash Memory B	I/O	These signals are the data I/O lines to/from the serial flash memory device B. See Driving external signals for details about the signal drive and timing behavior. Note that the signal pins of the serial flash memory device may change their function according to the SFM command executed, leaving them as control inputs when single and dual instructions are executed. The module supports driving these inputs to dedicated values. In single I/O mode, QuadSPI drives data on IOFB[0] and expects data on IOFB[1].
DQSFA	Data Strobe signal Flash Memory A	I	This is the data strobe signal for port A. Some flash memory vendors provide the Data Read Strobe (DQS) signal to which the read data is aligned in DDR mode.
DQSFB	Data Strobe signal Flash Memory B	I	This is the data strobe signal for port B. Some flash memory vendors provide the DQS signal to which the read data is aligned in DDR mode.
INTA	ECC error signal for Flash Memory A	I	Flash Memory A drives this signal to active low value in case of an ECC error.
INTB	ECC error signal for Flash Memory B	I	Flash Memory B drives this signal to active low value in case of an ECC error.

Note

- Please refer to the examples for more details about configurations of pin mux, driver strength, pull up and slew rate settings

3.3.3 QuadSPI supported read modes

For more details about supported read mode and their configurations, please refer to the chapter "Peripheral specifications" in the chip data sheet



The following table gives the QuadSPI configurations for S32G2

16.4.1.13 QuadSPI configurations

Table 54. QuadSPI configurations

-	DDR-200MHz	DDR-133MHz	SDR-133MHz	SDR-104MHz	DDR-66MHz
DQS mode	External DQS Edge Aligned	External DQS Edge Aligned	Internal pad loopback	Internal pad loopback	Internal pad loopback
Sampling mode	DDR	DDR	SDR	SDR	DDR
DLL Mode	DLL Enable	DLL Enable	DLL Bypass	DLL Bypass	DLL Enable
Data Learning	No	No	No	No	Yes
IO Voltage	1.8V	1.8V	1.8V	3.3V	1.8V/3.3V
Frequency	166/200 MHz	100/133 MHz	100/133 MHz	104 MHz	66 MHz
FLSHCR[TDH]	1	1	0	0	1
FLSHCR[TCSH]	3	3	3	3	3
FLSHCR[TCSS]	3	3	3	3	3
MCR[DLPEN]	0	0	0	0	1
DLLCR[DLLEN]	1	1	0	0	1
DLLCR[FREQEN]	1	0	0	0	0
DLLCR[DLL_REFCNTR]	2	2	NA	NA	2
DLLCR[DLLRES]	8	8	NA	NA	8
DLLCR[SLV_FINE_OFFSET]	0	0	0	0	0
DLLCR[SLV_DLY_OFFSET]	0	0	0	0	3
DLLCR[SLV_DLY_COARSE]	NA	NA	0	0	0
DLLCR[SLAVE_AUTO_UPDT]	1	1	0	0	1
DLLCR[SLV_EN]	1	1	1	1	1
DLLCR[SLV_DLL_BYPASS]	0	0	1	1	0
DLLCR[SLV_UPD]	1	1	1	1	1
SMPR[DLLFSMPF]	4	4	0	0	NA
SMPR[FSDLY]	0	0	0	0	1
SMPR[FSPHS]	NA	NA	1	1	NA

3.4 Deviations from Requirements

The driver deviates from the AUTOSAR FLS Driver software specification in someplaces. There are also some additional requirements (on top of requirements detailed in AUTOSAR FLS Driver software specification) which need to be satisfied for correct operation.

- Deviations Status Column Description

Term	Definition
N/S	Not In Scope
N/F	Not Fully Implemented
N/I	Not Implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, or out of scope for the driver.

- Driver Deviations Table

Requirement	Status	Description	Notes
SWS_Fls_00145	N/S	If possible, e.g. with interrupt controlled implementations, the FLS module shall start the first round of the erase job directly within the function Fls_Erase to reduce overall runtime.	Fls driver does not support interrupt
SWS_Fls_00146	N/S	If possible, e.g. with interrupt controlled implementations, the F↩ LS module shall start the first round of the write job directly within the function Fls_Write to reduce overall runtime.	Fls driver does not support interrupt
SWS_Fls_00232	N/S	The configuration parameter Fls↩ UseInterrupts shall switch between interrupt and polling controlled job processing if this is supported by the flash memory hardware.	Fls driver does not support interrupt
SWS_Fls_00233	N/S	The FLS module's implementer shall locate the interrupt service routine in Fls_Irq.c.	Fls driver does not support interrupt
SWS_Fls_00234	N/S	If interrupt controlled job processing is supported and enabled with the configuration parameter FlsUse↩ Interrupts, the interrupt service routine shall reset the interrupt flag, check for errors reported by the underlying hardware, reload the hardware finite state machine for the next round of the pending job or call the appropriate notification routine if the job is finished or aborted.	Fls driver does not support interrupt
CPR_RTD_00431.fls	N/S	The Flash driver shall support an external HyperRAM memory when this is available.	Fls driver does not support Hyper↩ RAM in S32CC platform

- As a deviation from standard: Fls_[VariantName]_PBcfg.c files will contain the definition for all parameters that are variant aware, independent of the configuration class that will be selected (PC, LT, PB) Fls_Cfg.c, Fls_Cfg.h file will contain the definition for all parameters that are not variant aware

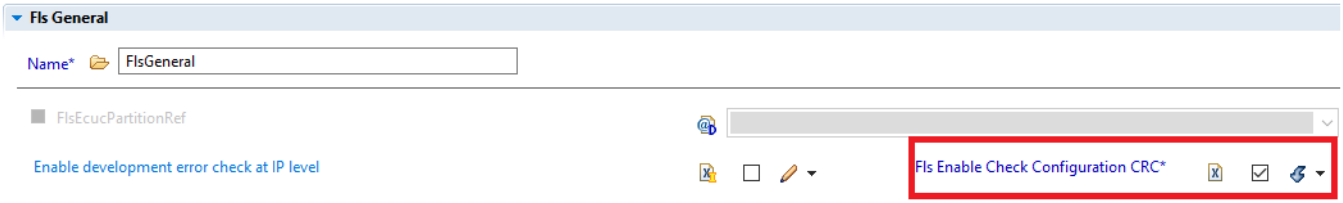
3.5 Driver Limitations

- Users must add and use MPU to configure the QSPI AHB address space as device memory to prevent speculative reads in this region
- The SFDP feature only supports standard JEDEC tables, all vendor-specific tables will be skipped

3.6 Driver usage and configuration tips

3.6.1 Fls Enable Check Configuration CRC

The CRC configuration check that takes an amount of time, which may not be necessary if the image is already authenticated. So the driver will support the CRC check optional for the FLS Configuration. With the default to OFF (disabled). User can enable this function on configuration interface



3.6.2 Development Error Description

Error Code	Value	Condition triggering the error
FLS_E_PARAM_CONFIG	1	API service called with wrong parameter
FLS_E_PARAM_ADDRESS	2	u32TargetAddress is not in range and aligned to first byte of flash sector
FLS_E_PARAM_LENGTH	3	u32TargetAddress is not in range and aligned to last byte of flash sector
FLS_E_PARAM_DATA	4	API service called with wrong parameter
FLS_E_UNINIT	5	API service called without module initialization
FLS_E_BUSY	6	API service called while driver still busy
FLS_E_PARAM_POINTER	10	API service called with NULL_PTR passed

3.7 Runtime errors

- The driver supports runtime generation of the errors listed in the table:

Error code	Function	Condition triggering the error
FLS_E_VERIFY_ERASE_FAILED	Fls_MainFunction()	Verify erasing operation failed before writing a flash block
		Verify erasing operation failed after erasing a flash block
FLS_E_VERIFY_WRITE_FAILED	Fls_MainFunction()	Verify writing operation failed after writing a flash block
FLS_E_TIMEOUT	Fls_Init()	Access timeout value to the flash controller has been exceeded
	Fls_MainFunction()	Maximum read / write / compare / erase time has been exceeded

- The driver supports Transient faults generation of the errors listed in the table:

Error Code	Function	Condition triggering the error
FLS_E_ERASE_FAILED	Fls_MainFunction()	A flash erase job fails due to a hardware error
FLS_E_WRITE_FAILED		A flash write job fails due to a hardware error
FLS_E_READ_FAILED		A flash read job fails due to a hardware error
FLS_E_COMPARE_FAILED		A flash compare job fails due to a hardware error
FLS_E_UNEXPECTED_FLASH_ID	Fls_Init()	The hardware ID of the external flash device mismatched the corresponding configuration parameter

- Development Error Description

Error Code	Value	Condition triggering the error
FLS_E_PARAM_CONFIG	1	API service called with wrong parameter
FLS_E_PARAM_ADDRESS	2	u32TargetAddress is not in range and aligned to first byte of flash sector
FLS_E_PARAM_LENGTH	3	u32TargetAddress is not in range and aligned to last byte of flash sector
FLS_E_PARAM_DATA	4	NULL_PTR == SourceAddressPtr
FLS_E_UNINIT	5	API service called without module initialization
FLS_E_BUSY	6	PI service called while driver still busy
FLS_E_PARAM_POINTER	10	NULL_PTR passed

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like:

#define <Mip>Conf_<Container_ShortName>_<Container_ID>

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module Fls
  - Container FlsConfigSet
    - Parameter FlsAcErase
    - Parameter FlsAcWrite
    - Parameter FlsAcErasePointer
    - Parameter FlsAcWritePointer
    - Parameter FlsCallCycle
    - Parameter FlsDefaultMode
    - Parameter FlsJobEndNotification
    - Parameter FlsJobErrorNotification
    - Parameter FlsMCoreTimeoutNotification
    - Parameter FlsQspiInitCallout
    - Parameter FlsQspiResetCallout
    - Parameter FlsQspiErrorCheckCallout
    - Parameter FlsQspiEccCheckCallout
    - Parameter FlsMaxReadFastMode
    - Parameter FlsMaxReadNormalMode
    - Parameter FlsMaxWriteFastMode
    - Parameter FlsMaxWriteNormalMode
    - Parameter FlsProtection
    - Container FlsExternalDriver
      - Reference FlsSpiReference
      - Container ControllerCfg
      - Parameter FlsHwUnitReadMode
      - Parameter FlsSerialFlashA1Size
      - Parameter FlsSerialFlashA2Size
      - Parameter FlsSerialFlashB1Size
      - Parameter FlsSerialFlashB2Size
      - Parameter FlsDdrCentrerAlignReadA
      - Parameter FlsClockOnDifferentialCknPadA

- Parameter FlsDdrCentrerAlignReadB
- Parameter FlsClockOnDifferentialCknPadB
- Parameter FlsHwUnitSamplingModeA
- Parameter FlsHwUnitSamplingModeB
- Parameter IdleSignalDriveIOFB3HighLvl
- Parameter IdleSignalDriveIOFB2HighLvl
- Parameter IdleSignalDriveIOFA3HighLvl
- Parameter IdleSignalDriveIOFA2HighLvl
- Parameter FlsHwUnitSamplingEdge
- Parameter FlsHwUnitSamplingDly
- Parameter FlsHwUnitDqsLatencyEnable
- Parameter FlsHwUnitTdh
- Parameter FlsHwUnitTcsh
- Parameter FlsHwUnitTcss
- Parameter FlsHwUnitColumnAddressWidth
- Parameter FlsHwUnitByteSwapping
- Parameter FlsHwUnitWordAddressable
- Container FlsAhbBuffer
- Parameter FlsAhbBufferInstance
- Parameter FlsAhbBufferMasterId
- Parameter FlsAhbBufferSize
- Parameter FlsAhbBufferAllMasters
- Container DllCfgA
- Parameter DllCfgADllMode
- Parameter DllCfgADllCraFreqEn
- Parameter DllCfgADllCraReferenceCounter
- Parameter DllCfgADllCraResolution
- Parameter DllCfgADllCraSlvFineOffset
- Parameter DllCfgADllCraSlvDlyOffset
- Parameter DllCfgADllCraSlvDlyCoarse
- Parameter DllCfgADllTapSelect
- Container DllCfgB
- Parameter DllCfgBDllMode
- Parameter DllCfgBDllCraFreqEn
- Parameter DllCfgBDllCrbReferenceCounter
- Parameter DllCfgBDllCrbResolution
- Parameter DllCfgBDllCraSlvFineOffset
- Parameter DllCfgBDllCraSlvDlyOffset
- Parameter DllCfgBDllCraSlvDlyCoarse
- Parameter DllCfgBDllTapSelect
- Container MemCfg
- Parameter MemCfgSize
- Parameter MemCfgPageSize
- Reference MemCfgReadLUT
- Reference MemCfgWriteLUT
- Reference MemCfgRead0xxLUT
- Reference MemCfgRead0xxLUTAHB

- Reference ctrlAutoCfgPtr
- Container MemCfgReadIdSettings
- Parameter MemCfgReadIdSize
- Parameter FlsQspiDeviceId
- Reference MemCfgReadIdLUT
- Container MemCfgEraseSettings
- Parameter MemCfgErase1Size
- Parameter MemCfgErase2Size
- Parameter MemCfgErase3Size
- Parameter MemCfgErase4Size
- Reference MemCfgErase1LUT
- Reference MemCfgErase2LUT
- Reference MemCfgErase3LUT
- Reference MemCfgErase4LUT
- Reference ChipEraseLUT
- Container statusConfig
- Parameter regSize
- Parameter busyOffset
- Parameter busyValue
- Parameter writeEnableOffset
- Parameter blockProtectionOffset
- Parameter blockProtectionWidth
- Parameter blockProtectionValue
- Reference statusRegInitReadLut
- Reference statusRegReadLut
- Reference statusRegWriteLut
- Reference writeEnableSRLut
- Reference writeEnableLut
- Container suspendSettings
- Reference eraseSuspendLut
- Reference eraseResumeLut
- Reference programSuspendLut
- Reference programResumeLut
- Container resetSettings
- Parameter resetCmdCount
- Reference resetCmdLut
- Container initResetSettings
- Parameter resetCmdCount
- Reference resetCmdLut
- Container initConfiguration
- Parameter opType
- Parameter addr
- Parameter size
- Parameter shift
- Parameter width
- Parameter value
- Reference command1Lut

- Reference command2Lut
- Reference weLut
- Reference ctrlCfgPtr
- Container FlsLUT
- Parameter FlsLUTIndex
- Container FlsInstructionOperandPair
- Parameter FlsInstrOperPairIndex
- Parameter FlsLUTInstruction
- Parameter FlsLUTPad
- Parameter FlsLUTOperand
- Container HyperflashCfg
- Parameter MemCfgSize
- Parameter MemCfgPageSize
- Parameter outputDriverStrength
- Parameter RWDSLowOnDualError
- Parameter secureRegionUnlocked
- Parameter readLatency
- Parameter paramSectorMap
- Reference ctrlAutoCfgPtr
- Container MemCfgReadIdSettings
- Parameter MemCfgReadIdLUT
- Parameter MemCfgReadIdWordAddr
- Parameter MemCfgReadIdSize
- Parameter FlsQspiDeviceId
- Container FlsController
- Parameter ControllerName
- Reference FlsControllerCfgRef
- Container FlsMem
- Parameter FlsMemName
- Parameter MemAlignment
- Parameter AHBReadEnable
- Parameter FlsMemUseSfdp
- Parameter connectionType
- Reference FlsMemCfgRef
- Reference qspiInstance

* Container FlsSectorList
  - Container FlsSector
  - Parameter FlsSectorIndex
  - Parameter FlsPhysicalSector
  - Parameter FlsNumberOfSectors
  - Parameter FlsPageSize
  - Parameter FlsSectorSize
  - Parameter FlsSectorStartaddress
  - Parameter FlsSectorEraseAsynch
  - Parameter FlsPageWriteAsynch
  - Parameter FlsHwCh
  - Parameter FlsSectorHwAddress

        - Reference flashInstance
    - Container AutosarExt
        - Parameter FlsEnableUserModeSupport
        - Parameter FlsQspiLockLUT
        - Parameter FlsQspiHangRecovery
        - Parameter FlsSynchronizeCache
        - Parameter FlsMCoreEnable
        - Parameter FlsMCoreQJobSemaphoreChannelNo
        - Parameter FlsExternalSectorsConfigured
    - Container FlsGeneral
        - Parameter FlsEnableDevAssert
        - Parameter FlsEnableCheckCfgCrc
        - Parameter FlsAcLoadOnJobStart
        - Parameter FlsBaseAddress
        - Parameter FlsBlankCheckApi
        - Parameter FlsCancelApi
        - Parameter FlsCompareApi
        - Parameter FlsDevErrorDetect
        - Parameter FlsDriverIndex
        - Parameter FlsGetJobResultApi
        - Parameter FlsGetStatusApi
        - Parameter FlsSetModeApi
        - Parameter FlsTotalSize
        - Parameter FlsUseInterrupts
        - Parameter FlsVersionInfoApi
        - Parameter FlsEraseVerificationEnabled
        - Parameter FlsWriteVerificationEnabled
        - Parameter FlsMaxEraseBlankCheck
        - Parameter FlsTimeoutSupervisionEnabled
        - Parameter FlsTimeoutMethod
        - Parameter FlsQspiIpTimeoutOsifCounterType
        - Parameter FlsQspiSyncReadTimeout
        - Parameter FlsQspiAsyncWriteTimeout
        - Parameter FlsQspiAsyncEraseTimeout
        - Parameter FlsQspiSyncWriteTimeout
        - Parameter FlsQspiSyncEraseTimeout
        - Parameter FlsQspiDllLockTimeout
        - Parameter FlsQspiCommandCompleteTimeout
        - Parameter FlsQspiResetTimeout
        - Parameter FlsQspiFlashInitTimeout
        - Parameter FlsQspiSoftwareResetDelay

    - Parameter FlsQspiTxBufferResetDelay
    - Parameter FlsQspiWriteEnableRetries
    - Parameter FlsMCoreArbitrationTimeout
    - Parameter FlsMCoreInitTimeout
    - Reference FlsEcucPartitionRef
  - Container FlsPublishedInformation
    - Parameter FlsAcLocationErase
    - Parameter FlsAcLocationWrite
    - Parameter FlsAcSizeErase
    - Parameter FlsAcSizeWrite
    - Parameter FlsEraseTime
    - Parameter FlsErasedValue
    - Parameter FlsECCValue
    - Parameter FlsExpectedHwId
    - Parameter FlsSpecifiedEraseCycles
    - Parameter FlsWriteTime
  - Container CommonPublishedInformation
    - Parameter ArReleaseMajorVersion
    - Parameter ArReleaseMinorVersion
    - Parameter ArReleaseRevisionVersion
    - Parameter ModuleId
    - Parameter SwMajorVersion
    - Parameter SwMinorVersion
    - Parameter SwPatchVersion
    - Parameter VendorApiInfix
    - Parameter VendorId

4.1 Module Fls

Configuration of the Fls (internal or external flash driver) module.

Included containers:

- FlsConfigSet
- AutosarExt
- FlsGeneral
- FlsPublishedInformation
- CommonPublishedInformation

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.2 Container FlsConfigSet

Container for runtime configuration parameters of the flash driver.

Implementation Type: Fls_ConfigType.

Included subcontainers:

- FlsExternalDriver
- FlsSectorList

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.3 Parameter FlsAcErase

Address offset in RAM to which the erase flash access code shall be loaded.

Used as function pointer to access the erase flash access code.

Note: To use Fls Access Code Erase be sure Fls Access Code Erase Pointer is NULL or NULL_PTR.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.4 Parameter FlsAcWrite

Address offset in RAM to which the write flash access code shall be loaded.

Used as function pointer to access the write flash access code.

Note: To use Fls Access Code Write be sure Fls Access Code Write Pointer is NULL or NULL_PTR.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.5 Parameter FlsAcErasePointer

Vendor specific: Pointer in RAM to which the erase flash access code shall be loaded.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.6 Parameter FlsAcWritePointer

Vendor specific: Pointer in RAM to which the write flash access code shall be loaded.

Used as function pointer to access the write flash access code.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	NULL_PTR

4.7 Parameter FlsCallCycle

Cycle time of calls of the flash driver main function

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0.2
max	1.0
min	0.0

4.8 Parameter FlsDefaultMode

This parameter is the default FLS device mode after initialization.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	MEMIF_MODE_SLOW
literals	['MEMIF_MODE_FAST', 'MEMIF_MODE_SLOW']

4.9 Parameter FlsJobEndNotification

Mapped to the job end notification routine provided by some upper layer module, typically the Fee module.

Note: Disable the end notification to have it set as NULL_PTR

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	Fee_JobEndNotification

4.10 Parameter FlsJobErrorNotification

Mapped to the job error notification routine provided by some upper layer module, typically the Fee module.

Note: Disable the error notification to have it set as NULL_PTR

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	Fee_JobErrorNotification

4.11 Parameter FlsMCoreTimeoutNotification

Vendor specific: Flash Multi Core Timeout Notification.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	Fls_MCoreTimeoutNotif

4.12 Parameter FlsQspiInitCallout

Vendor specific: Callout function called by the driver at the end of the QSPI Init phase.

The intended purpose of this callout is to provide to the application the

possibility of performing additional configuration to the QSPI hardware IP or

to the external memories connected(for ex: sending the lock/unlock sequences

for the external flash sectors, altering QSPI IP timing, etc.)

Note: Disable the callout in order to have it set as NULL_PTR.

Note: The callout can be configured only if the FlsExternalDriver is enabled.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FlsQspiInitCallout

4.13 Parameter FlsQspiResetCallout

Vendor specific: Callout function called by the driver at the beginning of a new job. The intended purpose of this callout is to provide to the application thepossibility of reseting the external memory to an idle and error free state.

If the callout is disabled, at the beginnig of a new job the Fls_MainFunction will check the external memory status and if not, poll and wait for it to become idle.

If the callout is enabled and the memory is not idle, the Fls_MainFunction will also call the configured function to allow the application to send extra commands to the external memory(software reset, abort any suspended operation, error flags clearing, etc.)

Note: Disable the callout in order to have it set as NULL_PTR.

Note: The callout can be configured only if the FlsExternalDriver is enabled.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FlsQspiResetCallout

4.14 Parameter FlsQspiErrorCheckCallout

Vendor specific: Callout function called by the driver at the end of each program and erase job

The intended purpose of this callout is to provide to the application the

possibility of interrogating the error status of the memory after each program and erase job.

The application should check any error or status bits available and reset the memory after interrogation

in case an error condition was detected.

If the callout is enabled, at the end of each job, the callout is called and the return

value is checked to determine if there was any error during the memory operation.

Return values: E_OK(0) E_NOT_OK(1).

If E_OK(0) is received, the job is considered successful.

If E_NOT_OK(1) is received, the job is considered unsuccessful and marked as failed.

If the callout is disabled, the job is assumed as successful from the memory status point of view.

Note: Disable the callout in order to have it set as NULL_PTR.

Note: The callout can be configured only if the FlsExternalDriver is enabled.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FlsQspiErrorCheckCallout

4.15 Parameter FlsQspiEccCheckCallout

Vendor specific: Callout function called by the driver at the end of each read operation

The intended purpose of this callout is to provide to the application the

possibility of interrogating the ECC status of the memory after each read operation.

The callout provide the hardware channel, start address and the size of the read operation.

The application should check there is any ECC error in the current read data

If the callout is enabled, at the end of each read operation, the callout is called and the return

value is checked to determine if there was any error during the memory operation.

Return values: E_OK(0) E_NOT_OK(1).

If E_OK(0) is received, the job is considered successful.

If E_NOT_OK(1) is received, the job is considered unsuccessful and marked as failed.

If the callout is disabled, the job is assumed as successful from the memory status point of view.

Note: Disable the callout in order to have it set as NULL_PTR.

Note: The callout can be configured only if the FlsExternalDriver is enabled.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD

4.16 Parameter FlsMaxReadFastMode

The maximum number of bytes to read or compare in one cycle of the flash driver's job processing function in fast mode.

Note: If external sectors are configured and if FlsHwUnitWordAddressable is set,

the FlsMaxReadFastMode must be an even value(two bytes aligned).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1048576
max	4294967295
min	0

4.17 Parameter FlsMaxReadNormalMode

The maximum number of bytes to read or compare in one cycle of the flash driver's job processing function in normal mode.

Note: If external sectors are configured and if FlsHwUnitWordAddressable is set,

the FlsMaxReadNormalMode must be an even value(two bytes aligned).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1024
max	4294967295
min	0

4.18 Parameter FlsMaxWriteFastMode

The maximum number of bytes to write in one cycle of the flash driver's job processing function in fast mode.

Note: If external sectors are configured, the FlsMaxWriteFastMode must be an integer multiple of the FlsPageSize.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	256
max	4294967295
min	0

4.19 Parameter FlsMaxWriteNormalMode

The maximum number of bytes to write in one cycle of the flash driver's job processing function in normal mode.

Note: If external sectors are configured, the FlsMaxWriteFastMode must be an integer multiple of the FlsPageSize.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	8
max	4294967295
min	0

4.20 Parameter FlsProtection

Erase/write protection settings.Note:Not supported by the driver.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.21 Container FlsExternalDriver

This container is present for external Flash drivers only. Internal Flash drivers do not use the parameter listed in this container, hence its multiplicity is 0 for internal drivers.

Included subcontainers:

- ControllerCfg
- MemCfg
- HyperflashCfg
- FlsController
- FlsMem

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.22 Reference FlsSpiReference

Reference to SPI sequence. Not used in current implementation.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	true
destination	/AUTOSAR/EcucDefs/Spi/SpiDriver/SpiSequence

4.23 Container ControllerCfg

Vendor specific: Container for defining configurations for QSPI controllers.

These configurations are not tied to a particular controller.

Included subcontainers:

- FlsAhbBuffer
- DllCfgA
- DllCfgB

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.24 Parameter FlsHwUnitReadMode

Vendor specific: The hardware unit read mode:

QSPI_IP_DATA_RATE_SDR (single data rate) which samples incoming data on a single edge.

QSPI_IP_DATA_RATE_DDR (double data rate) which samples incoming data on both edges.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_DATA_RATE_SDR
literals	['QSPI_IP_DATA_RATE_SDR', 'QSPI_IP_DATA_RATE_DDR']

4.25 Parameter FlsSerialFlashA1Size

Vendor specific: Size of flash device connected to side A1 of the controller. Set to 0 if no flash device is connected.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.26 Parameter FlsSerialFlashA2Size

Vendor specific: Size of flash device connected to side A2 of the controller. Set to 0 if no flash device is connected.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.27 Parameter FlsSerialFlashB1Size

Vendor specific: Size of flash device connected to side B1 of the controller. Set to 0 if no flash device is connected.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.28 Parameter FlsSerialFlashB2Size

Vendor specific: Size of flash device connected to side B2 of the controller. Set to 0 if no flash device is connected.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.29 Parameter FlsDdrCentrerAlignReadA

Vendor specific: Set this bit to '1' to enable CK2 output 90 degree phase shifted clock for flash A.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.30 Parameter FlsClockOnDifferentialCknPadA

Vendor specific: Set this bit to '1' to enable clock on differential CKN pad of Flash A.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.31 Parameter FlsDdrCentrerAlignReadB

Vendor specific: Set this bit to '1' to enable CK2 output 90 degree phase shifted clock for flash B.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.32 Parameter FlsClockOnDifferentialCknPadB

Vendor specific: Set this bit to '1' to enable clock on differential CKN pad of Flash A.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.33 Parameter FlsHwUnitSamplingModeA

Vendor specific: It selects DQS clock for sampling read data at Flash A QuadSPI port:

QSPI_IP_READ_MODE_INTERNAL_DQS = DQS internal (Default).

QSPI_IP_READ_MODE_LOOPBACK = Pad loopback.

QSPI_IP_READ_MODE_LOOPBACK_DQS = DQS pad loopback.

QSPI_IP_READ_MODE_EXTERNAL_DQS = External DQS.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_READ_MODE_LOOPBACK
literals	['QSPI_IP_READ_MODE_LOOPBACK', 'QSPI_IP_READ_MODE_EX↩ TERNAL_DQS']

4.34 Parameter FlsHwUnitSamplingModeB

Vendor specific: It selects DQS clock for sampling read data at Flash B QuadSPI port:

QSPI_IP_READ_MODE_INTERNAL_DQS = DQS internal (Default).

QSPI_IP_READ_MODE_LOOPBACK = Pad loopback.

QSPI_IP_READ_MODE_LOOPBACK_DQS = DQS pad loopback.

QSPI_IP_READ_MODE_EXTERNAL_DQS = External DQS.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_READ_MODE_LOOPBACK
literals	['QSPI_IP_READ_MODE_LOOPBACK', 'QSPI_IP_READ_MODE_EX↩ TERNAL_DQS']

4.35 Parameter IdleSignalDriveIOFB3HighLvl

Vendor specific: Idle Signal Drive IOFB[3] Flash B. This bit determines the logic level the IOFB[3] output of the QuadSPI module is driven to in the inactive state.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.36 Parameter IdleSignalDriveIOFB2HighLvl

Vendor specific: Idle Signal Drive IOFB[2] Flash B. This bit determines the logic level the IOFB[2] output of the QuadSPI module is driven to in the inactive state.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.37 Parameter IdleSignalDriveIOFA3HighLvl

Vendor specific: Idle Signal Drive IOFA[3] Flash A. This bit determines the logic level the IOFA[3] output of the QuadSPI module is driven to in the inactive state.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.38 Parameter IdleSignalDriveIOFA2HighLvl

Vendor specific: Idle Signal Drive IOFA[2] Flash A. This bit determines the logic level the IOFA[2] output of the QuadSPI module is driven to in the inactive state.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.39 Parameter FlsHwUnitSamplingEdge

Vendor specific: Full-speed phase selection for SDR instructions.

This field selects the edge of the sampling clock valid for full-speed commands.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_SAMPLE_PHASE_NON_INVERTED
literals	['QSPI_IP_SAMPLE_PHASE_NON_INVERTED', 'QSPI_IP_SAMPLE_↩ PHASE_INVERTED']

4.40 Parameter FlsHwUnitSamplingDly

Vendor specific: Full-speed delay selection for internal/pad loop back DQS sampling.

This field selects the delay in accordance with the reference edge for the valid sample point.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_SAMPLE_DELAY_SAME_DQS
literals	['QSPI_IP_SAMPLE_DELAY_SAME_DQS', 'QSPI_IP_SAMPLE_DELA↩ Y_HALFCYCLE_EARLY_DQS']

4.41 Parameter FlsHwUnitDqsLatencyEnable

Vendor specific: DQS Latency Enable. Feature used to support external devices which add latency cycles

in the DQS signal. When no signal is provided by the external devices, data is not sampled,

thus the memory stretches the timing.

DQS Latency is applicable only for DQS sampling mode: FLS_EXTERNAL_DQS.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.42 Parameter FlsHwUnitTdh

Vendor specific: TDH: Serial flash data in hold time. Should be set to QSPI_IP_FLASH_DATA_ALIGN_REFCLK

for QSPI_IP_DATA_RATE_SDR mode.

TDH parameter delays data sent to flash, in order to meet the input hold time requirement of flash.

QSPI_IP_FLASH_DATA_ALIGN_REFCLK = Data aligned with the posedge of Internal reference clock of Quad-SPI.

QSPI_IP_FLASH_DATA_ALIGN_2X_REFCLK = Data aligned with 2x serial flash half clock.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_FLASH_DATA_ALIGN_REFCLK
literals	['QSPI_IP_FLASH_DATA_ALIGN_REFCLK', 'QSPI_IP_FLASH_DAT↩ A_ALIGN_2X_REFCLK']

4.43 Parameter FlsHwUnitTcsh

Vendor specific: TCSH: Serial flash CS hold time in terms of serial flash clock cycles.

A bigger value will release the CS signal later after the transaction ends.

The actual delay between chip select and clock is defined as:

TCSH = 1 SCK clk if N= 0/1 else, N SCK clk if N>1, where N is the setting of TCSH

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	3
max	15
min	0

4.44 Parameter FlsHwUnitTcss

Vendor specific: TCSS: Serial flash CS setup time in terms of serial flash clock cycles.

A bigger value will pull the CS signal earlier before the transaction starts.

The actual delay between chip select and clock is defined as:

TCSS = 0.5 SCK clk if N= 0/1 else, N+0.5 SCK clk if N>1, where N is the setting of TCSS.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	3
max	15
min	0

4.45 Parameter FlsHwUnitColumnAddressWidth

Vendor specific: Column Address Space. Defines the width of the column address.

Example: If the coulmn address is for example [2:0] of QSPI_SFAR/AHB address,

then CAS must be 3. If there is no column address separation in any

serial flash this value must be programmed to 0.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true

Property	Value
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.46 Parameter FlsHwUnitByteSwapping

Vendor specific: In case of Octal DDR mode, this bit controls whether a word unit composed of two bytes from posedge

and negedge of a single DQS cycle needs to be swapped.

- Disable : One word of two bytes at [nth, n+1th] address.

- Enable : One word of two bytes at [n+1th, nth] address

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.47 Parameter FlsHwUnitWordAddressable

Vendor specific: Defines whether the serial flash is a byte addressable flash or a word addressable flash.

According to this bit configuration the address is re-mapped to the flash interface.

DISABLED: Byte addressable serial flash mode.

ENABLED: Word (2 byte) addressable serial flash mode. If the

incoming address is 0x2004, the controller re-maps this address

to access the flash location 0x1002.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.48 Container FlsAhbBuffer

Container for the configuration of the AHB read buffers. Holds the configuration for each

AHB buffer configured for AHB read mode.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	4
upperMultiplicity	4
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.49 Parameter FlsAhbBufferInstance

Vendor specific: Selects the AHB buffer instance for which this configuration applies.

If an instance is not present, the corresponding AHB buffer will be configured with size 0.

The size of the AHB_BUFFER_3 instance will be configured to at least the selected size, or more, up until the maximum

value is reached. For more details about the maximum avaialable size see chip specific details.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	AHB_BUFFER_1
literals	['AHB_BUFFER_0', 'AHB_BUFFER_1', 'AHB_BUFFER_2', 'AHB_BUF↩ FER_3']

4.50 Parameter FlsAhbBufferMasterId

Vendor specific: The ID of the AHB master associated with this buffer. Any AHB access with this master port number is routed to this buffer. It must be ensured that the master IDs associated with all buffers must be different.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	63
min	0

4.51 Parameter FlsAhbBufferSize

Vendor specific: The size allocated to this AHB Buffer instance. The minimum size is 8 bytes, the maximum size

is the entire AHB Buffer.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.52 Parameter FlsAhbBufferAllMasters

Vendor specific: When set, buffer3 acts as an all-master buffer. Any AHB access with a master port number not matching with the master ID of buffer0 or buffer1 or buffer2 is routed to buffer3.

When set, the Master ID parameter for this buffer is ignored.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.53 Container DllCfgA

Vendor specific: Container for DLL settings for side A
Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.54 Parameter DllCfgADllMode

Vendor specific: Choose the mode of DLL feature for flash A.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_DLL_BYPASSED
literals	['QSPI_IP_DLL_BYPASSED', 'QSPI_IP_DLL_MANUAL_UPDATE', 'QS↩ PI_IP_DLL_AUTO_UPDATE']

4.55 Parameter DllCfgADllCraFreqEn

Vendor specific: Frequency enable for flash A. These are 60-133Mhz (low freq) and 133-200Mhz (high freq)

Disable - Selects delay-chain for low frequency of operation.

Enable - Selects delay-chain for high frequency of operation.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.56 Parameter DllCfgADllCraReferenceCounter

Vendor specific: Select the "n+1" interval of DLL phase detection and reference delay updating interval (minimum recommended value = 1).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	15
min	0

4.57 Parameter DllCfgADllCraResolution

Vendor specific: Minimum resolution for DLL phase detector to remain locked/unlocked based on flash memory clock jitter.

The minimum value is 2, and should be programmed to a more suitable value, such as 6.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2
max	15
min	0

4.58 Parameter DllCfgADllCraSlvFineOffset

Vendor specific: Fine offset delay elements in incoming DQS for flash A

This field sets the number of fine offset delay elements up to 16 in incoming DQS; default should be 1 element

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.59 Parameter DllCfgADllCraSlvDlyOffset

Vendor specific: T/16 offset delay elements in incoming DQS for flash A

This field sets the number of T/16 offset delay elements in incoming DQS; default is 0.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	7
min	0

4.60 Parameter DllCfgADllCraSlvDlyCoarse

Vendor specific: Delay elements in each delay tap for flash A

This field sets the number of delay elements in each delay tap. The field is used to overwrite DLL generated delay values and works in BYPASS Mode.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.61 Parameter DllCfgADllTapSelect

Vendor specific: Selects the nth tap provided by slave delay chain for flash memory A.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	7
min	0

4.62 Container DllCfgB

Vendor specific: Container for DLL settings for side A

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.63 Parameter DllCfgBDllMode

Vendor specific: Choose the mode of DLL feature for flash A.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_DLL_BYPASSED
literals	['QSPI_IP_DLL_BYPASSED', 'QSPI_IP_DLL_MANUAL_UPDATE', 'QS↩ PI_IP_DLL_AUTO_UPDATE']

4.64 Parameter DllCfgBDllCraFreqEn

Vendor specific: Frequency enable for flash A. These are 60-133Mhz (low freq) and 133-200Mhz (high freq)

Disable - Selects delay-chain for low frequency of operation.

Enable - Selects delay-chain for high frequency of operation.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.65 Parameter DllCfgBDllCrbReferenceCounter

Vendor specific: Select the "n+1" interval of DLL phase detection and reference delay updating interval (minimum recommended value = 1).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	15
min	0

4.66 Parameter DllCfgBDllCrbResolution

Vendor specific: Minimum resolution for DLL phase detector to remain locked/unlocked based on flash memory clock jitter.

The minimum value is 2, and should be programmed to a more suitable value, such as 6.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2
max	15
min	0

4.67 Parameter DllCfgBDllCraSlvFineOffset

Vendor specific: Fine offset delay elements in incoming DQS for flash A

This field sets the number of fine offset delay elements up to 16 in incoming DQS; default should be 1 element

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.68 Parameter DllCfgBDllCraSlvDlyOffset

Vendor specific: T/16 offset delay elements in incoming DQS for flash A

This field sets the number of T/16 offset delay elements in incoming DQS; default is 0.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	7
min	0

4.69 Parameter DllCfgBDllCraSlvDlyCoarse

Vendor specific: Delay elements in each delay tap for flash A

This field sets the number of delay elements in each delay tap. The field is used to overwrite DLL generated delay values and works in BYPASS Mode.

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.70 Parameter DllCfgBDllTapSelect

Vendor specific: Selects the nth tap provided by slave delay chain for flash memory A.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	7
min	0

4.71 Container MemCfg

Vendor specific: Container for defining configurations for serial flash devices.

These configurations are not tied to a particular device instance.

Included subcontainers:

- MemCfgReadIdSettings

- MemCfgEraseSettings
- statusConfig
- suspendSettings
- resetSettings
- initResetSettings
- initConfiguration
- FlsLUT

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.72 Parameter MemCfgSize

Vendor specific: The size in bytes of this flash device.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	8388608
max	4294967295
min	0

4.73 Parameter MemCfgPageSize

Vendor specific: The page size in bytes of this flash device.

Page size is the maximum amount of data that the flash device can write in a single write operation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	256
max	4294967295
min	0

4.74 Reference MemCfgReadLUT

Vendor specific: Reference to the LUT Sequence ID which will be used for read operations

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.75 Reference MemCfgWriteLUT

Vendor specific: Reference to the LUT Sequence ID which will be used for write operations

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.76 Reference MemCfgRead0xxLUT

Vendor specific: Reference to the LUT Sequence ID which will be used for optimized read operations, if supported by the device.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.77 Reference MemCfgRead0xxLUTAHB

Vendor specific: Reference to the LUT Sequence ID which will be used for optimized read operations through AHB reads, if supported by the device.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false

Property	Value
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.78 Reference ctrlAutoCfgPtr

Vendor specific: Reference to configuration which will be used for initializing the controller when the flash device is initialized.

This is needed for devices which need to change controller configuration during device initialization (e.g. switch to External DQS after activating DOPI mode).

Resetting the flash device will re-apply this configuration.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/ControllerCfg

4.79 Container MemCfgReadIdSettings

Vendor specific: Container for Read Device/Manufacturer ID command

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.80 Parameter MemCfgReadIdSize

Vendor specific: The size in bytes of the information returned by the readId command.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	3
max	4
min	0

4.81 Parameter FlsQspiDeviceId

Vendor specific: External memory ID. If the associated "FLS_E_UNEXPECTED_FLASH_ID" error is enabled, at Init,

the configured value is checked against the value read from memory.

The memory ID is read from the memory using the configured READ_ID LUT sequence.

Example for a Macronix device:

Configured value of FlsQspiDeviceId = 0x3A:81:C2, meaning Memory density: 0x3A, Memory type: 0x81, Manufacturer ID: 0xC2.

The configured READ_ID LUT sequence schedules a read id command (ex: RDID 0x9F) with read length 3 bytes.

Note: This parameter can be configured only when Read Id LUT index reference is used.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0x3A:81:C2

4.82 Reference MemCfgReadIdLUT

Vendor specific: Reference to the LUT Sequence ID which will be used for reading device/manufacturer Id.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.83 Container MemCfgEraseSettings

Vendor specific: Container for erase commands supported by the device

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.84 Parameter MemCfgErase1Size

Vendor specific: The size in bytes of the erased area: 2 ^ size; e.g. 0x0C means 4 Kbytes

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	12
max	32
min	1

4.85 Parameter MemCfgErase2Size

Vendor specific: The size in bytes of the erased area: 2 ^ size; e.g. 0x0C means 4 Kbytes

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	12
max	32
min	1

4.86 Parameter MemCfgErase3Size

Vendor specific: The size in bytes of the erased area: 2 ^ size; e.g. 0x0C means 4 Kbytes

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	12
max	32
min	1

4.87 Parameter MemCfgErase4Size

Vendor specific: The size in bytes of the erased area: 2 ^ size; e.g. 0x0C means 4 Kbytes

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	12

Property	Value
max	32
min	1

4.88 Reference MemCfgErase1LUT

Vendor specific: Reference to the LUT Sequence ID for erase type 1.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.89 Reference MemCfgErase2LUT

Vendor specific: Reference to the LUT Sequence ID for erase type 2.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.90 Reference MemCfgErase3LUT

Vendor specific: Reference to the LUT Sequence ID for erase type 3.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.91 Reference MemCfgErase4LUT

Vendor specific: Reference to the LUT Sequence ID for erase type 4.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.92 Reference ChipEraseLUT

Vendor specific: Reference to the LUT Sequence ID for chip erase command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.93 Container statusConfig

Vendor specific: Container for settings related to the status register of the flash device

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.94 Parameter regSize

Vendor specific: The size in bytes of the status register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	4
min	1

4.95 Parameter busyOffset

Vendor specific: Position of "busy" bit inside status register. This bit is indicates whether the device is busy with a high voltage operation or not.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	31
min	0

4.96 Parameter busyValue

Vendor specific: Value of "busy" bit which indicates that the device is busy; can be 0 or 1

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	1
min	0

4.97 Parameter writeEnableOffset

Vendor specific: Position of "write enable" bit inside the status register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	31
min	0

4.98 Parameter blockProtectionOffset

Vendor specific: Offset of block protection bits inside the status register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2
max	31
min	0

4.99 Parameter blockProtectionWidth

Vendor specific: Width of block protection bitfield inside the status register

A value of 0 disables protection setting.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	4
max	32
min	0

4.100 Parameter blockProtectionValue

Vendor specific: Value of block protection bitfield inside the status register

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	15
min	0

4.101 Reference statusRegInitReadLut

Vendor specific: Reference to the LUT Sequence ID for Read status register command.

This sequence is used during the initializaton stage.

For example if the initial state of the flash is SPI, this should be a SPI sequence.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.102 Reference statusRegReadLut

Vendor specific: Reference to the LUT Sequence ID for Read status register command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.103 Reference statusRegWriteLut

Vendor specific: Reference to the LUT Sequence ID for Write status register command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.104 Reference writeEnableSRLut

Vendor specific: Reference to the LUT Sequence ID for Status register write enable command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.105 Reference writeEnableLut

Vendor specific: Reference to the LUT Sequence ID for Write enable command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.106 Container suspendSettings

Vendor specific: Container related to write/erase suspend and resume commands, if supported by the device.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.107 Reference eraseSuspendLut

Vendor specific: Reference to the LUT Sequence ID for Erase suspend command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP

Property	Value
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.108 Reference eraseResumeLut

Vendor specific: Reference to the LUT Sequence ID for Erase resume command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.109 Reference programSuspendLut

Vendor specific: Reference to the LUT Sequence ID for Program suspend command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false

Property	Value
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.110 Reference programResumeLut

Vendor specific: Reference to the LUT Sequence ID for Program resume command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.111 Container resetSettings

Vendor specific: Container related to software reset command, for resettings the flash device.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.112 Parameter resetCmdCount

Vendor specific: Number of commands in the reset sequence

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	255
min	1

4.113 Reference resetCmdLut

Vendor specific: Reference to the LUT Sequence ID for the first command from the reset sequence.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.114 Container initResetSettings

Vendor specific: Container related to software reset command, for resettings the flash device. This reset procedure applies only at driver intialization. It might be different from the normal reset command, depending on the initial state of the flash. If not, set the same as reset command.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.115 Parameter resetCmdCount

Vendor specific: Number of commands in the reset sequence

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	255
min	1

4.116 Reference resetCmdLut

Vendor specific: Reference to the LUT Sequence ID for the first command from the reset sequence.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false

Property	Value
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.117 Container initConfiguration

Vendor specific: This container describes the list of operations which must be performed at initialization time to bring the memory in the desired operating state.

Example: activate XPI mode, activate 4-byte addressing.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	255
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.118 Parameter opType

Vendor specific: Operation type can be one of the following:

QSPI_IP_OP_TYPE_CMD - Simple command

QSPI_IP_OP_TYPE_WRITE_REG - Write value in external flash register

QSPI_IP_OP_TYPE_RMW_REG - RMW command on external flash register

QSPI_IP_OP_TYPE_READ_REG - Read external flash register until expected value is read

QSPI_IP_OP_TYPE_QSPI_CFG - Re-configure QSPI controller

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_OP_TYPE_CMD
literals	['QSPI_IP_OP_TYPE_CMD', 'QSPI_IP_OP_TYPE_WRITE_REG', 'QS↩ PI_IP_OP_TYPE_RMW_REG', 'QSPI_IP_OP_TYPE_READ_REG', 'Q↩ SPI_IP_OP_TYPE_QSPI_CFG']

4.119 Parameter addr

Vendor specific: Address, if used in command

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.120 Parameter size

Vendor specific: Size in bytes of configuration register, where it applies.

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	4
min	1

4.121 Parameter shift

Vendor specific: Offset of configuration field, where it applies.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	32
min	0

4.122 Parameter width

Vendor specific: Witdh of configuration field, where it applies.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	32
min	0

4.123 Parameter value

Vendor specific: Value to set/expect in the bit-field.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	4294967295
min	0

4.124 Reference command1Lut

Vendor specific: Index of first command sequence in Lut; for RMW type this is the read command

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.125 Reference command2Lut

Vendor specific: Index of second command sequence in Lut, only used for RMW type, this is the write command.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.126 Reference weLut

Vendor specific: Index of write enable command, if needed before a write command. Only used for Write and RMW operations.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg/FlsLUT

4.127 Reference ctrlCfgPtr

Vendor specific: Reference to configuration which will be used for initializing the controller.

Valid only for QSPI_IP_OP_TYPE_QSPI_CFG operations

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/ControllerCfg

4.128 Container FlsLUT

Vendor specific: Container for the configuration of the Look Up Table holding all the Instruction/Operands sequences.

A sequence consists of a series of up to 8 instruction/operands pairs, which can ocupy up to 4 LUTs,

which are executed whenever a command is triggered to the external flash memory.

Included subcontainers:

- FlsInstructionOperandPair

Property	Value

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	65534
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.129 Parameter FlsLUTIndex

Vendor specific: Fls LUT Index is an invariant index, used to order the LUT entries and loop

over them in the correct, configured order. Its value should be equal with the position of the

configured LUT inside the configured LUT list (the same value as the shown index).

Rationale: The generated .epc configuration might reorder the LUT elements (alphabetically), thus the default index parameter

changes, becoming out of sync with the real intended order (the values are not generated in the intended order, they are sorted).

Range:

min = 0

max = 65534

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	65534
min	0

4.130 Container FlsInstructionOperandPair

Vendor specific: One command set which holds one memory command-operand pair.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.131 Parameter FlsInstrOperPairIndex

Vendor specific: Fls Instruction Operand Pair Index is an invariant index, used to order the Instr.Oper. entries and loop

over them in the correct, configured order. Its value should be equal with the position of the

configured pair inside the configured pair list (the same value as the shown index).

Rationale: The generated .epc configuration might reorder the instr.oper. pairs (alphabetically), thus the index parameter

changes, becoming out of sync with the real intended order (the values are not generated in the intended order, they are sorted).

Range:

min = 0

max = 65534

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	65534
min	0

4.132 Parameter FlsLUTInstruction

Vendor specific: The instruction type used to identify the command used by the QSPI IP when sending the command to the memory.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_LUT_INSTR_CMD
literals	['QSPI_IP_LUT_INSTR_CMD', 'QSPI_IP_LUT_INSTR_ADDR', 'QSPI_↩ IP_LUT_INSTR_DUMMY', 'QSPI_IP_LUT_INSTR_MODE', 'QSPI_IP_↩ LUT_INSTR_MODE2', 'QSPI_IP_LUT_INSTR_MODE4', 'QSPI_IP_LU↩ T_INSTR_READ', 'QSPI_IP_LUT_INSTR_WRITE', 'QSPI_IP_LUT_IN↩ STR_JMP_ON_CS', 'QSPI_IP_LUT_INSTR_ADDR_DDR', 'QSPI_IP_L↩ UT_INSTR_MODE_DDR', 'QSPI_IP_LUT_INSTR_MODE2_DDR', 'QS↩ PI_IP_LUT_INSTR_MODE4_DDR', 'QSPI_IP_LUT_INSTR_READ_D↩ DR', 'QSPI_IP_LUT_INSTR_WRITE_DDR', 'QSPI_IP_LUT_INSTR_D↩ ATA_LEARN', 'QSPI_IP_LUT_INSTR_CMD_DDR', 'QSPI_IP_LUT_I↩ NSTR_CADDR', 'QSPI_IP_LUT_INSTR_CADDR_DDR', 'QSPI_IP_LU↩ T_INSTR_STOP']

4.133 Parameter FlsLUTPad

Vendor specific: Number of pads/pins used for the current command.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_LUT_PADS_1
literals	['QSPI_IP_LUT_PADS_1', 'QSPI_IP_LUT_PADS_2', 'QSPI_IP_LUT_↩ PADS_4', 'QSPI_IP_LUT_PADS_8']

4.134 Parameter FlsLUTOperand

Vendor specific: The operand of the instruction command sent to memory.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	255
min	0

4.135 Container HyperflashCfg

Container for defining configurations for hyper flash devices.

These configurations are not tied to a particular device instance.

Included subcontainers:

- MemCfgReadIdSettings

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.136 Parameter MemCfgSize

Vendor specific: The size in bytes of this flash device.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	8388608
max	4294967295
min	0

4.137 Parameter MemCfgPageSize

Vendor specific: The page size in bytes of this flash device.

Page size is the maximum amount of data that the flash device can write in a single write operation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	512
max	4294967295
min	0

4.138 Parameter outputDriverStrength

Output driver level of the device:

QSPI_IP_HF_DRV_STRENGTH_000 : Typical Impedance for 1.8V: 27; Typical Impedance 3V: 20.

QSPI_IP_HF_DRV_STRENGTH_001 : Typical Impedance for 1.8V: 117; Typical Impedance 3V: 71.

QSPI_IP_HF_DRV_STRENGTH_002 : Typical Impedance for 1.8V: 68; Typical Impedance 3V: 40.

QSPI_IP_HF_DRV_STRENGTH_003 : Typical Impedance for 1.8V: 45; Typical Impedance 3V: 27.

QSPI_IP_HF_DRV_STRENGTH_004 : Typical Impedance for 1.8V: 34; Typical Impedance 3V: 20.

QSPI_IP_HF_DRV_STRENGTH_005 : Typical Impedance for 1.8V: 27; Typical Impedance 3V: 16.

QSPI_IP_HF_DRV_STRENGTH_006 : Typical Impedance for 1.8V: 24; Typical Impedance 3V: 14.

QSPI_IP_HF_DRV_STRENGTH_007 : Typical Impedance for 1.8V: 20; Typical Impedance 3V: 12.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_HF_DRV_STRENGTH_000
literals	['QSPI_IP_HF_DRV_STRENGTH_000', 'QSPI_IP_HF_DRV_STRENG↩ TH_001', 'QSPI_IP_HF_DRV_STRENGTH_002', 'QSPI_IP_HF_DRV_↩ STRENGTH_003', 'QSPI_IP_HF_DRV_STRENGTH_004', 'QSPI_IP_HF↩ _DRV_STRENGTH_005', 'QSPI_IP_HF_DRV_STRENGTH_006', 'QSPI↩ _IP_HF_DRV_STRENGTH_007']

4.139 Parameter RWDSLowOnDualError

Specifies if RWDS will stall upon Dual Error Detect

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.140 Parameter secureRegionUnlocked

If true, the secure silicon region will be unlocked

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.141 Parameter readLatency

Read latency in cycles. Must be set considering the operation frequency:

QSPI_IP_HF_READ_LATENCY_5_CLOCKS : Read latency 5 clocks; max frequency : 52 MHz

QSPI_IP_HF_READ_LATENCY_6_CLOCKS : Read latency 6 clocks; max frequency : 62 MHz

QSPI_IP_HF_READ_LATENCY_7_CLOCKS : Read latency 7 clocks; max frequency : 72 MHz

QSPI_IP_HF_READ_LATENCY_8_CLOCKS : Read latency 8 clocks; max frequency : 83 MHz

QSPI_IP_HF_READ_LATENCY_9_CLOCKS : Read latency 9 clocks; max frequency : 93 MHz

QSPI_IP_HF_READ_LATENCY_10_CLOCKS : Read latency 10 clocks; max frequency : 104 MHz

QSPI_IP_HF_READ_LATENCY_11_CLOCKS : Read latency 11 clocks; max frequency : 114 MHz

QSPI_IP_HF_READ_LATENCY_12_CLOCKS : Read latency 12 clocks; max frequency : 125 MHz

QSPI_IP_HF_READ_LATENCY_13_CLOCKS : Read latency 13 clocks; max frequency : 135 MHz

QSPI_IP_HF_READ_LATENCY_14_CLOCKS : Read latency 14 clocks; max frequency : 145 MHz

QSPI_IP_HF_READ_LATENCY_15_CLOCKS : Read latency 15 clocks; max frequency : 156 MHz

QSPI_IP_HF_READ_LATENCY_16_CLOCKS : Read latency 16 clocks; max frequency : 166 MHz

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_HF_READ_LATENCY_16_CLOCKS
literals	['QSPI_IP_HF_READ_LATENCY_5_CLOCKS', 'QSPI_IP_HF_READ_↩ LATENCY_6_CLOCKS', 'QSPI_IP_HF_READ_LATENCY_7_CLOCKS', 'QSPI_IP_HF_READ_LATENCY_8_CLOCKS', 'QSPI_IP_HF_READ_↩ LATENCY_9_CLOCKS', 'QSPI_IP_HF_READ_LATENCY_10_CLOCKS', 'QSPI_IP_HF_READ_LATENCY_11_CLOCKS', 'QSPI_IP_HF_READ_↩ LATENCY_12_CLOCKS', 'QSPI_IP_HF_READ_LATENCY_13_CLOC↩ KS', 'QSPI_IP_HF_READ_LATENCY_14_CLOCKS', 'QSPI_IP_HF_RE↩ AD_LATENCY_15_CLOCKS', 'QSPI_IP_HF_READ_LATENCY_16_C↩ LOCKS']

4.142 Parameter paramSectorMap

Define the mapping of the 4-KB Parameter Sectors:

QSPI_IP_HF_PARAM_AND_PASSWORD_MAP_LOW : Parameter-Sectors and Read Password Sectors mapped into lowest addresses

QSPI_IP_HF_PARAM_AND_PASSWORD_MAP_HIGH : Parameter-Sectors and Read Password Sectors mapped into highest addresses

QSPI_IP_HF_UNIFORM_SECTORS_READ_PASSWORD_LOW : Uniform Sectors with Read Password Sector mapped into lowest addresses

QSPI_IP_HF_UNIFORM_SECTORS_READ_PASSWORD_HIGH : Uniform Sectors with Read Password Sector mapped into highest addresses

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_HF_UNIFORM_SECTORS_READ_PASSWORD_LOW
literals	['QSPI_IP_HF_PARAM_AND_PASSWORD_MAP_LOW', 'QSPI_IP_H↩ F_PARAM_AND_PASSWORD_MAP_HIGH', 'QSPI_IP_HF_UNIFORM↩ _SECTORS_READ_PASSWORD_LOW', 'QSPI_IP_HF_UNIFORM_SE↩ CTORS_READ_PASSWORD_HIGH']

4.143 Reference ctrlAutoCfgPtr

Vendor specific: Reference to configuration which will be used for initializing the controller when the flash device is initialized.

This is needed for devices which need to change controller configuration during device initialization (e.g. switch to External DQS after activating DOPI mode).

Resetting the flash device will re-apply this configuration.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true

Property	Value
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/ControllerCfg

4.144 Container MemCfgReadIdSettings

Vendor specific: Container for Read Device/Manufacturer ID command

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.145 Parameter MemCfgReadIdLUT

Vendor specific: Reference to the LUT Sequence ID which will be used for reading device/manufacturer Id.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	False
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_HF_LUT_READ
literals	['QSPI_IP_HF_LUT_READ']

4.146 Parameter MemCfgReadIdWordAddr

Vendor specific: The word address of the device ID in ASO.

This value will be converted to by address for the read ID transaction.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.147 Parameter MemCfgReadIdSize

Vendor specific: The size in bytes of the information returned by the readId command.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	3
max	4
min	0

4.148 Parameter FlsQspiDeviceId

Vendor specific: External memory ID. If the associated "FLS_E_UNEXPECTED_FLASH_ID" error is enabled, at Init,

the configured value is checked against the value read from memory.

The memory ID is read from the memory using the configured READ_ID LUT sequence.

Example for a Macronix device:

Configured value of FlsQspiDeviceId = 0x3A:81:C2, meaning Memory density: 0x3A, Memory type: 0x81, Manufacturer ID: 0xC2.

The configured READ_ID LUT sequence schedules a read id command (ex: RDID 0x9F) with read length 3 bytes.

Note: This parameter can be configured only when Read Id LUT index reference is used.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0x3A:81:C2

4.149 Container FlsController

Container for selecting the start configuration of QSPI controllers.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.150 Parameter ControllerName

Vendor specific: The name of the configured harwdare unit name. The configured parameters will apply to this hardware unit name only.

The name of the hardware unit name represents the physical hardware unit available on chip.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FLS_QSPI_0
literals	['FLS_QSPI_0']

4.151 Reference FlsControllerCfgRef

Vendor specific: Reference to configuration which will be used for initializing the controller.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/ControllerCfg

4.152 Container FlsMem

Container for selecting the start configuration and connection information of serial flash devices.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.153 Parameter FlsMemName

Vendor specific: The name of the configured flash device. The configured parameters will apply to this device only.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	Device_0

4.154 Parameter MemAlignment

Vendor specific: The address alignment required by the external flash (1, 2 or 4 bytes, ...), needed in the OCTA DTR Mode (DOPI) or hyperbus devices.

For read operation:

- The driver will decrease the address if it is not aligned, and increasing the size to compensate.

- After the actual read, the driver ignores the first few bytes before starting the copy/comparison to the user data.

For write operation: send extra data with FFh to overwrite the overlapping memory area

? If there is a need to program from odd starting address, keep the even input address and the input data shall start with FFh.

? If there is a need to program with odd ending address, simply provide extra data with FFh in the last falling edge of clock.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	16
min	1

4.155 Parameter AHBReadEnable

Vendor specific: When set, Qspi_Ip_AhbReadEnable() will be called from Fls_Init() to allow reads via AHB.

The application can read directly through Flash memory devices address mapping.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.156 Parameter FlsMemUseSfdp

Vendor specific: Select this option to attempt auto-configuration using the information read from the SFDP table

This only works for flash devices which support the SFDP feature.

SFDP (Serial Flash Discoverable Parameters) is a JEDEC standard - JESD216D.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.157 Parameter connectionType

Vendor specific: Connection type of the flash device to the controller:

QSPI_IP_SIDE_A1 - Serial flash connected on side A1

QSPI_IP_SIDE_A2 - Serial flash connected on side A2

QSPI_IP_SIDE_B1 - Serial flash connected on side B1

QSPI_IP_SIDE_B2 - Serial flash connected on side B2

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	QSPI_IP_SIDE_A1
literals	['QSPI_IP_SIDE_A1', 'QSPI_IP_SIDE_A2', 'QSPI_IP_SIDE_B1', 'QSPI↩ _IP_SIDE_B2']

4.158 Reference FlsMemCfgRef

Vendor specific: Reference to configuration which will be used for initializing the flash memory device.

Property	Value
type	ECUC-CHOICE-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destinations	['/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/MemCfg', '/TS↩ _T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/HyperflashCfg']

4.159 Reference qspiInstance

Vendor specific: QSPI controller instance to which this flash device is connected.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/FlsController

4.160 Container FlsSectorList

List of flashable sectors and pages.

Included subcontainers:

- FlsSector

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.161 Container FlsSector

Configuration description of a flashable sector

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD

4.162 Parameter FlsSectorIndex

Vendor specific: Fls Sector Index is an invariant index, used to order flash sectors and loop over them in the correct, configured order. Its value should be equal with the position of the configured sector inside the configured sector list (the same value as the shown index).

Rationale: The generated .epc configuration might reorder the flash sectors(alphabetically), thus the index parameter changes, becoming out of sync with the real intended order (for example: Fls Sector Start Addresses).

Range:

min = 0

max = 65534

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	65534
min	0

4.163 Parameter FlsPhysicalSector

Vendor specific: Physical flash device sector.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FLS_EXT_SECTOR
literals	['FLS_EXT_SECTOR']

4.164 Parameter FlsNumberOfSectors

Number of continuous sectors with the above characteristics.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	65535
min	1

4.165 Parameter FlsPageSize

Size of one page of this sector. Implementation Type: Fls_LengthType.

For internal flash, page size is 8 byte

For external flash, page size is chip specific.

For example: In Macronix devices, the ECC algorithm uses a Hamming code that can correct a single bit error per 16-Byte page.

It is recommended that data be programmed in multiples of 16 bytes using the Page Program command instead of programming a byte or a word at a time using the Program command.

Each group of 16 bytes must fall within the same 16-Byte boundary.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	8
max	4294967295
min	0

4.166 Parameter FlsSectorSize

Size of this sector. Implementation Type: Fls_LengthType.

Note: Size of the sector should be a multiple of the page size.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	4096
max	4294967295
min	0

4.167 Parameter FlsSectorStartaddress

Start address of this sector.

Implementation Type: Fls_AddressType.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.168 Parameter FlsSectorEraseAsynch

Vendor specific: Enable asynchronous execution of the erase job in the Fls_MainFunction function which doesn't wait (block)

for completion of the sector erase operation. The flash driver doesn't use the erase access code to the erase flash sector

in asynchronous mode so it can be used only on flash sectors which belong to flash array different from flash array the

application is executing from.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.169 Parameter FlsPageWriteAsynch

Vendor specific: Enable asynchronous execution of the write job in the Fls_MainFunction function which doesn't wait (block)

for completion of the page write operation(s). The flash driver doesn't use the write access code to the write flash page(s)

in asynchronous mode so it can be used only on flash sectors which belong to flash array different from flash array the

application is executing from.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.170 Parameter FlsHwCh

Vendor specific: The hardware channel type of the current sector: internal flash or external QSPI flash sector.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	FLS_CH_QSPI
literals	['FLS_CH_QSPI']

4.171 Parameter FlsSectorHwAddress

Vendor specific: Hardware address of this sector, as needed by the external flash device (usually starting from 0).

Applicable only to external sectors. This value is used to access the hardware sector on the attached device

and will be sent as parameter of flash comands, so it should be completed to meet the requirements of the

external flash memory type and configured operating mode.

Internally, this address is added to the MCU base addresses of each channel, configured in SF{A/B}{1/2}AD registers, in order

to select the corresponding external device hw channel.

Example: FlsSectorHwAddress = 0x100

Sector hardware channel = FLS_CH_EXTERN_QSPI_0_A2

FlsSerialFlashA1TopAddr = 0x24000000

The address used by the driver internally will be 0x24000100, thus selecting external

flash device A2 and accessing internal location 0x100 of the memory.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.172 Reference flashInstance

Vendor specific: External flash device instance to which this sector belongs.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/TS_T40D11M40I2R0/Fls/FlsConfigSet/FlsExternalDriver/FlsMem

4.173 Container AutosarExt

Vendor specific: This container contains the global Non-Autosar configuration parameters of the Fls driver.
This container is a MultipleConfigurationContainer, i.e. this container and
its sub-containers exist once per configuration set.
Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.174 Parameter FlsEnableUserModeSupport

Vendor specific: When this parameter is enabled, the FLS module will adapt to run from User Mode, with the following measures:

configuring REG_PROT for Fls IPs so that the registers under protection can be accessed from user mode by setting UAA bit in REG_PROT_GCR to 1

for more information and availability on this platform, please see chapter User Mode Support in IM

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.175 Parameter FlsQspiLockLUT

Vendor specific: Enable the Lock/Unlock of the LUT for the external QuadSPI memory.

If Enabled, the LUT is unlocked at the beginning of the Init phase and locked at the end of it.

If Disabled, the LUT has to be unlocked if the Init phase is supposed to populate it.

Note: not used in the driver code.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.176 Parameter FlsQspiHangRecovery

Vendor specific: Enable the hang recovery feature for the external QuadSPI controller.

If Enabled, the driver will perform the software reset sequence in case of being stuck in BUSY state.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.177 Parameter FlsSynchronizeCache

Vendor specific:

Synchronize the memory by invalidating the cache after each flash hardware operation.

The FLS driver needs to maintain the memory coherency by means of three methods:

1. Disable data cache, or

2. Configure the flash region upon which the driver operates, as non-cacheable, or

3. Enable the FlsSynchronizeCache feature.

Depending on the application configuration, one option may be more beneficial than other.

Enabled: The FLS driver will call Mcl cache API functions in order to invalidate the cache

after each high voltage operation(write,erase) and before each read operation, in order

to ensure that the cache and the modified flash memory are in sync.

If enabled, the driver will attempt to invalidate only the modified lines from the cache.

If the size of the region to be invalidated is greater than half of the cache size, then

the entire cache is invalidated.

Note: If enabled, the MclLmemEnableCacheApi parameter has to be enabled and the MCL plugin included as a dependency.

Disabled: The upper layers have to ensure that the flash region upon which the driver operates is not cached.

This can be obtained by either disabling the data cache or by configuring the memory region as non-cacheable.

Note: This feature is applicable only if supported on the current platform.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.178 Parameter FlsMCoreEnable

Vendor specific:

Enable the multicore synchronization feature.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.179 Parameter FlsMCoreQJobSemaphoreChannelNo

Vendor specific:

The channel number of the multicore semaphore used for requesting an external job access.

The channel number should be passed to the MCL gate APIs.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.180 Parameter FlsExternalSectorsConfigured

Vendor specific:

Boolean parameter which must be enabled if external flash sectors are configured.

Enabled: At least one external flash sector is configured in any variant.

Disabled: No external flash sector is configured in any variant, only internal flash sectors are present.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.181 Container FlsGeneral

Container for general parameters of the flash driver. These parameters are always pre-compile.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.182 Parameter FlsEnableDevAssert

Vendor specific:

true: Development error checking at IP level is enabled.

false: Development error checking at IP level is disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.183 Parameter FlsEnableCheckCfgCrc

Vendor specific:

true: Enable calculates CRC for Fls Configuration

false: Disable calculates CRC for Fls Configuration.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.184 Parameter FlsAcLoadOnJobStart

The flash driver shall load the flash access code to RAM whenever an erase or write job is started and unload (overwrite) it after that job has been finished or canceled.

true: Flash access code loaded on job start / unloaded on job end or error.

false: Flash access code not loaded to / unloaded from RAM at all.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.185 Parameter FlsBaseAddress

The flash memory start address (see also FLS118).

FLS169: This parameter defines the lower boundary for read / write / erase and compare jobs.Note:Not needed / supported by the driver.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.186 Parameter FlsBlankCheckApi

Compile switch to enable and disable the Fls_BlankCheck function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.187 Parameter FlsCancelApi

Compile switch to enable and disable the Fls_Cancel function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.188 Parameter FlsCompareApi

Compile switch to enable and disable the Fls_Compare function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.189 Parameter FlsDevErrorDetect

Pre-processor switch to enable and disable development error detection.

true: Development error detection enabled.

false: Development error detection disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.190 Parameter FlsDriverIndex

Index of the driver, used by FEE.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC

Property	Value
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	254
min	0

4.191 Parameter FlsGetJobResultApi

Compile switch to enable and disable the Fls_GetJobResult function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.192 Parameter FlsGetStatusApi

Compile switch to enable and disable the Fls_GetStatus function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.193 Parameter FlsSetModeApi

Compile switch to enable and disable the Fls_SetMode function.

true: API supported / function provided.

false: API not supported / function not provided

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.194 Parameter FlsTotalSize

The total amount of flash memory in bytes (see also FLS118). FLS170: This parameter in conjunction with FLS_BASE_ADDRESS defines the upper boundary for read / write / erase and compare jobs.

Note:Not needed / supported by the driver.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4294967295
min	0

4.195 Parameter FlsUseInterrupts

Not supported by the driver.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.196 Parameter FlsVersionInfoApi

Pre-processor switch to enable / disable the API to read out the modules version information.

true: Version info API enabled.

false: Version info API disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.197 Parameter FlsEraseVerificationEnabled

Pre-processor switch to enable / disable the erase blank check. After a flash block has been erased, the erase blank check compares the contents of the addressed memory area against the value of an erased flash cell to check that the block has been completely erased.

true: Memory region is checked to be erased.

false: Memory region is not checked to be erased.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.198 Parameter FlsWriteVerificationEnabled

Pre-processor switch to enable / disable the write verify check. After writing a flash block, the write verify check compares the contents of the reprogrammed memory area against the contents of the provided application buffer to check that the block has been completely reprogrammed.

true: Written data is compared directly after write.

false: Written date is not compared directly after write.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.199 Parameter FlsMaxEraseBlankCheck

Vendor specific: The maximum number of bytes to blank check in one cycle of the flash driver's job processing function. Affects only the flash blocks that have enabled asynchronous execution of the erase job (FlsSectorEraseAsynch=true).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	256
max	65536
min	8

4.200 Parameter FlsTimeoutSupervisionEnabled

Compile switch to enable timeout supervision.

true: timeout supervision for read/erase/write/compare jobs enabled.

false: timeout supervision for read/erase/write/compare jobs disabled.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.201 Parameter FlsTimeoutMethod

Vendor specific: Counter type used in timeout detection for FLS service request.

Based on selected counter type the timeout value will be interpreted as follows:

OSIF_COUNTER_DUMMY - Counts the number of iterations of the waiting loop. The actual timeout depends on many factors: operation type, compiler optimizations, interrupts or other tasks in the system, etc.

OSIF_COUNTER_SYSTEM - Microseconds.

OSIF_COUNTER_CUSTOM - Defined by user implementation of timing services

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COU↩ NTER_CUSTOM']

4.202 Parameter FlsQspiIpTimeoutOsifCounterType

Vendor specific: Counter type used in timeout detection for QSPI operations.

Based on selected counter type the timeout value will be interpreted as follows:

OSIF_COUNTER_DUMMY - Counts the number of iterations of the waiting loop. The actual timeout depends on many factors: operation type, compiler optimizations, interrupts or other tasks in the system, etc.

OSIF_COUNTER_SYSTEM - Microseconds.

OSIF_COUNTER_CUSTOM - Defined by user implementation of timing services

Note: Qspi always uses timeout

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COU↩ NTER_CUSTOM']

4.203 Parameter FlsQspiSyncReadTimeout

Vendor specific: Fls Qspi Sync Read Timeout is the timeout value for read operation.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE

Property	Value
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2147483647
max	2147483647
min	0

4.204 Parameter FlsQspiAsyncWriteTimeout

Vendor specific: Fls Qspi Async Write Timeout is the timeout value for QSPI write operation in asynchronous mode.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2147483647
max	2147483647
min	0

4.205 Parameter FlsQspiAsyncEraseTimeout

Vendor specific: Fls Qspi Async Erase Timeout is the timeout value for QSPI erase operation in asynchronous mode.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2147483647

Property	Value
max	2147483647
min	0

4.206 Parameter FlsQspiSyncWriteTimeout

Vendor specific: Fls Qspi Sync Write Timeout is the timeout value for QSPI write operation in synchronous mode.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2147483647
max	2147483647
min	0

4.207 Parameter FlsQspiSyncEraseTimeout

Vendor specific: Fls Qspi Sync Erase Timeout is the timeout value for QSPI erase operation in synchronous mode.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2147483647
max	2147483647
min	0

4.208 Parameter FlsQspiDllLockTimeout

Vendor specific: Fls Qspi DLL Lock Timeout is the timeout value for waiting DLL lock bit.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	2147483647
max	2147483647
min	0

4.209 Parameter FlsQspiCommandCompleteTimeout

Vendor specific: Fls Qspi Command Complete Timeout is the timeout value for waiting for a QSPI command to be completed.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	2147483647
min	0

4.210 Parameter FlsQspiResetTimeout

Vendor specific: Fls Qspi Reset Timeout is the timeout for waiting for the external device to become available after

a software reset.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	2147483647
min	0

4.211 Parameter FlsQspiFlashInitTimeout

Vendor specific: Fls Flash Init Timeout is the timeout for completing the initializtion operation sequence for the external flash at startup.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1
max	2147483647
min	0

4.212 Parameter FlsQspiSoftwareResetDelay

Vendor specific: Fls Qspi Reset Delay is the waiting time after changing the value of the QSPI software reset bits in MCR register.

Note: The default value is calculated in the number of CPU cycles for the worst case scenario (with maximum possible CPU frequency and minimum possible flash clock frequency).

See the note of MCR register of QSPI chapter in Reference manual for more information.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	276
max	2147483647
min	0

4.213 Parameter FlsQspiTxBufferResetDelay

Vendor specific: Fls Qspi TX Buffer Reset Delay is the waiting time after changing the value of the QSPI TX FIFO/buffer reset bits in MCR register.

Note: The default value is calculated in the number of CPU cycles for the worst case scenario (with maximum possible CPU frequency and minimum possible flash clock frequency).

See the note of MCR register of QSPI chapter in Reference manual for more information.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	430
max	2147483647
min	0

4.214 Parameter FlsQspiWriteEnableRetries

Vendor specific: Number of attempts when sending the Write Enable command to the external flash.

The driver will read back the status register after each attempt and check the Busy bit.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	3
max	100
min	0

4.215 Parameter FlsMCoreArbitrationTimeout

Vendor specific: Fls MultiCore Arbitration Timeout Time is the timeout value after which, if the Job Semaphore is not

acquired, the job is aborted.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	32767
max	2147483647
min	0

4.216 Parameter FlsMCoreInitTimeout

Vendor specific: Fls Multi Core Initialization Timeout is the timeout value for aborting the drive initialization.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	32767
max	2147483647
min	0

4.217 Reference FlsEcucPartitionRef

Maps the Flash driver to zero or one ECUC partition to make the driver API available in this partition.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.218 Container FlsPublishedInformation

Additional published parameters not covered by CommonPublishedInformation container.

Note that these parameters do not have any configuration class setting, since they are published information.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.219 Parameter FlsAcLocationErase

Position in RAM, to which the erase flash access code has to be loaded.

Only relevant if the erase flash access code is not position independent. If this information is not provided it is assumed that the erase flash access code is position independent and that therefore the RAM position can be freely configured.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	4294967295
min	0

4.220 Parameter FlsAcLocationWrite

Vendor specific: Position in RAM, to which the write flash access code has to be loaded.

Only relevant if the write flash access code is not position independent. If this information is not provided it is assumed that the write flash access code is position independent and that therefore the RAM position can be freely configured.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	4294967295
min	0

4.221 Parameter FlsAcSizeErase

Number of bytes in RAM needed for the erase flash access code.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	4294967295
min	0

4.222 Parameter FlsAcSizeWrite

Number of bytes in RAM needed for the write flash access code.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	4294967295
min	0

4.223 Parameter FlsEraseTime

Maximum time to erase one complete flash sector [sec].

Note:This value can be found on DS as the maximum erase time occurs after the specified number of program/erase cycles .

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	5.0
max	5.0
min	0.0

4.224 Parameter FlsErasedValue

The contents of an erased flash memory cell.

Property	Value
type	ECUC-INTEGER-PARAM-DEF

Property	Value
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4294967295
max	4294967295
min	0

4.225 Parameter FlsECCValue

Vendor specific: The contents of an ECC flash memory line.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4294967295
max	4294967295
min	0

4.226 Parameter FlsExpectedHwId

Unique identifier of the hardware device that is expected by this driver (the device for which this driver has been implemented).

Only relevant for external flash drivers.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0

4.227 Parameter FlsSpecifiedEraseCycles

Number of erase cycles specified for the flash device (usually given in the device data sheet).

FLS198: If the number of specified erase cycles depends on the operating environment (temperature, voltage, ...) during reprogramming of the flash device, the minimum number for which a data retention of at least 15 years over the temperature range from -40C .. +125C can be guaranteed shall be given.

Note:If there are different numbers of specified erase cycles for different flash sectors of the device this parameter has to be extended to a parameter list (similar to the sector list above).

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	100000
max	4294967295
min	0

4.228 Parameter FlsWriteTime

Maximum time to program one complete flash page [sec].

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	5.0E-4
max	5.0E-4
min	0.0

4.229 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.230 Parameter ArReleaseMajorVersion

Vendor specific: Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.231 Parameter ArReleaseMinorVersion

Vendor specific: Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.232 Parameter ArReleaseRevisionVersion

Vendor specific: Patch version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.233 Parameter ModuleId

Vendor specific: Module ID of this module.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	92
max	92
min	92

4.234 Parameter SwMajorVersion

Vendor specific: Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.235 Parameter SwMinorVersion

Vendor specific: Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.236 Parameter SwPatchVersion

Vendor specific: Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	2
max	2
min	2

4.237 Parameter VendorApiInfix

Vendor specific: In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_<VendorId>_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.238 Parameter VendorId

Vendor specific: Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Chapter 6

Module Documentation

6.1 FLS Driver

6.1.1 Detailed Description

Data Structures

- struct Fls_QspiCfgConfigType
  Fls Qspi CfgConfig Type. *More...*
- struct Fls_ConfigType
  Fls Config Type. *More...*

Macros

- #define FLS_DEVICE_INSTANCE_INVALID
- #define FLS_API_VENDOR_ID
  Version Check parameters.
- #define FLS_MODULE_ID
  AUTOSAR module identification.
- #define FLS_INSTANCE_ID
  AUTOSAR module instance identification.
- #define FLS_E_PARAM_CONFIG
  Development error codes (passed to DET).
- #define FLS_E_PARAM_ADDRESS
  API service called with wrong address parameter.
- #define FLS_E_PARAM_LENGTH
  API service called with wrong length parameter.
- #define FLS_E_PARAM_DATA
  API service called with wrong data parameter.
- #define FLS_E_UNINIT
  API service called without module initialization.

- #define FLS_E_BUSY
  API service called while driver still busy.
- #define FLS_E_PARAM_POINTER
  API service called with NULL pointer.
- #define FLS_E_VERIFY_ERASE_FAILED
  Runtime error codes (passed to DET).
- #define FLS_E_VERIFY_WRITE_FAILED
  Write verification (compare) failed.
- #define FLS_E_TIMEOUT
  Timeout exceeded.
- #define FLS_E_ERASE_FAILED
  Transient Faults codes (passed to DET).
- #define FLS_E_WRITE_FAILED
  Flash write failed (HW)
- #define FLS_E_READ_FAILED
  Flash read failed (HW)
- #define FLS_E_COMPARE_FAILED
  Flash compare failed (HW)
- #define FLS_INIT_ID
  All service IDs (passed to DET).
- #define FLS_ERASE_ID
  service ID of function: Fls_Erase. (passed to DET)
- #define FLS_WRITE_ID
  service ID of function: Fls_Write. (passed to DET)
- #define FLS_CANCEL_ID
  service ID of function: Fls_Cancel. (passed to DET)
- #define FLS_GETJOBRESULT_ID
  service ID of function: Fls_GetJobResult. (passed to DET)
- #define FLS_MAINFUNCTION_ID
  service ID of function: Fls_MainFunction. (passed to DET)
- #define FLS_READ_ID
  service ID of function: Fls_Read. (passed to DET)
- #define FLS_COMPARE_ID
  service ID of function: Fls_Compare. (passed to DET)
- #define FLS_SETMODE_ID
  service ID of function: Fls_SetMode. (passed to DET)
- #define FLS_GETVERSIONINFO_ID
  service ID of function: Fls_GetVersionInfo. (passed to DET)
- #define FLS_BLANK_CHECK_ID
  service ID of function: Fls_BlankCheck. (passed to DET)
- #define FLS_SECTOR_ERASE_ASYNCH
  All sector flags.
- #define FLS_PAGE_WRITE_ASYNCH
  fls page write asynch
- #define FLS_START_SEC_CODE
  Start of Fls section CODE.
- #define FLS_STOP_SEC_CODE
  Stop of Fls section CODE.

Types Reference

- typedef uint32 Fls_SectorIndexType
  Logical sector index.
- typedef uint32 Fls_AddressType
  Fls Address Type.
- typedef uint32 Fls_LengthType
  Fls Length Type.
- typedef uint32 Fls_SectorCountType
  Fls Sector Count Type.
- typedef void(∗ Fls_JobEndNotificationPtrType) (void)
  Fls Job End Notification Pointer Type.
- typedef void(∗ Fls_JobErrorNotificationPtrType) (void)
  Fls Job Error Notification Pointer Type.
- typedef void(∗ Fls_MCoreTimeoutNotifPtrType) (Fls_MCoreTimeoutJobType eMCoreTimeoutJob)
  Fls Multi Core Notification Pointer Type.

Enum Reference

- enum Fls_HwChType
  Flash sector channel type.
- enum Fls_JobType
  Type of job currently executed by Fls_MainFunction.
- enum Fls_LLDReturnType
  Result of low-level flash operation.
- enum Fls_LLDJobType
  Type of job currently executed by Fls_LLDMainFunction.
- enum Fls_CrcDataSizeType
  Size of data to be processeed by CRC.
- enum Fls_MCoreReqReturnType
  Fls Multi Core Request Return Type.
- enum Fls_MCoreHwJobStatusType
  Fls Multi Core hardware job status.
- enum Fls_MCoreTimeoutJobType
  Fls Multi Core timeout notification jobs.

Function Reference

- void Fls_Init (const Fls_ConfigType ∗ConfigPtr)
  The function initializes Fls module.
- Std_ReturnType Fls_Write (Fls_AddressType TargetAddress, const uint8 ∗SourceAddressPtr, Fls_LengthType Length)
  Write one or more complete flash pages to the flash device.
- Std_ReturnType Fls_Erase (Fls_AddressType TargetAddress, Fls_LengthType Length)
  Erase one or more complete flash sectors.
- Std_ReturnType Fls_Read (Fls_AddressType SourceAddress, uint8 ∗TargetAddressPtr, Fls_LengthType Length)
  Reads from flash memory.

Variables

- Fls_AddressType Fls_u32JobAddrIt
  Logical address of data block currently processed by Fls_MainFunction.
- Fls_AddressType Fls_u32JobAddrEnd
  Last logical address to be processed by a job.
- volatile Fls_SectorIndexType Fls_u32JobSectorIt
  Index of flash sector currently processed by a job.
- Fls_SectorIndexType Fls_u32JobSectorEnd
  Index of last flash sector by current job.
- volatile MemIf_JobResultType Fls_eLLDJobResult
  Result of last flash hardware job.
- Fls_LLDJobType Fls_eLLDJob
  Type of current flash hardware job - used for asynchronous operating mode.
- const Fls_ConfigType * Fls_pConfigPtr
  Pointer to current flash module configuration set.

6.1.2 Data Structure Documentation

6.1.2.1 struct Fls_QspiCfgConfigType

Fls Qspi CfgConfig Type.

Fls Qspi CfgConfig Type

Definition at line 360 of file Fls_Types.h.

Data Fields

- const uint8(* u8SectFlashUnit)[]
  External flash unit assigned to each sector. Size: u32SectorCount.
- const uint8 u8FlashUnitsCount
  Number of serial flash instances.
- const Qspi_Ip_MemoryConnectionType(* paFlashConnectionCfg)[]
  Connection for each external memory device to available controllers. Size: u8FlashUnitsCount.
- const uint8(* u8FlashConfig)[]
  Configuration index used for each flash unit. Size: u8FlashUnitsCount.
- const boolean(* paAHBReadCfg)[]
  AHB direct reads configurations. Size: u8FlashUnitsCount.
- const uint8 u8FlashConfigCount
  Number of serial flash configurations.
- const Qspi_Ip_MemoryConfigType(* paFlashCfg)[]
  External memory devices configurations. Size: u8FlashConfigCount.
- const uint8 u8QspiUnitsCount
  Number of QSPI hardware instances.

- const uint8(* u8QspiInstance)[]

  Hardware instances for each QSPI unit. Size: u8QspiUnitsCount.
- const uint8(* u8QspiConfig)[]

  Configuration for each QSPI unit. Size: u8QspiUnitsCount.
- const uint8 u8QspiConfigCount

  Number of QSPI configurations.
- const Qspi_Ip_ControllerConfigType(* paQspiUnitCfg)[]

  QSPI configurations. Size: u8QspiConfigCount.

6.1.2.1.1 Field Documentation

6.1.2.1.1.1 u8SectFlashUnit `const uint8(* u8SectFlashUnit)[]`

External flash unit assigned to each sector. Size: u32SectorCount.

Definition at line 365 of file Fls_Types.h.

6.1.2.1.1.2 u8FlashUnitsCount `const uint8 u8FlashUnitsCount`

Number of serial flash instances.

Definition at line 369 of file Fls_Types.h.

6.1.2.1.1.3 paFlashConnectionCfg `const Qspi_Ip_MemoryConnectionType(* paFlashConnectionCfg)[]`

Connection for each external memory device to available controllers. Size: u8FlashUnitsCount.

Definition at line 373 of file Fls_Types.h.

6.1.2.1.1.4 u8FlashConfig `const uint8(* u8FlashConfig)[]`

Configuration index used for each flash unit. Size: u8FlashUnitsCount.

Definition at line 377 of file Fls_Types.h.

6.1.2.1.1.5 paAHBReadCfg `const boolean(* paAHBReadCfg)[]`

AHB direct reads configurations. Size: u8FlashUnitsCount.

Definition at line 381 of file Fls_Types.h.

6.1.2.1.1.6 u8FlashConfigCount `const uint8 u8FlashConfigCount`

Number of serial flash configurations.

Definition at line 386 of file Fls_Types.h.

6.1.2.1.1.7 paFlashCfg `const Qspi_Ip_MemoryConfigType(* paFlashCfg)[]`

External memory devices configurations. Size: u8FlashConfigCount.

Definition at line 390 of file Fls_Types.h.

6.1.2.1.1.8 u8QspiUnitsCount `const uint8 u8QspiUnitsCount`

Number of QSPI hardware instances.

Definition at line 395 of file Fls_Types.h.

6.1.2.1.1.9 u8QspiInstance `const uint8(* u8QspiInstance)[]`

Hardware instances for each QSPI unit. Size: u8QspiUnitsCount.

Definition at line 399 of file Fls_Types.h.

6.1.2.1.1.10 u8QspiConfig `const uint8(* u8QspiConfig)[]`

Configuration for each QSPI unit. Size: u8QspiUnitsCount.

Definition at line 403 of file Fls_Types.h.

6.1.2.1.1.11 u8QspiConfigCount `const uint8 u8QspiConfigCount`

Number of QSPI configurations.

Definition at line 407 of file Fls_Types.h.

6.1.2.1.1.12 paQspiUnitCfg `const Qspi_Ip_ControllerConfigType(* paQspiUnitCfg)[]`

QSPI configurations. Size: u8QspiConfigCount.

Definition at line 411 of file Fls_Types.h.

6.1.2.2 struct Fls_ConfigType

Fls Config Type.

Fls module initialization data structure

Definition at line 419 of file Fls_Types.h.

Data Fields

- Fls_JobEndNotificationPtrType jobEndNotificationPtr
  pointer to job end notification function
- Fls_JobErrorNotificationPtrType jobErrorNotificationPtr
  pointer to job error notification function
- Fls_MCoreTimeoutNotifPtrType MCoreTimeoutNotifPtr
  pointer to finished flash access notification
- MemIf_ModeType eDefaultMode
  default FLS device mode after initialization (MEMIF_MODE_FAST, MEMIF_MODE_SLOW)
- Fls_LengthType u32MaxReadFastMode
  max number of bytes to read in one cycle of Fls_MainFunction (fast mode)
- Fls_LengthType u32MaxReadNormalMode
  max number of bytes to read in one cycle of Fls_MainFunction (normal mode)
- Fls_LengthType u32MaxWriteFastMode
  max number of bytes to write in one cycle of Fls_MainFunction (fast mode)
- Fls_LengthType u32MaxWriteNormalMode
  max number of bytes to write in one cycle of Fls_MainFunction (normal mode)
- Fls_SectorCountType u32SectorCount
  number of configured logical sectors
- const Fls_AddressType(* paSectorEndAddr)[]
  pointer to array containing last logical address of each configured sector
- const Fls_LengthType(* paSectorSize)[]
  pointer to array containing sector size of each configured sector
- const uint8(* paSectorFlags)[]

pointer to array containing flags set of each configured sector

- const Fls_LengthType(* paSectorPageSize)[]

  pointer to array containing page size information of each configured sector

- const Fls_HwChType(* paHwCh)[]

  Pointer to array containing the hardware channel(internal, external_qspi, external_emmc) of each configured sector.

- const uint32(* paSectorHwAddress)[]

  Pointer to array containing the configured hardware start address of each external sector.

- const Fls_QspiCfgConfigType * pFlsQspiCfgConfig

  Pointer to configuration structure of QSPI.

6.1.2.2.1 Field Documentation

6.1.2.2.1.1 jobEndNotificationPtr `Fls_JobEndNotificationPtrType jobEndNotificationPtr`

pointer to job end notification function

Definition at line 424 of file Fls_Types.h.

6.1.2.2.1.2 jobErrorNotificationPtr `Fls_JobErrorNotificationPtrType jobErrorNotificationPtr`

pointer to job error notification function

Definition at line 428 of file Fls_Types.h.

6.1.2.2.1.3 MCoreTimeoutNotifPtr `Fls_MCoreTimeoutNotifPtrType MCoreTimeoutNotifPtr`

pointer to finished flash access notification

Definition at line 433 of file Fls_Types.h.

6.1.2.2.1.4 eDefaultMode `MemIf_ModeType eDefaultMode`

default FLS device mode after initialization (MEMIF_MODE_FAST, MEMIF_MODE_SLOW)

Definition at line 438 of file Fls_Types.h.

6.1.2.2.1.5 u32MaxReadFastMode `Fls_LengthType u32MaxReadFastMode`

max number of bytes to read in one cycle of Fls_MainFunction (fast mode)

Definition at line 442 of file Fls_Types.h.

6.1.2.2.1.6 u32MaxReadNormalMode `Fls_LengthType u32MaxReadNormalMode`

max number of bytes to read in one cycle of Fls_MainFunction (normal mode)

Definition at line 446 of file Fls_Types.h.

6.1.2.2.1.7 u32MaxWriteFastMode `Fls_LengthType u32MaxWriteFastMode`

max number of bytes to write in one cycle of Fls_MainFunction (fast mode)

Definition at line 450 of file Fls_Types.h.

6.1.2.2.1.8 u32MaxWriteNormalMode `Fls_LengthType u32MaxWriteNormalMode`

max number of bytes to write in one cycle of Fls_MainFunction (normal mode)

Definition at line 454 of file Fls_Types.h.

6.1.2.2.1.9 u32SectorCount `Fls_SectorCountType u32SectorCount`

number of configured logical sectors

Definition at line 458 of file Fls_Types.h.

6.1.2.2.1.10 paSectorEndAddr `const Fls_AddressType(* paSectorEndAddr)[]`

pointer to array containing last logical address of each configured sector

Definition at line 462 of file Fls_Types.h.

6.1.2.2.1.11 paSectorSize `const Fls_LengthType(* paSectorSize)[]`

pointer to array containing sector size of each configured sector

Definition at line 466 of file Fls_Types.h.

6.1.2.2.1.12 paSectorFlags `const uint8(* paSectorFlags)[]`

pointer to array containing flags set of each configured sector

Definition at line 470 of file Fls_Types.h.

6.1.2.2.1.13 paSectorPageSize `const Fls_LengthType(* paSectorPageSize)[]`

pointer to array containing page size information of each configured sector

Definition at line 474 of file Fls_Types.h.

6.1.2.2.1.14 paHwCh `const Fls_HwChType(* paHwCh)[]`

Pointer to array containing the hardware channel(internal, external_qspi, external_emmc) of each configured sector.

Definition at line 478 of file Fls_Types.h.

6.1.2.2.1.15 paSectorHwAddress `const uint32(* paSectorHwAddress)[]`

Pointer to array containing the configured hardware start address of each external sector.

Definition at line 482 of file Fls_Types.h.

6.1.2.2.1.16 pFlsQspiCfgConfig `const Fls_QspiCfgConfigType* pFlsQspiCfgConfig`

Pointer to configuration structure of QSPI.

Definition at line 485 of file Fls_Types.h.

6.1.3 Macro Definition Documentation

6.1.3.1 FLS_DEVICE_INSTANCE_INVALID

```
#define FLS_DEVICE_INSTANCE_INVALID
```

Invalid device instance

Definition at line 147 of file Fls.h.

6.1.3.2 FLS_API_VENDOR_ID

```
#define FLS_API_VENDOR_ID
```

Version Check parameters.

Definition at line 57 of file Fls_Api.h.

6.1.3.3 FLS_MODULE_ID

```
#define FLS_MODULE_ID
```

AUTOSAR module identification.

Definition at line 103 of file Fls_Api.h.

6.1.3.4 FLS_INSTANCE_ID

```
#define FLS_INSTANCE_ID
```

AUTOSAR module instance identification.

Definition at line 107 of file Fls_Api.h.

6.1.3.5 FLS_E_PARAM_CONFIG

```
#define FLS_E_PARAM_CONFIG
```

Development error codes (passed to DET).

API service called with wrong config parameter

Definition at line 115 of file Fls_Api.h.

6.1.3.6 FLS_E_PARAM_ADDRESS

```
#define FLS_E_PARAM_ADDRESS
```

API service called with wrong address parameter.

Definition at line 119 of file Fls_Api.h.

6.1.3.7 FLS_E_PARAM_LENGTH

```
#define FLS_E_PARAM_LENGTH
```

API service called with wrong length parameter.

Definition at line 123 of file Fls_Api.h.

6.1.3.8 FLS_E_PARAM_DATA

```
#define FLS_E_PARAM_DATA
```

API service called with wrong data parameter.

Definition at line 127 of file Fls_Api.h.

6.1.3.9 FLS_E_UNINIT

```
#define FLS_E_UNINIT
```

API service called without module initialization.

Definition at line 131 of file Fls_Api.h.

6.1.3.10 FLS_E_BUSY

```
#define FLS_E_BUSY
```

API service called while driver still busy.

Definition at line 135 of file Fls_Api.h.

6.1.3.11 FLS_E_PARAM_POINTER

```
#define FLS_E_PARAM_POINTER
```

API service called with NULL pointer.

Definition at line 139 of file Fls_Api.h.

6.1.3.12 FLS_E_VERIFY_ERASE_FAILED

```
#define FLS_E_VERIFY_ERASE_FAILED
```

Runtime error codes (passed to DET).

Erase verification (blank check) failed

Definition at line 147 of file Fls_Api.h.

6.1.3.13 FLS_E_VERIFY_WRITE_FAILED

```
#define FLS_E_VERIFY_WRITE_FAILED
```

Write verification (compare) failed.

Definition at line 151 of file Fls_Api.h.

6.1.3.14 FLS_E_TIMEOUT

```
#define FLS_E_TIMEOUT
```

Timeout exceeded.

Definition at line 155 of file Fls_Api.h.

6.1.3.15 FLS_E_ERASE_FAILED

```
#define FLS_E_ERASE_FAILED
```

Transient Faults codes (passed to DET).

Flash erase failed (HW)

Definition at line 163 of file Fls_Api.h.

6.1.3.16 FLS_E_WRITE_FAILED

```
#define FLS_E_WRITE_FAILED
```

Flash write failed (HW)

Definition at line 167 of file Fls_Api.h.

6.1.3.17 FLS_E_READ_FAILED

```
#define FLS_E_READ_FAILED
```

Flash read failed (HW)

Definition at line 171 of file Fls_Api.h.

6.1.3.18 FLS_E_COMPARE_FAILED

```
#define FLS_E_COMPARE_FAILED
```

Flash compare failed (HW)

Definition at line 175 of file Fls_Api.h.

6.1.3.19 FLS_INIT_ID

```
#define FLS_INIT_ID
```

All service IDs (passed to DET).

service ID of function: Fls_Init. (passed to DET)

Definition at line 187 of file Fls_Api.h.

6.1.3.20 FLS_ERASE_ID

```
#define FLS_ERASE_ID
```

service ID of function: Fls_Erase. (passed to DET)

Definition at line 191 of file Fls_Api.h.

6.1.3.21 FLS_WRITE_ID

```
#define FLS_WRITE_ID
```

service ID of function: Fls_Write. (passed to DET)

Definition at line 195 of file Fls_Api.h.

6.1.3.22 FLS_CANCEL_ID

```
#define FLS_CANCEL_ID
```

service ID of function: Fls_Cancel. (passed to DET)

Definition at line 199 of file Fls_Api.h.

6.1.3.23 FLS_GETJOBRESULT_ID

```
#define FLS_GETJOBRESULT_ID
```

service ID of function: Fls_GetJobResult. (passed to DET)

Definition at line 203 of file Fls_Api.h.

6.1.3.24 FLS_MAINFUNCTION_ID

```
#define FLS_MAINFUNCTION_ID
```

service ID of function: Fls_MainFunction. (passed to DET)

Definition at line 207 of file Fls_Api.h.

6.1.3.25 FLS_READ_ID

```
#define FLS_READ_ID
```

service ID of function: Fls_Read. (passed to DET)

Definition at line 211 of file Fls_Api.h.

6.1.3.26 FLS_COMPARE_ID

```
#define FLS_COMPARE_ID
```

service ID of function: Fls_Compare. (passed to DET)

Definition at line 215 of file Fls_Api.h.

6.1.3.27 FLS_SETMODE_ID

```
#define FLS_SETMODE_ID
```

service ID of function: Fls_SetMode. (passed to DET)

Definition at line 219 of file Fls_Api.h.

6.1.3.28 FLS_GETVERSIONINFO_ID

```
#define FLS_GETVERSIONINFO_ID
```

service ID of function: Fls_GetVersionInfo. (passed to DET)

Definition at line 223 of file Fls_Api.h.

6.1.3.29 FLS_BLANK_CHECK_ID

```
#define FLS_BLANK_CHECK_ID
```

service ID of function: Fls_BlankCheck. (passed to DET)

Definition at line 227 of file Fls_Api.h.

6.1.3.30 FLS_SECTOR_ERASE_ASYNCH

```
#define FLS_SECTOR_ERASE_ASYNCH
```

All sector flags.

fls sector erase asynch

Definition at line 236 of file Fls_Api.h.

6.1.3.31 FLS_PAGE_WRITE_ASYNCH

```
#define FLS_PAGE_WRITE_ASYNCH
```

fls page write asynch

Definition at line 240 of file Fls_Api.h.

6.1.3.32 FLS_START_SEC_CODE

```
#define FLS_START_SEC_CODE
```

Start of Fls section CODE.

Definition at line 252 of file Fls_Api.h.

6.1.3.33 FLS_STOP_SEC_CODE

```
#define FLS_STOP_SEC_CODE
```

Stop of Fls section CODE.

Definition at line 503 of file Fls_Api.h.

6.1.4 Types Reference

6.1.4.1 Fls_SectorIndexType

```
typedef uint32 Fls_SectorIndexType
```

Logical sector index.

Definition at line 231 of file Fls_Types.h.

6.1.4.2 Fls_AddressType

```
typedef uint32 Fls_AddressType
```

Fls Address Type.

Address offset from the configured flash base address to access a certain flash memory area.

Definition at line 247 of file Fls_Types.h.

6.1.4.3 Fls_LengthType

```
typedef uint32 Fls_LengthType
```

Fls Length Type.

Number of bytes to read,write,erase,compare

Definition at line 253 of file Fls_Types.h.

6.1.4.4 Fls_SectorCountType

```
typedef uint32 Fls_SectorCountType
```

Fls Sector Count Type.

Number of configured sectors

Definition at line 259 of file Fls_Types.h.

6.1.4.5 Fls_JobEndNotificationPtrType

```
typedef void(* Fls_JobEndNotificationPtrType) (void)
```

Fls Job End Notification Pointer Type.

Pointer type of Fls_JobEndNotification function

Definition at line 339 of file Fls_Types.h.

6.1.4.6 Fls_JobErrorNotificationPtrType

```
typedef void(* Fls_JobErrorNotificationPtrType) (void)
```

Fls Job Error Notification Pointer Type.

Pointer type of Fls_JobErrorNotification function

Definition at line 345 of file Fls_Types.h.

6.1.4.7 Fls_MCoreTimeoutNotifPtrType

```
typedef void(* Fls_MCoreTimeoutNotifPtrType) (Fls_MCoreTimeoutJobType eMCoreTimeoutJob)
```

Fls Multi Core Notification Pointer Type.

Pointer type of Fls_MCoreTimeoutNotifPtrType function

Definition at line 352 of file Fls_Types.h.

6.1.5 Enum Reference

6.1.5.1 Fls_HwChType

```
enum Fls_HwChType
```

Flash sector channel type.

Definition at line 137 of file Fls_Types.h.

6.1.5.2 Fls_JobType

```
enum Fls_JobType
```

Type of job currently executed by Fls_MainFunction.

Enumerator

FLS_JOB_ERASE	erase one or more complete flash sectors
FLS_JOB_WRITE	write one or more complete flash pages
FLS_JOB_READ	read one or more bytes from flash memory
FLS_JOB_COMPARE	compare data buffer with content of flash memory
FLS_JOB_BLANK_CHECK	check content of erased flash memory area

Definition at line 146 of file Fls_Types.h.

6.1.5.3 Fls_LLDReturnType

enum Fls_LLDReturnType

Result of low-level flash operation.

Enumerator

FLASH_E_OK	operation succeeded
FLASH_E_FAILED	operation failed due to hardware error
FLASH_E_BLOCK_INCONSISTENT	data buffer doesn't match with content of flash memory
FLASH_E_PENDING	operation is pending
FLASH_E_PARTITION_ERR	FlexNVM partition ratio error.

Definition at line 173 of file Fls_Types.h.

6.1.5.4 Fls_LLDJobType

enum Fls_LLDJobType

Type of job currently executed by Fls_LLDMainFunction.

Enumerator

FLASH_JOB_NONE	no job executed by Fls_LLDMainFunction
FLASH_JOB_ERASE	erase one flash sector
FLASH_JOB_ERASE_TEMP	complete erase and start an interleaved erase flash sector
FLASH_JOB_WRITE	write one or more complete flash pages
FLASH_JOB_ERASE_BLANK_CHECK	erase blank check of flash sector

Definition at line 185 of file Fls_Types.h.

6.1.5.5 Fls_CrcDataSizeType

enum Fls_CrcDataSizeType

Size of data to be processeed by CRC.

Enumerator

FLS_CRC_8_BITS	crc 8 bits
FLS_CRC_16_BITS	crc 16 bits

Definition at line 215 of file Fls_Types.h.

6.1.5.6 Fls_MCoreReqReturnType

enum Fls_MCoreReqReturnType

Fls Multi Core Request Return Type.

The return value for the function requesting multi core access.

Enumerator

FLS_MCORE_ERROR	return error
FLS_MCORE_TIMEOUT	return timeout
FLS_MCORE_PENDING	return pending
FLS_MCORE_GRANTED	return granted
FLS_MCORE_CANCELLED	return cancelled

Definition at line 268 of file Fls_Types.h.

6.1.5.7 Fls_MCoreHwJobStatusType

enum Fls_MCoreHwJobStatusType

Fls Multi Core hardware job status.

The status of a multicore core flash job, in hardware. Used to determine if a flash job subject to multicore arbitration was started/suspended/aborted in hardware, in the flash controller. This can be used for example, to clear a semaphore granted for erase directly, if the job was not actually started in hardware, instead of attempting to suspend it.

Enumerator

FLS_MCORE_HW_JOB_IDLE	idle status
FLS_MCORE_HW_JOB_MAINF_STARTED	mainf started status
FLS_MCORE_HW_JOB_STARTED	started status
FLS_MCORE_HW_JOB_CANCELLED	cancelled status

Definition at line 301 of file Fls_Types.h.

6.1.5.8 Fls_MCoreTimeoutJobType

```
enum Fls_MCoreTimeoutJobType
```

Fls Multi Core timeout notification jobs.

The timeout notification specifies the job during which timeout occurred.

Enumerator

FLS_MCORE_TIMEOUT_ERASE	timeout job erase
FLS_MCORE_TIMEOUT_WRITE	timeout job write
FLS_MCORE_TIMEOUT_READ	timeout job read
FLS_MCORE_TIMEOUT_COMPARE	timeout job compare
FLS_MCORE_TIMEOUT_BLANK_CHECK	timeout job blank check

Definition at line 326 of file Fls_Types.h.

6.1.6 Function Reference

6.1.6.1 Fls_Init()

```
void Fls_Init (
            const Fls_ConfigType * ConfigPtr )
```

The function initializes Fls module.

The function sets the internal module variables according to given configuration set.

Parameters

in	*ConfigPtr*	Pointer to flash driver configuration set.

Precondition

`ConfigPtr` must not be `NULL_PTR` and the module status must not be `MEMIF_BUSY`.

6.1.6.2 Fls_Write()

```
Std_ReturnType Fls_Write (
            Fls_AddressType TargetAddress,
            const uint8 * SourceAddressPtr,
            Fls_LengthType Length )
```

Write one or more complete flash pages to the flash device.

Starts a write job asynchronously. The actual job is performed by `Fls_MainFunction`.

Parameters

in	*TargetAddress*	Target address in flash memory. This address offset will be added to the flash memory base address.
in	*SourceAddressPtr*	Pointer to source data buffer.
in	*Length*	Number of bytes to write.

Returns

Std_ReturnType

Return values

E_OK	Write command has been accepted.
E_NOT_OK	Write command has not been accepted.

Precondition

The module has to be initialized and not busy.

Postcondition

`Fls_Write` changes module status and some internal variables (`Fls_u32JobSectorIt`, `Fls_u32Job↩AddrIt`, `Fls_u32JobAddrEnd`, `Fls_pJobDataSrcPtr`, `Fls_eJob`, `Fls_eJobResult`).

6.1.6.3 Fls_Erase()

```
Std_ReturnType Fls_Erase (
            Fls_AddressType TargetAddress,
            Fls_LengthType Length )
```

Erase one or more complete flash sectors.

Starts an erase job asynchronously. The actual job is performed by the Fls_MainFunction.

Parameters

in	*TargetAddress*	Target address in flash memory.
in	*Length*	Number of bytes to erase.

Returns

Std_ReturnType

Return values

E_OK	Erase command has been accepted.
E_NOT_OK	Erase command has not been accepted.

Precondition

The module has to be initialized and not busy.

Postcondition

Fls_Erase changes module status and some internal variables (Fls_u32JobSectorIt, Fls_u32Job↩ SectorEnd, Fls_Job, Fls_eJobResult).

6.1.6.4 Fls_Read()

```
Std_ReturnType Fls_Read (
            Fls_AddressType SourceAddress,
            uint8 * TargetAddressPtr,
            Fls_LengthType Length )
```

Reads from flash memory.

Starts a read job asynchronously. The actual job is performed by Fls_MainFunction.

Parameters

in	*SourceAddress*	Source address in flash memory. This address offset will be added to the flash memory base address.
in	*Length*	Number of bytes to read.
out	*TargetAddressPtr*	Pointer to target data buffer.

Returns

Std_ReturnType

Return values

E_OK	Read command has been accepted
E_NOT_OK	Read command has not been accepted

Precondition

The module has to be initialized and not busy.

Postcondition

`Fls_Read` changes module status and some internal variables (`Fls_u32JobSectorIt`, `Fls_u32JobAddrIt`, `Fls_u32JobAddrEnd`, `Fls_pJobDataDestPtr`, `Fls_eJob`, `Fls_eJobResult`).

6.1.7 Variable Documentation

6.1.7.1 Fls_u32JobAddrIt

```
Fls_AddressType Fls_u32JobAddrIt  [extern]
```

Logical address of data block currently processed by Fls_MainFunction.

6.1.7.2 Fls_u32JobAddrEnd

```
Fls_AddressType Fls_u32JobAddrEnd  [extern]
```

Last logical address to be processed by a job.

6.1.7.3 Fls_u32JobSectorIt

```
volatile Fls_SectorIndexType Fls_u32JobSectorIt  [extern]
```

Index of flash sector currently processed by a job.

Used by all types of job

6.1.7.4 Fls_u32JobSectorEnd

```
Fls_SectorIndexType Fls_u32JobSectorEnd  [extern]
```

Index of last flash sector by current job.

Used to check status of all external flash chips before start jobs or is the last sector in Erease job

6.1.7.5 Fls_eLLDJobResult

```
volatile MemIf_JobResultType Fls_eLLDJobResult  [extern]
```

Result of last flash hardware job.

6.1.7.6 Fls_eLLDJob

```
Fls_LLDJobType Fls_eLLDJob  [extern]
```

Type of current flash hardware job - used for asynchronous operating mode.

6.1.7.7 Fls_pConfigPtr

```
const Fls_ConfigType* Fls_pConfigPtr  [extern]
```

Pointer to current flash module configuration set.

6.2 QSPI IPV Driver

6.2.1 Detailed Description

Data Structures

- struct Qspi_Ip_HyperFlashConfigType
  Hyperflash configuration structure. *More...*
- struct Qspi_Ip_DllSettingsType
  DLL configuration structure. *More...*
- struct Qspi_Ip_ControllerAhbConfigType
  AHB configuration structure. *More...*
- struct Qspi_Ip_ControllerConfigType
  Driver configuration structure. *More...*
- struct Qspi_Ip_StatusConfigType
  Status register configuration structure. *More...*
- struct Qspi_Ip_EraseVarConfigType
  Describes one type of erase. *More...*
- struct Qspi_Ip_EraseConfigType
  Erase capabilities configuration structure. *More...*
- struct Qspi_Ip_ReadIdConfigType
  Read Id capabilities configuration structure. *More...*
- struct Qspi_Ip_SuspendConfigType
  Suspend capabilities configuration structure. *More...*
- struct Qspi_Ip_ResetConfigType
  Soft Reset capabilities configuration structure. *More...*
- struct Qspi_Ip_LutConfigType
  List of LUT sequences. *More...*
- struct Qspi_Ip_InitOperationType
  Initialization operation. *More...*
- struct Qspi_Ip_InitConfigType
  Initialization sequence. *More...*
- struct Qspi_Ip_MemoryConfigType
  Driver configuration structure. *More...*
- struct Qspi_Ip_MemoryConnectionType
  Flash-controller conections configuration structure. *More...*

Macros

- #define QSPI_IP_MAX_READ_SIZE
- #define QSPI_IP_MAX_WRITE_SIZE
- #define QSPI_IP_ERASE_TYPES
- #define QSPI_IP_AHB_BUFFERS
  Number of AHB buffers in the device.
- #define QSPI_IP_LUT_INVALID
- #define QSPI_IP_LUT_SEQ_END
- #define QSPI_IP_PACK_LUT_REG(ops0, ops1)

Types Reference

- typedef uint16 Qspi_Ip_InstrOpType

  Operation in a LUT sequence.
- typedef Qspi_Ip_StatusType(* Qspi_Ip_InitCalloutPtrType) (uint32 instance)

  Init callout pointer type.
- typedef Qspi_Ip_StatusType(* Qspi_Ip_ResetCalloutPtrType) (uint32 instance)

  Reset callout pointer type.
- typedef Qspi_Ip_StatusType(* Qspi_Ip_ErrorCheckCalloutPtrType) (uint32 instance)

  Error Check callout pointer type.
- typedef Qspi_Ip_StatusType(* Qspi_Ip_EccCheckCalloutPtrType) (uint32 instance, uint32 startAddress, uint32 dataLength)

  Ecc Check callout pointer type.

Enum Reference

- enum Qspi_Ip_HyperflashParamSectorMapType

  Parameter sector map.
- enum Qspi_Ip_HyperflashDrvStrengthType

  Drive strength.
- enum Qspi_Ip_HyperflashReadLatencyType

  Read latency.
- enum Qspi_Ip_HyperflashAsoEntryCommandsType
- enum Qspi_Ip_HyperflashSectorProtectionType

  Sector protection type.
- enum Qspi_Ip_StatusType

  qspi return codes
- enum Qspi_Ip_ConnectionType

  flash connection to the QSPI module
- enum Qspi_Ip_OpType

  flash operation type
- enum Qspi_Ip_LutCommandsType

  Lut commands.
- enum Qspi_Ip_LutPadsType

  Lut pad options.
- enum Qspi_Ip_ReadModeType

  Read mode.
- enum Qspi_Ip_DataRateType

  Clock phase used for sampling Rx data.
- enum Qspi_Ip_SampleDelayType

  Delay used for sampling Rx data.
- enum Qspi_Ip_SamplePhaseType

  Clock phase used for sampling Rx data.
- enum Qspi_Ip_FlashDataAlignType

  Alignment of outgoing data with serial clock.
- enum Qspi_Ip_DllModeType

DLL configuration modes.

- enum Qspi_Ip_LastCommandType

  Last command that was executed by the device flash.

- enum Qspi_Ip_FlashMemoryType

  Parameter memory type.

Function Reference

- Qspi_Ip_StatusType Qspi_Ip_Init (uint32 instance, const Qspi_Ip_MemoryConfigType *pConfig, const Qspi_Ip_MemoryConnectionType *pConnect)

  Initializes the serial flash memory driver.

- Qspi_Ip_StatusType Qspi_Ip_Deinit (uint32 instance)

  De-initializes the serial flash memory driver.

- Qspi_Ip_StatusType Qspi_Ip_EraseBlock (uint32 instance, uint32 address, uint32 size)

  Erase a sector in the serial flash.

- Qspi_Ip_StatusType Qspi_Ip_EraseChip (uint32 instance)

  Erase the entire serial flash.

- Qspi_Ip_StatusType Qspi_Ip_GetMemoryStatus (uint32 instance)

  Check the status of the flash device.

- Qspi_Ip_StatusType Qspi_Ip_SetProtection (uint32 instance, uint8 value)

  Sets the protection bits to the requested value.

- Qspi_Ip_StatusType Qspi_Ip_GetProtection (uint32 instance, uint8 *value)

  Returns the current value of the protection bits.

- Qspi_Ip_StatusType Qspi_Ip_Reset (uint32 instance)

  Resets the flash device.

- Qspi_Ip_StatusType Qspi_Ip_Enter0XX (uint32 instance)

  Enters 0-X-X (no command) mode. This mode assumes only reads are performed.

- Qspi_Ip_StatusType Qspi_Ip_Exit0XX (uint32 instance)

  Exits 0-X-X (no command) mode. This allows operations other than reads to be performed.

- Qspi_Ip_StatusType Qspi_Ip_ProgramSuspend (uint32 instance)

  Suspends a program operation.

- Qspi_Ip_StatusType Qspi_Ip_ProgramResume (uint32 instance)

  Resumes a program operation.

- Qspi_Ip_StatusType Qspi_Ip_EraseSuspend (uint32 instance)

  Suspends an erase operation.

- Qspi_Ip_StatusType Qspi_Ip_EraseResume (uint32 instance)

  Resumes an erase operation.

- Qspi_Ip_StatusType Qspi_Ip_Read (uint32 instance, uint32 address, uint8 *data, uint32 size)

  Read data from serial flash.

- Qspi_Ip_StatusType Qspi_Ip_ReadId (uint32 instance, uint8 *data)

  Read manufacturer ID/device ID from serial flash.

- Qspi_Ip_StatusType Qspi_Ip_ProgramVerify (uint32 instance, uint32 address, const uint8 *data, uint32 size)

  Verifies the correctness of the programmed data.

- Qspi_Ip_StatusType Qspi_Ip_EraseVerify (uint32 instance, uint32 address, uint32 size)

  Checks whether or not an area in the serial flash is erased.

- Qspi_Ip_StatusType Qspi_Ip_Program (uint32 instance, uint32 address, const uint8 *data, uint32 size)

Writes data in serial flash.

- Qspi_Ip_StatusType Qspi_Ip_RunCommand (uint32 instance, uint16 lut, uint32 addr)

  Launches a simple command for the serial flash.
- Qspi_Ip_StatusType Qspi_Ip_RunReadCommand (uint32 instance, uint16 lut, uint32 addr, uint8 *dataRead, const uint8 *dataCmp, uint32 size)

  Launches a read command for the serial flash.
- Qspi_Ip_StatusType Qspi_Ip_RunWriteCommand (uint32 instance, uint16 lut, uint32 addr, const uint8 *data, uint32 size)

  Launches a write command for the serial flash.
- Qspi_Ip_StatusType Qspi_Ip_AhbReadEnable (uint32 instance)

  Sets up AHB reads to the serial flash.
- Qspi_Ip_StatusType Qspi_Ip_ControllerGetStatus (uint32 instance)

  Check the status of the QSPI controller.
- Qspi_Ip_StatusType Qspi_Ip_ControllerInit (uint32 instance, const Qspi_Ip_ControllerConfigType *user↩ ConfigPtr)

  Initializes the qspi driver.
- Qspi_Ip_StatusType Qspi_Ip_ControllerDeinit (uint32 instance)

  De-initialize the qspi driver.
- Qspi_Ip_StatusType Qspi_Ip_Abort (uint32 instance)

  Aborts any on-going transactions.
- Qspi_Ip_StatusType Qspi_Ip_ReadSfdp (Qspi_Ip_MemoryConfigType *pConfig, const Qspi_Ip_MemoryConnectionTyp *pConnect)

  Initializes the serial flash memory configuration from SFDP table.

QuadSPI Driver

- void Qspi_Ip_SetLut (uint32 instance, uint8 LutRegister, Qspi_Ip_InstrOpType operation0, Qspi_Ip_InstrOpType operation1)

  Configures LUT commands.
- void Qspi_Ip_WriteLuts_Privileged (uint32 Instance, uint8 StartLutRegister, const uint32 *Data, uint8 Size)

  Configures LUT commands.
- void Qspi_Ip_SetAhbSeqId_Privileged (uint32 instance, uint8 seqID)

  Sets sequence ID for AHB operations.
- uint32 Qspi_Ip_GetBaseAdress (uint32 instance, Qspi_Ip_ConnectionType connectionType)

  Returns the physical base address of a flash device.
- Qspi_Ip_StatusType Qspi_Ip_IpCommand (uint32 instance, uint8 SeqId, uint32 addr)

  Launches a simple IP command.
- Qspi_Ip_StatusType Qspi_Ip_IpRead (uint32 instance, uint8 SeqId, uint32 addr, uint8 *dataRead, const uint8 *dataCmp, uint32 size)

  Launches an IP read command.
- Qspi_Ip_StatusType Qspi_Ip_IpWrite (uint32 instance, uint8 SeqId, uint32 addr, const uint8 *data, uint32 size)

  Launches an IP write command.
- #define **FLS_START_SEC_CODE**
- #define **FLS_STOP_SEC_CODE**

6.2.2 Data Structure Documentation

6.2.2.1 struct Qspi_Ip_HyperFlashConfigType

Hyperflash configuration structure.

This structure is used to provide configuration parameters for HyperFlash at initialization time.

Definition at line 216 of file Qspi_Ip_HyperflashTypes.h.

Data Fields

Type	Name	Description
Qspi_Ip_HyperflashDrvStrengthType	outputDriverStrength	Output driver level of the device
boolean	RWDSLowOnDualError	Specifies if RWDS will stall upon Dual Error Detect
boolean	secureRegionUnlocked	If true, the secure silicon region will be locked
Qspi_Ip_HyperflashReadLatencyType	readLatency	Read latency
Qspi_Ip_HyperflashParamSectorMapType	paramSectorMap	Parameter sector mapping
uint32	deviceIdWordAddress	The word address of the device Id in ASO

6.2.2.2 struct Qspi_Ip_DllSettingsType

DLL configuration structure.

This structure contains initialization settings for DLL and slave delay chain

Definition at line 322 of file Qspi_Ip_Types.h.

Data Fields

Type	Name	Description
Qspi_Ip_DllModeType	dllMode	Mode in which DLL is used
boolean	freqEnable	Selects delay-chain for high frequency of operation
uint8	referenceCounter	Select the "n+1" interval of DLL phase detection and reference delay updating interval
uint8	resolution	Minimum resolution for DLL phase detector
uint8	coarseDelay	Coarse delay DLL slave delay chain
uint8	fineDelay	Fine delay DLL slave delay chain
uint8	tapSelect	Selects the Nth tap provided by the slave delay-chain

6.2.2.3 struct Qspi_Ip_ControllerAhbConfigType

AHB configuration structure.

This structure is used to provide configuration parameters for AHB access to the external flash

Definition at line 339 of file Qspi_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	masters[4U]	List of AHB masters assigned to each buffer
uint16	sizes[4U]	List of buffer sizes
boolean	allMasters	Indicates that any master may access the last buffer

6.2.2.4 struct Qspi_Ip_ControllerConfigType

Driver configuration structure.

This structure is used to provide configuration parameters for the qspi driver at initialization time.

Definition at line 371 of file Qspi_Ip_Types.h.

Data Fields

Type	Name	Description
Qspi_Ip_DataRateType	dataRate	Single/double data rate
uint32	memSizeA1	Size of serial flash A1
uint32	memSizeA2	Size of serial flash A2
uint32	memSizeB1	Size of serial flash B1
uint32	memSizeB2	Size of serial flash B2
uint8	csHoldTime	CS hold time, expressed in serial clock cycles
uint8	csSetupTime	CS setup time, expressed in serial clock cycles
uint8	columnAddr	Width of the column address, 0 if not used
boolean	wordAddresable	True if serial flash is word addressable
Qspi_Ip_ReadModeType	readModeA	Read mode for incoming data from serial flash A

Data Fields

Type	Name	Description
Qspi_Ip_ReadModeType	readModeB	Read mode for incoming data from serial flash B
Qspi_Ip_SampleDelayType	sampleDelay	Delay (in clock cycles) used for sampling Rx data
Qspi_Ip_SamplePhaseType	samplePhase	Clock phase used for sampling Rx data
Qspi_Ip_DllSettingsType	dllSettingsA	DLL settings for side A
Qspi_Ip_DllSettingsType	dllSettingsB	DLL settings for side B
boolean	centerAlignedStrobeA	Enable center-aligned read strobe
boolean	centerAlignedStrobeB	Enable center-aligned read strobe
boolean	differentialClockA	Enable clock on differential CKN pad
boolean	differentialClockB	Enable clock on differential CKN pad
boolean	dqsLatency	Enable DQS latency for reads (Hyperflash)
Qspi_Ip_FlashDataAlignType	dataAlign	Alignment of output data sent to serial flash
uint8	io2IdleValueA	(0 / 1) Logic level of IO[2] signal when not used on side A
uint8	io3IdleValueA	(0 / 1) Logic level of IO[3] signal when not used on side A
uint8	io2IdleValueB	(0 / 1) Logic level of IO[2] signal when not used on side B
uint8	io3IdleValueB	(0 / 1) Logic level of IO[3] signal when not used on side B
boolean	byteSwap	Enable byte swap in octal DDR mode
Qspi_Ip_ControllerAhbConfigType	ahbConfig	AHB buffers configuration

6.2.2.5 struct Qspi_Ip_StatusConfigType

Status register configuration structure.

This structure contains information about the status registers of the external flash

Definition at line 419 of file Qspi_Ip_Types.h.

Data Fields

Type	Name	Description
uint16	statusRegInitReadLut	Command used to read the status register during initialization
uint16	statusRegReadLut	Command used to read the status register
uint16	statusRegWriteLut	Command used to write the status register
uint16	writeEnableSRLut	Write enable command used before writing to status register
uint16	writeEnableLut	Write enable command used before write or erase operations
uint8	regSize	Size in bytes of status register
uint8	busyOffset	Position of "busy" bit inside status register
uint8	busyValue	Value of "busy" bit which indicates that the device is busy; can be 0 or 1
uint8	writeEnableOffset	Position of "write enable" bit inside status register
uint8	blockProtectionOffset	Offset of block protection bits inside status register
uint8	blockProtectionWidth	Width of block protection bitfield
uint8	blockProtectionValue	Value of block protection bitfield, indicate the protected area

6.2.2.6 struct Qspi_Ip_EraseVarConfigType

Describes one type of erase.

This structure contains information about one type of erase supported by the external flash

Definition at line 441 of file Qspi_Ip_Types.h.

Data Fields

Type	Name	Description
uint16	eraseLut	Lut index for erase command
uint8	size	Size of the erased area: 2 ^ size; e.g. 0x0C means 4 Kbytes

6.2.2.7 struct Qspi_Ip_EraseConfigType

Erase capabilities configuration structure.

This structure contains information about the erase capabilities of the external flash

Definition at line 453 of file Qspi_Ip_Types.h.

Data Fields

Type	Name	Description
Qspi_Ip_EraseVarConfigType	eraseTypes[4U]	Erase types supported by the device
uint16	chipEraseLut	Lut index for chip erase command

6.2.2.8 struct Qspi_Ip_ReadIdConfigType

Read Id capabilities configuration structure.

This structure contains information about the read manufacturer/device ID command

Definition at line 465 of file Qspi_Ip_Types.h.

Data Fields

Type	Name	Description
uint16	readIdLut	Read Id command
uint8	readIdSize	Size of data returned by Read Id command
uint8	readIdExpected[4U]	Device ID configured value (Memory density | Memory type | Manufacturer ID)

6.2.2.9 struct Qspi_Ip_SuspendConfigType

Suspend capabilities configuration structure.

This structure contains information about the Program / Erase Suspend capabilities of the external flash

Definition at line 478 of file Qspi_Ip_Types.h.

Data Fields

Type	Name	Description
uint16	eraseSuspendLut	Lut index for the erase suspend operation
uint16	eraseResumeLut	Lut index for the erase resume operation
uint16	programSuspendLut	Lut index for the program suspend operation
uint16	programResumeLut	Lut index for the program resume operation

6.2.2.10 struct Qspi_Ip_ResetConfigType

Soft Reset capabilities configuration structure.

This structure contains information about the Soft Reset capabilities of the external flash

Definition at line 492 of file Qspi_Ip_Types.h.

Data Fields

Type	Name	Description
uint16	resetCmdLut	First command in reset sequence
uint8	resetCmdCount	Number of commands in reset sequence

6.2.2.11 struct Qspi_Ip_LutConfigType

List of LUT sequences.

List of LUT sequences. Each sequence describes a command to the external flash. Sequences are separated by a 0 operation

Definition at line 520 of file Qspi_Ip_Types.h.

Data Fields

Type	Name	Description
uint16	opCount	Number of operations in the LUT table
Qspi_Ip_InstrOpType *	lutOps	List of operations

6.2.2.12 struct Qspi_Ip_InitOperationType

Initialization operation.

This structure describes one initialization operation.

Definition at line 532 of file Qspi_Ip_Types.h.

Data Fields

Type	Name	Description
Qspi_Ip_OpType	opType	Operation type

Data Fields

Type	Name	Description
uint16	command1Lut	Index of first command sequence in Lut; for RMW type this is the read command
uint16	command2Lut	Index of second command sequence in Lut, only used for RMW type, this is the write command
uint16	weLut	Index of write enable sequence in Lut, only used for Write and RMW type
uint32	addr	Address, if used in command.
uint8	size	Size in bytes of configuration register
uint8	shift	Position of configuration field inside the register
uint8	width	Width in bits of configuration field.
uint32	value	Value to set in the field
const Qspi_Ip_ControllerConfigType *	ctrlCfgPtr	New controller configuration, valid only for QSPI_IP_OP_TYPE_QSPI_CFG type

6.2.2.13 struct Qspi_Ip_InitConfigType

Initialization sequence.

Describe sequence that will be performed only once during initialization to put the flash in the desired state for operation. This may include, for example, setting the QE bit, activating 4-byte addressing, activating XPI mode

Definition at line 553 of file Qspi_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	opCount	Number of operations
Qspi_Ip_InitOperationType *	operations	List of operations

6.2.2.14 struct Qspi_Ip_MemoryConfigType

Driver configuration structure.

This structure is used to provide configuration parameters for the external flash driver at initialization time.

Definition at line 576 of file Qspi_Ip_Types.h.

Data Fields

Type	Name	Description
Qspi_Ip_FlashMemoryType	memType	Mmemory device type
const Qspi_Ip_HyperFlashConfigType *	hfConfig	Hyperflash configuration, NULL if not used
uint32	memSize	Memory size (in bytes)
uint32	pageSize	Page size (in bytes)
uint16	readLut	Command used to read data from flash
uint16	writeLut	Command used to write data to flash
uint16	read0xxLut	0-x-x mode read command
uint16	read0xxLutAHB	0-x-x mode AHB read command
Qspi_Ip_ReadIdConfigType	readIdSettings	Erase settings of the external flash
Qspi_Ip_EraseConfigType	eraseSettings	Erase settings of the external flash
Qspi_Ip_StatusConfigType	statusConfig	Status register information
Qspi_Ip_SuspendConfigType	suspendSettings	Program / Erase Suspend settings
Qspi_Ip_ResetConfigType	resetSettings	Soft Reset settings, used at runtime
Qspi_Ip_ResetConfigType	initResetSettings	Soft Reset settings, used for first time reset
Qspi_Ip_InitConfigType	initConfiguration	Operations for initial flash configuration
Qspi_Ip_LutConfigType	lutSequences	List of LUT sequences describing flash commands
Qspi_Ip_InitCalloutPtrType	initCallout	Pointer to init callout
Qspi_Ip_ResetCalloutPtrType	resetCallout	Pointer to reset callout
Qspi_Ip_ErrorCheckCalloutPtrType	errorCheckCallout	Pointer to error check callout
Qspi_Ip_EccCheckCalloutPtrType	eccCheckCallout	Pointer to ecc check callout
const Qspi_Ip_ControllerConfigType *	ctrlAutoCfgPtr	Initial controller configuration, if needed

6.2.2.15 struct Qspi_Ip_MemoryConnectionType

Flash-controller conections configuration structure.

This structure specifies thte connecctions of each flash device to QSPI controllers at initialization time.

Definition at line 608 of file Qspi_Ip_Types.h.

Data Fields

Type	Name	Description
uint32	qspiInstance	QSPI Instance where this device is connected
Qspi_Ip_ConnectionType	connectionType	Device connection to QSPI module
uint8	memAlignment	Memory alignment required by the external flash

6.2.3 Macro Definition Documentation

6.2.3.1 QSPI_IP_MAX_READ_SIZE

```
#define QSPI_IP_MAX_READ_SIZE
```

Maximum number of bytes then can be read in one operation

Definition at line 100 of file Qspi_Ip.h.

6.2.3.2 QSPI_IP_MAX_WRITE_SIZE

```
#define QSPI_IP_MAX_WRITE_SIZE
```

Maximum number of bytes then can be written in one operation

Definition at line 102 of file Qspi_Ip.h.

6.2.3.3 QSPI_IP_ERASE_TYPES

```
#define QSPI_IP_ERASE_TYPES
```

Number of erase types that can be supported by a flash device

Definition at line 138 of file Qspi_Ip_Types.h.

6.2.3.4 QSPI_IP_AHB_BUFFERS

```
#define QSPI_IP_AHB_BUFFERS
```

Number of AHB buffers in the device.

Definition at line 141 of file Qspi_Ip_Types.h.

6.2.3.5 QSPI_IP_LUT_INVALID

```
#define QSPI_IP_LUT_INVALID
```

Invalid index in virtual LUT, used for unsupported features

Definition at line 144 of file Qspi_Ip_Types.h.

6.2.3.6 QSPI_IP_LUT_SEQ_END

```
#define QSPI_IP_LUT_SEQ_END
```

End operation for a LUT sequence

Definition at line 146 of file Qspi_Ip_Types.h.

6.2.3.7 QSPI_IP_PACK_LUT_REG

```
#define QSPI_IP_PACK_LUT_REG(
             ops0,
             ops1 )
```

Pack the two operations into a LUT register (each operation is a pair of instruction-operand)

Definition at line 148 of file Qspi_Ip_Types.h.

6.2.4 Types Reference

6.2.4.1 Qspi_Ip_InstrOpType

```
typedef uint16 Qspi_Ip_InstrOpType
```

Operation in a LUT sequence.

This type describes one basic operation inside a LUT sequence. Each operation contains:

- instruction (6 bits)
- number of PADs (2 bits)
- operand (8 bits) Qspi_Ip_LutCommandsType and Qspi_Ip_LutPadsType types should be used to form operations

Definition at line 235 of file Qspi_Ip_Types.h.

6.2.4.2 Qspi_Ip_InitCalloutPtrType

```
typedef Qspi_Ip_StatusType(* Qspi_Ip_InitCalloutPtrType) (uint32 instance)
```

Init callout pointer type.

Definition at line 298 of file Qspi_Ip_Types.h.

6.2.4.3 Qspi_Ip_ResetCalloutPtrType

```
typedef Qspi_Ip_StatusType(* Qspi_Ip_ResetCalloutPtrType) (uint32 instance)
```

Reset callout pointer type.

Definition at line 302 of file Qspi_Ip_Types.h.

6.2.4.4 Qspi_Ip_ErrorCheckCalloutPtrType

```
typedef Qspi_Ip_StatusType(* Qspi_Ip_ErrorCheckCalloutPtrType) (uint32 instance)
```

Error Check callout pointer type.

Definition at line 306 of file Qspi_Ip_Types.h.

6.2.4.5 Qspi_Ip_EccCheckCalloutPtrType

```
typedef Qspi_Ip_StatusType(* Qspi_Ip_EccCheckCalloutPtrType) (uint32 instance, uint32 startAddress, uint32 dataLength)
```

Ecc Check callout pointer type.

Definition at line 310 of file Qspi_Ip_Types.h.

6.2.5 Enum Reference

6.2.5.1 Qspi_Ip_HyperflashParamSectorMapType

```
enum Qspi_Ip_HyperflashParamSectorMapType
```

Parameter sector map.

This structure is used to configure how the Parameter-Sectors are used and how they are mapped into the address map.

Definition at line 130 of file Qspi_Ip_HyperflashTypes.h.

6.2.5.2 Qspi_Ip_HyperflashDrvStrengthType

```
enum Qspi_Ip_HyperflashDrvStrengthType
```

Drive strength.

Hyperflash driver strength settings.

Enumerator

QSPI_IP_HF_DRV_STRENGTH_000	Typical Impedance for 1.8V: 27, Typical Impedance 3V: 20
QSPI_IP_HF_DRV_STRENGTH_001	Typical Impedance for 1.8V: 117, Typical Impedance 3V: 71
QSPI_IP_HF_DRV_STRENGTH_002	Typical Impedance for 1.8V: 68, Typical Impedance 3V: 40
QSPI_IP_HF_DRV_STRENGTH_003	Typical Impedance for 1.8V: 45, Typical Impedance 3V: 27
QSPI_IP_HF_DRV_STRENGTH_004	Typical Impedance for 1.8V: 34, Typical Impedance 3V: 20
QSPI_IP_HF_DRV_STRENGTH_005	Typical Impedance for 1.8V: 27, Typical Impedance 3V: 16
QSPI_IP_HF_DRV_STRENGTH_006	Typical Impedance for 1.8V: 24, Typical Impedance 3V: 14
QSPI_IP_HF_DRV_STRENGTH_007	Typical Impedance for 1.8V: 20, Typical Impedance 3V: 12

Definition at line 144 of file Qspi_Ip_HyperflashTypes.h.

6.2.5.3 Qspi_Ip_HyperflashReadLatencyType

enum Qspi_Ip_HyperflashReadLatencyType

Read latency.

Enumerator

QSPI_IP_HF_READ_LATENCY_5_CLOCKS	Read latency 5 clocks
QSPI_IP_HF_READ_LATENCY_6_CLOCKS	Read latency 6 clocks
QSPI_IP_HF_READ_LATENCY_7_CLOCKS	Read latency 7 clocks
QSPI_IP_HF_READ_LATENCY_8_CLOCKS	Read latency 8 clocks
QSPI_IP_HF_READ_LATENCY_9_CLOCKS	Read latency 9 clocks
QSPI_IP_HF_READ_LATENCY_10_CLOCKS	Read latency 10 clocks
QSPI_IP_HF_READ_LATENCY_11_CLOCKS	Read latency 11 clocks
QSPI_IP_HF_READ_LATENCY_12_CLOCKS	Read latency 12 clocks
QSPI_IP_HF_READ_LATENCY_13_CLOCKS	Read latency 13 clocks
QSPI_IP_HF_READ_LATENCY_14_CLOCKS	Read latency 14 clocks
QSPI_IP_HF_READ_LATENCY_15_CLOCKS	Read latency 15 clocks
QSPI_IP_HF_READ_LATENCY_16_CLOCKS	Read latency 16 clocks

Definition at line 161 of file Qspi_Ip_HyperflashTypes.h.

6.2.5.4 Qspi_Ip_HyperflashAsoEntryCommandsType

enum Qspi_Ip_HyperflashAsoEntryCommandsType

Enumerator

QSPI_IP_HF_PASSWORD_ASO_ENTRY	Password ASO Entry command
QSPI_IP_HF_PPB_ASO_ENTRY	PPB ASO Entry command
QSPI_IP_HF_PPB_LOCK_ASO_ENTRY	PPB Lock ASO Entry command
QSPI_IP_HF_DYB_ASO_ENTRY	DYB ASO Entry command
QSPI_IP_HF_ECC_ASO_ENTRY	ECC ASO Entry command
QSPI_IP_HF_SSR_ASO_ENTRY	Secure Silicon Region command

Enumerator

QSPI_IP_HF_CRC_ASO_ENTRY	CRC ASO Entry command
QSPI_IP_HF_ASPR_ASO_ENTRY	ASP Configuration Register ASO entry command
QSPI_IP_HF_FLASH_MEMORY_ARRAY	No ASO entry

Definition at line 179 of file Qspi_Ip_HyperflashTypes.h.

6.2.5.5 Qspi_Ip_HyperflashSectorProtectionType

enum Qspi_Ip_HyperflashSectorProtectionType

Sector protection type.

Definition at line 197 of file Qspi_Ip_HyperflashTypes.h.

6.2.5.6 Qspi_Ip_StatusType

enum Qspi_Ip_StatusType

qspi return codes

Enumerator

STATUS_QSPI_IP_SUCCESS	Successful job
STATUS_QSPI_IP_ERROR	IP is performing an operation
STATUS_QSPI_IP_BUSY	Error - general code
STATUS_QSPI_IP_TIMEOUT	Error - exceeded timeout
STATUS_QSPI_IP_ERROR_PROGRAM_VERIFY	Error - selected memory area doesn't contain desired value

Definition at line 157 of file Qspi_Ip_Types.h.

6.2.5.7 Qspi_Ip_ConnectionType

enum Qspi_Ip_ConnectionType

flash connection to the QSPI module

Enumerator

QSPI_IP_SIDE_A1	Serial flash connected on side A1
QSPI_IP_SIDE_A2	Serial flash connected on side A2
QSPI_IP_SIDE_B1	Serial flash connected on side B1
QSPI_IP_SIDE_B2	Serial flash connected on side B2

Definition at line 169 of file Qspi_Ip_Types.h.

6.2.5.8 Qspi_Ip_OpType

enum Qspi_Ip_OpType

flash operation type

Enumerator

QSPI_IP_OP_TYPE_CMD	Simple command
QSPI_IP_OP_TYPE_WRITE_REG	Write value in external flash register
QSPI_IP_OP_TYPE_RMW_REG	RMW command on external flash register
QSPI_IP_OP_TYPE_READ_REG	Read external flash register until expected value is read
QSPI_IP_OP_TYPE_QSPI_CFG	Re-configure QSPI controller

Definition at line 180 of file Qspi_Ip_Types.h.

6.2.5.9 Qspi_Ip_LutCommandsType

enum Qspi_Ip_LutCommandsType

Lut commands.

Enumerator

QSPI_IP_LUT_INSTR_STOP	End of sequence

Enumerator

QSPI_IP_LUT_INSTR_CMD	Command
QSPI_IP_LUT_INSTR_ADDR	Address
QSPI_IP_LUT_INSTR_DUMMY	Dummy cycles
QSPI_IP_LUT_INSTR_MODE	8-bit mode
QSPI_IP_LUT_INSTR_MODE2	2-bit mode
QSPI_IP_LUT_INSTR_MODE4	4-bit mode
QSPI_IP_LUT_INSTR_READ	Read data
QSPI_IP_LUT_INSTR_WRITE	Write data
QSPI_IP_LUT_INSTR_JMP_ON_CS	Jump on chip select deassert and stop
QSPI_IP_LUT_INSTR_ADDR_DDR	Address - DDR mode
QSPI_IP_LUT_INSTR_MODE_DDR	8-bit mode - DDR mode
QSPI_IP_LUT_INSTR_MODE2_DDR	2-bit mode - DDR mode
QSPI_IP_LUT_INSTR_MODE4_DDR	4-bit mode - DDR mode
QSPI_IP_LUT_INSTR_READ_DDR	Read data - DDR mode
QSPI_IP_LUT_INSTR_WRITE_DDR	Write data - DDR mode
QSPI_IP_LUT_INSTR_DATA_LEARN	Data learning pattern
QSPI_IP_LUT_INSTR_CMD_DDR	Command - DDR mode
QSPI_IP_LUT_INSTR_CADDR	Column address
QSPI_IP_LUT_INSTR_CADDR_DDR	Column address - DDR mode
QSPI_IP_LUT_INSTR_JMP_TO_SEQ	Jump on chip select deassert and continue

Definition at line 191 of file Qspi_Ip_Types.h.

6.2.5.10 Qspi_Ip_LutPadsType

```
enum Qspi_Ip_LutPadsType
```

Lut pad options.

Enumerator

QSPI_IP_LUT_PADS↩_1	1 Pad
QSPI_IP_LUT_PADS↩_2	2 Pads
QSPI_IP_LUT_PADS↩_4	4 Pads
QSPI_IP_LUT_PADS↩_8	8 Pads

Definition at line 218 of file Qspi_Ip_Types.h.

6.2.5.11 Qspi_Ip_ReadModeType

enum Qspi_Ip_ReadModeType

Read mode.

Enumerator

QSPI_IP_READ_MODE_LOOPBACK	Use loopback clock from PAD as strobe signal
QSPI_IP_READ_MODE_EXTERNAL_DQS	Use external strobe signal

Definition at line 239 of file Qspi_Ip_Types.h.

6.2.5.12 Qspi_Ip_DataRateType

enum Qspi_Ip_DataRateType

Clock phase used for sampling Rx data.

Enumerator

QSPI_IP_DATA_RATE_SDR	Single data rate
QSPI_IP_DATA_RATE_DDR	Double data rate

Definition at line 256 of file Qspi_Ip_Types.h.

6.2.5.13 Qspi_Ip_SampleDelayType

enum Qspi_Ip_SampleDelayType

Delay used for sampling Rx data.

Enumerator

QSPI_IP_SAMPLE_DELAY_SAME_DQS	Same DQS
QSPI_IP_SAMPLE_DELAY_HALFCYCLE_EARLY_DQS	Half-cycle early DQS

Definition at line 265 of file Qspi_Ip_Types.h.

6.2.5.14 Qspi_Ip_SamplePhaseType

enum Qspi_Ip_SamplePhaseType

Clock phase used for sampling Rx data.

Enumerator

QSPI_IP_SAMPLE_PHASE_NON_INVERTED	Sampling at non-inverted clock
QSPI_IP_SAMPLE_PHASE_INVERTED	Sampling at inverted clock

Definition at line 273 of file Qspi_Ip_Types.h.

6.2.5.15 Qspi_Ip_FlashDataAlignType

enum Qspi_Ip_FlashDataAlignType

Alignment of outgoing data with serial clock.

Enumerator

QSPI_IP_FLASH_DATA_ALIGN_REFCLK	Data aligned with the posedge of Internal reference clock of QSPI
QSPI_IP_FLASH_DATA_ALIGN_2X_REFCLK	Data aligned with 2x serial flash half clock

Definition at line 281 of file Qspi_Ip_Types.h.

6.2.5.16 Qspi_Ip_DllModeType

enum Qspi_Ip_DllModeType

DLL configuration modes.

Enumerator

QSPI_IP_DLL_BYPASSED	DLL bypass mode
QSPI_IP_DLL_MANUAL_UPDATE	DLL manual update mode
QSPI_IP_DLL_AUTO_UPDATE	DLL auto update mode

Definition at line 289 of file Qspi_Ip_Types.h.

6.2.5.17 Qspi_Ip_LastCommandType

enum Qspi_Ip_LastCommandType

Last command that was executed by the device flash.

Definition at line 502 of file Qspi_Ip_Types.h.

6.2.5.18 Qspi_Ip_FlashMemoryType

enum Qspi_Ip_FlashMemoryType

Parameter memory type.

Enumerator

QSPI_IP_HYPER_FLASH	Hyperbus devices
QSPI_IP_SERIAL_FLASH	Traditional xSPI devices

Definition at line 563 of file Qspi_Ip_Types.h.

6.2.6 Function Reference

6.2.6.1 Qspi_Ip_Init()

```
Qspi_Ip_StatusType Qspi_Ip_Init (
            uint32 instance,
            const Qspi_Ip_MemoryConfigType * pConfig,
            const Qspi_Ip_MemoryConnectionType * pConnect )
```

Initializes the serial flash memory driver.

This function initializes the external flash driver and prepares it for operation.

Parameters

instance	External flash instance number
pConfig	Pointer to the driver configuration structure.
pConnect	Pointer to the flash device connection structure.

Returns

Error or success status returned by API

6.2.6.2 Qspi_Ip_Deinit()

```
Qspi_Ip_StatusType Qspi_Ip_Deinit (
            uint32 instance )
```

De-initializes the serial flash memory driver.

This function de-initializes the qspi driver. The driver can't be used again until reinitialized. The state structure is no longer needed by the driver and may be freed after calling this function.

Parameters

instance	External flash instance number

Returns

Error or success status returned by API

6.2.6.3 Qspi_Ip_EraseBlock()

```
Qspi_Ip_StatusType Qspi_Ip_EraseBlock (
            uint32 instance,
            uint32 address,
            uint32 size )
```

Erase a sector in the serial flash.

This function performs one erase sector (block) operation on the external flash. The erase size must match one of the device's erase types.

Parameters

instance	External flash instance number
address	Address of sector to be erased
size	Size of the sector to be erase. The sector size must match one of the supported erase sizes of the device.

Returns

Error or success status returned by API

6.2.6.4 Qspi_Ip_EraseChip()

```
Qspi_Ip_StatusType Qspi_Ip_EraseChip (
            uint32 instance )
```

Erase the entire serial flash.

Parameters

instance	External flash instance number

Returns

Error or success status returned by API

6.2.6.5 Qspi_Ip_GetMemoryStatus()

```
Qspi_Ip_StatusType Qspi_Ip_GetMemoryStatus (
            uint32 instance )
```

Check the status of the flash device.

Parameters

instance	External flash instance number

Returns

Error or success status returned by API

6.2.6.6 Qspi_Ip_SetProtection()

```
Qspi_Ip_StatusType Qspi_Ip_SetProtection (
            uint32 instance,
            uint8 value )
```

Sets the protection bits to the requested value.

Parameters

instance	External flash instance number
value	New value for the protection bits

Returns

Error or success status returned by API

6.2.6.7 Qspi_Ip_GetProtection()

```
Qspi_Ip_StatusType Qspi_Ip_GetProtection (
            uint32 instance,
            uint8 * value )
```

Returns the current value of the protection bits.

Parameters

instance	External flash instance number
value	Current value of the protection bits

Returns

Error or success status returned by API

6.2.6.8 Qspi_Ip_Reset()

```
Qspi_Ip_StatusType Qspi_Ip_Reset (
            uint32 instance )
```

Resets the flash device.

Parameters

instance	External flash instance number

Returns

Error or success status returned by API

6.2.6.9 Qspi_Ip_Enter0XX()

```
Qspi_Ip_StatusType Qspi_Ip_Enter0XX (
            uint32 instance )
```

Enters 0-X-X (no command) mode. This mode assumes only reads are performed.

Parameters

instance	External flash instance number

Returns

Error or success status returned by API

6.2.6.10 Qspi_Ip_Exit0XX()

```
Qspi_Ip_StatusType Qspi_Ip_Exit0XX (
            uint32 instance )
```

Exits 0-X-X (no command) mode. This allows operations other than reads to be performed.

Parameters

instance	External flash instance number

Returns

Error or success status returned by API

6.2.6.11 Qspi_Ip_ProgramSuspend()

```
Qspi_Ip_StatusType Qspi_Ip_ProgramSuspend (
            uint32 instance )
```

Suspends a program operation.

Parameters

instance	External flash instance number

Returns

Error or success status returned by API

6.2.6.12 Qspi_Ip_ProgramResume()

```
Qspi_Ip_StatusType Qspi_Ip_ProgramResume (
            uint32 instance )
```

Resumes a program operation.

Parameters

instance	External flash instance number

Returns

Error or success status returned by API

6.2.6.13 Qspi_Ip_EraseSuspend()

```
Qspi_Ip_StatusType Qspi_Ip_EraseSuspend (
            uint32 instance )
```

Suspends an erase operation.

Parameters

instance	External flash instance number

Returns

Error or success status returned by API

6.2.6.14 Qspi_Ip_EraseResume()

```
Qspi_Ip_StatusType Qspi_Ip_EraseResume (
            uint32 instance )
```

Resumes an erase operation.

Parameters

instance	External flash instance number

Returns

Error or success status returned by API

6.2.6.15 Qspi_Ip_Read()

```
Qspi_Ip_StatusType Qspi_Ip_Read (
            uint32 instance,
            uint32 address,
            uint8 * data,
            uint32 size )
```

Read data from serial flash.

Parameters

instance	External flash instance number
address	Start address for read operation
data	Buffer where to store read data
size	Size of data buffer

Returns

Error or success status returned by API

6.2.6.16 Qspi_Ip_ReadId()

```
Qspi_Ip_StatusType Qspi_Ip_ReadId (
            uint32 instance,
            uint8 * data )
```

Read manufacturer ID/device ID from serial flash.

Parameters

instance	External flash instance number
data	Buffer where to store read data. Buffer size must match ReadId initialization settings.

Returns

Error or success status returned by API

6.2.6.17 Qspi_Ip_ProgramVerify()

```
Qspi_Ip_StatusType Qspi_Ip_ProgramVerify (
            uint32 instance,
            uint32 address,
            const uint8 * data,
            uint32 size )
```

Verifies the correctness of the programmed data.

Parameters

instance	External flash instance number
address	Start address of area to be verified
data	Data to be verified
size	Size of area to be verified

Returns

Error or success status returned by API

6.2.6.18 Qspi_Ip_EraseVerify()

```
Qspi_Ip_StatusType Qspi_Ip_EraseVerify (
            uint32 instance,
            uint32 address,
            uint32 size )
```

Checks whether or not an area in the serial flash is erased.

Parameters

instance	External flash instance number
address	Start address of area to be verified
size	Size of area to be verified

Returns

Error or success status returned by API

6.2.6.19 Qspi_Ip_Program()

```
Qspi_Ip_StatusType Qspi_Ip_Program (
            uint32 instance,
            uint32 address,
            const uint8 * data,
            uint32 size )
```

Writes data in serial flash.

Writes data in serial flash memory then exits (Async mode) The status of the flash memory must be verified by calling asynchronously the Qspi_Ip_GetMemoryStatus function until it is not busy, meaning that the write operation is complete. The maximum supported size is equal to the Qspi hardware TxBuffer size.

Parameters

instance	External flash instance number
address	Start address of area to be programmed
data	Data to be programmed in flash
size	Size of data buffer

Returns

Error or success status returned by API

6.2.6.20 Qspi_Ip_RunCommand()

```
Qspi_Ip_StatusType Qspi_Ip_RunCommand (
            uint32 instance,
            uint16 lut,
            uint32 addr )
```

Launches a simple command for the serial flash.

Parameters

instance	External flash instance number
lut	Index of command in virtual LUT
addr	Address used in the command, or base address of the target serial flash

Returns

Error or success status returned by API

6.2.6.21 Qspi_Ip_RunReadCommand()

```
Qspi_Ip_StatusType Qspi_Ip_RunReadCommand (
            uint32 instance,
            uint16 lut,
            uint32 addr,
            uint8 * dataRead,
            const uint8 * dataCmp,
            uint32 size )
```

Launches a read command for the serial flash.

This function can launch a read command in 3 modes:

- normal read (dataRead != NULL_PTR): Data is read from serial flash and placed in the buffer
- verify (dataRead == NULL_PTR, dataCmp != NULL_PTR): Data is read from serial flash and compared to the reference buffer
- blank check (dataRead == NULL_PTR, dataCmp == NULL_PTR): Data is read from serial flash and compared to 0xFF

Parameters

instance	External flash instance number
lut	Index of command in virtual LUT
addr	Start address for read operation in serial flash
dataRead	Buffer where to store read data
dataCmp	Buffer to be compared to read data
size	Size of data buffer

Returns

Error or success status returned by API

6.2.6.22 Qspi_Ip_RunWriteCommand()

```
Qspi_Ip_StatusType Qspi_Ip_RunWriteCommand (
            uint32 instance,
            uint16 lut,
            uint32 addr,
            const uint8 * data,
            uint32 size )
```

Launches a write command for the serial flash.

Parameters

instance	External flash instance number
lut	Index of command in virtual LUT
addr	Start address for write operation in serial flash
data	Data to be programmed in flash
size	Size of data buffer

Returns

Error or success status returned by API

6.2.6.23 Qspi_Ip_AhbReadEnable()

```
Qspi_Ip_StatusType Qspi_Ip_AhbReadEnable (
            uint32 instance )
```

Sets up AHB reads to the serial flash.

Parameters

instance	External flash instance number

Returns

Error or success status returned by API

6.2.6.24 Qspi_Ip_ControllerGetStatus()

```
Qspi_Ip_StatusType Qspi_Ip_ControllerGetStatus (
            uint32 instance )
```

Check the status of the QSPI controller.

Parameters

instance	QSPI peripheral instance number

Returns

Error or success status returned by API

6.2.6.25 Qspi_Ip_ControllerInit()

```
Qspi_Ip_StatusType Qspi_Ip_ControllerInit (
            uint32 instance,
            const Qspi_Ip_ControllerConfigType * userConfigPtr )
```

Initializes the qspi driver.

This function initializes the qspi driver and prepares it for operation.

Parameters

instance	QSPI peripheral instance number
userConfigPtr	Pointer to the qspi configuration structure.

Returns

Error or success status returned by API

6.2.6.26 Qspi_Ip_ControllerDeinit()

```
Qspi_Ip_StatusType Qspi_Ip_ControllerDeinit (
            uint32 instance )
```

De-initialize the qspi driver.

This function de-initializes the qspi driver. The driver can't be used again until reinitialized. The context structure is no longer needed by the driver and can be freed after calling this function.

Parameters

instance	QSPI peripheral instance number

Returns

Error or success status returned by API

6.2.6.27 Qspi_Ip_Abort()

```
Qspi_Ip_StatusType Qspi_Ip_Abort (
            uint32 instance )
```

Aborts any on-going transactions.

Force the Qspi controller to cancel the on-going IP transaction by performing the software reset sequence.

Parameters

instance	QSPI peripheral instance number

Returns

Error or success status returned by API

6.2.6.28 Qspi_Ip_ReadSfdp()

```
Qspi_Ip_StatusType Qspi_Ip_ReadSfdp (
            Qspi_Ip_MemoryConfigType * pConfig,
            const Qspi_Ip_MemoryConnectionType * pConnect )
```

Initializes the serial flash memory configuration from SFDP table.

This function uses the information in the SFDP table to auto-fill the memory configuration structure.

Parameters

pConfig	Pointer to the driver configuration structure.
pConnect	Pointer to the flash device connection structure.

Returns

Error or success status returned by API

6.2.6.29 Qspi_Ip_SetLut()

```
void Qspi_Ip_SetLut (
            uint32 instance,
            uint8 LutRegister,
            Qspi_Ip_InstrOpType operation0,
            Qspi_Ip_InstrOpType operation1 )
```

Configures LUT commands.

This function configures a pair of LUT commands in the specified LUT register. LUT sequences start at index multiple of 4 and can have up to 8 commands

Parameters

instance	QuadSPI peripheral instance number
LutRegister	Index in physical LUT array
operation0	First operation
operation1	Second operation Implements Qspi_Ip_SetLut_Activity

6.2.6.30 Qspi_Ip_WriteLuts_Privileged()

```
void Qspi_Ip_WriteLuts_Privileged (
            uint32 Instance,
```

```
            uint8 StartLutRegister,
            const uint32 * Data,
            uint8 Size )
```

Configures LUT commands.

This function configures pairs of LUT commands from the specified LUT register.

Parameters

Instance	QuadSPI peripheral instance number
StartLutRegister	Start index in physical LUT array
Data	Data to be written in LUT register
Size	Size of data buffer

6.2.6.31 Qspi_Ip_SetAhbSeqId_Privileged()

```
void Qspi_Ip_SetAhbSeqId_Privileged (
            uint32 instance,
            uint8 seqID )
```

Sets sequence ID for AHB operations.

Parameters

instance	QuadSPI peripheral instance number
seqID	Sequence ID in LUT for read operation Implements Qspi_Ip_SetAhbSeqId_Activity

6.2.6.32 Qspi_Ip_GetBaseAdress()

```
uint32 Qspi_Ip_GetBaseAdress (
            uint32 instance,
            Qspi_Ip_ConnectionType connectionType )
```

Returns the physical base address of a flash device.

This function returns the physical base address of a flash device, depending on the QSPI connection. The controller must be initialized prior to calling this function.

Parameters

instance	QuadSPI peripheral instance number
connectionType	Connection of the flash device to QSPI

6.2.6.33 Qspi_Ip_IpCommand()

```
Qspi_Ip_StatusType Qspi_Ip_IpCommand (
            uint32 instance,
            uint8 SeqId,
            uint32 addr )
```

Launches a simple IP command.

Parameters

instance	QuadSPI peripheral instance number
SeqId	Sequence ID where the command is to be found
addr	Address of the target serial flash

Returns

Error or success status returned by API

6.2.6.34 Qspi_Ip_IpRead()

```
Qspi_Ip_StatusType Qspi_Ip_IpRead (
            uint32 instance,
            uint8 SeqId,
            uint32 addr,
            uint8 * dataRead,
            const uint8 * dataCmp,
            uint32 size )
```

Launches an IP read command.

This function can launch a read command in 3 modes:

- normal read (dataRead != NULL_PTR): Data is read from serial flash and placed in the buffer
- verify (dataRead == NULL_PTR, dataCmp != NULL_PTR): Data is read from serial flash and compared to the reference buffer
- blank check (dataRead == NULL_PTR, dataCmp == NULL_PTR): Data is read from serial flash and compared to 0xFF Only normal read mode can use DMA.

Parameters

instance	QuadSPI peripheral instance number
SeqId	Sequence ID where the command is to be found
addr	Start address for read operation in serial flash
dataRead	Buffer where to store read data
dataCmp	Buffer to be compared to read data
size	Size of data buffer

Returns

Error or success status returned by API

6.2.6.35 Qspi_Ip_IpWrite()

```
Qspi_Ip_StatusType Qspi_Ip_IpWrite (
            uint32 instance,
            uint8 SeqId,
            uint32 addr,
            const uint8 * data,
            uint32 size )
```

Launches an IP write command.

Parameters

instance	QuadSPI peripheral instance number
SeqId	Sequence ID where the command is to be found
addr	Start address for write operation in serial flash
data	Data to be programmed in flash
size	Size of data buffer

Returns

Error or success status returned by API

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.